```
using Abelkhan;
using
Microsoft.AspNetCore.DataProtection.KeyManagement;
using Microsoft.AspNetCore.Http;
using Service;
using System;
using System.Threading;
using System.Xml.Linq;

namespace center_svr
{
    class Program
    {
        public static RedisHandle _redis_handle;

        static void Main(string[] args)
        {
            var _center = new
Abelkhan.Center(args[0], args[1]);
            _center.on_svr_disconnect +=
(Abelkhan.SvrProxy _proxy) =>
            {
                Log.Log.err("svr:{0},{1} exception!",
_proxy.type, _proxy.name);
            };

            if (!
_center._root_cfg.has_key("redis_for_mq_pwd"))
            {
                _redis_handle = new
RedisHandle(_center._root_cfg.get_value_string("redis
_for_cache"), string.Empty);
            }
            else
            {
                _redis_handle = new
RedisHandle(_center._root_cfg.get_value_string("redis
_for_cache"),
_center._root_cfg.get_value_string("redis_for_mq_pwd"
));
            }
```

```
            if
(_center._config.get_value_bool("init_peak_strength_i
d"))
            {
                var peak_strength_id =
RandomHelper.RandomInt(100);
                _redis_handle.SetData(RedisHelp.BuildPeakStrengthID()
, peak_strength_id);
            }

            _center._timer.addloopweekdaytime(System.DayOfWeek.Mo
nday, 0, 0, 0, reset_peak_strength);

            Log.Log.trace("Center start ok");

            _center.run();
        }

        private static async void
reset_peak_strength(System.DateTime _)
        {
            try
            {
                var key =
RedisHelp.BuildPeakStrengthID();
                var peak_strength_id = await
_redis_handle.GetData<long>(key);
                await _redis_handle.SetData(key, +
+peak_strength_id);
            }
            catch (System.Exception ex)
            {
                Log.Log.err("reset_peak_strength:
{0}", ex);
            }
        }
    }
}
```

```
using System;
using System.Threading;

namespace dbproxy_svr
{
    class Program
    {
        static void Main(string[] args)
        {
            var _dbproxy = new
DBProxy.DBProxy(args[0], args[1]);

            Log.Log.trace("dbproxy start ok");

            _dbproxy.run().Wait();
        }
    }
}
```

```
using System;
using System.Threading;
using static
System.Runtime.InteropServices.JavaScript.JSType;

namespace gate_svr
{
    class Program
    {
        static void Main(string[] args)
        {
            var _gate = new Gate.GateService(args[0],
args[1]);

            Log.Log.trace("gate_svr start ok");

            _gate.run().Wait();
        }
    }
}
```

```
using Abelkhan;

namespace Login
{
    class client_msg_handle
    {
        private login_module login_Module = new login_module();

        public client_msg_handle()
        {
            login_Module.on_player_login_no_token += Login_Module_on_player_login_no_token;
            login_Module.on_player_login_dy += Login_Module_on_player_login_dy;
            login_Module.on_player_login_wx += Login_Module_on_player_login_wx;
        }

        private void try_player_login(player_proxy _proxy, string account, login_player_login_no_token_rsp rsp)
        {
            Log.Log.trace("try_player_login _proxy.name:{0}", _proxy.name);

            var key = RedisHelp.BuildPlayerSvrCacheKey(account);
            Login._redis_handle.SetStrData(key, _proxy.name, RedisHelp.PlayerSvrInfoCacheTimeout);

            _proxy.player_login_no_token(account).callBack((token, is_new) => {
                rsp.rsp(_proxy.name, token, is_new);
            }, (err) => {
                rsp.err(err);
            }).timeout(20000, () => {
                Log.Log.trace("try_player_login _proxy.name:{0} timeout!", _proxy.name);
                rsp.err((int)em_error.timeout);
```

```
            });
        }

        private async void
random_player_svr_rsp(string account,
login_player_login_no_token_rsp rsp)
        {
            var _proxy = await
Login._player_proxy_mng.random_idle_player();
            if (_proxy != null)
            {
                try_player_login(_proxy, account,
rsp);
            }
            else
            {
                rsp.err((int)em_error.server_busy);
            }
        }

        private async void
Login_Module_on_player_login_no_token(string account)
        {
            var uuid =
Hub.Hub._gates.current_client_uuid;
            var rsp = login_Module.rsp as
login_player_login_no_token_rsp;

            Log.Log.trace("on_player_login_no_author
begin! player account:{0}, uuid:{1}", account, uuid);

            var lock_key =
RedisHelp.BuildPlayerSvrCacheLockKey(account);
            var token = $"lock_{account}";
            try
            {
                await
Login._redis_handle.Lock(lock_key, token, 1000);

                var key =
RedisHelp.BuildPlayerSvrCacheKey(account);
```

```
                var _player_proxy_name = await
Login._redis_handle.GetStrData(key);
                if
(string.IsNullOrEmpty(_player_proxy_name))
                {
                    random_player_svr_rsp(account,
rsp);
                }
                else
                {
                    var _proxy =
Login._player_proxy_mng.get_player(_player_proxy_name
);
                    if (_proxy != null)
                    {
                        await
Login._redis_handle.Expire(key,
RedisHelp.PlayerSvrInfoCacheTimeout);
                        try_player_login(_proxy,
account, rsp);
                    }
                    else
                    {
                        random_player_svr_rsp(account, rsp);
                    }
                }

                var gate_key =
RedisHelp.BuildPlayerGateCacheKey(account);
                await
Login._redis_handle.SetStrData(gate_key,
Hub.Hub._gates.get_client_gate_name(uuid),
RedisHelp.PlayerSvrInfoCacheTimeout);
            }
            catch (System.Exception ex)
            {
                Log.Log.err($"{ex}");
                rsp.err((int)em_error.db_error);
            }
            finally
```

```
            {
                await
Login._redis_handle.UnLock(lock_key, token);
            }
        }

        private void try_player_login(player_proxy
_proxy, string account, string code, string
anonymous_code, login_player_login_dy_rsp rsp)
        {
            Log.Log.trace("try_player_login
_proxy.name:{0}", _proxy.name);

            var key =
RedisHelp.BuildPlayerSvrCacheKey(account);
            Login._redis_handle.SetStrData(key,
_proxy.name, RedisHelp.PlayerSvrInfoCacheTimeout);

            _proxy.player_login(code,
anonymous_code).callBack((token, is_new) => {
                rsp.rsp(_proxy.name, token, is_new);
            }, (err) => {
                rsp.err(err);
            }).timeout(10000, () => {
                rsp.err((int)em_error.timeout);
            });
        }

        private async void
random_player_svr_rsp(string account, string code,
string anonymous_code, login_player_login_dy_rsp rsp)
        {
            var _proxy = await
Login._player_proxy_mng.random_idle_player();
            if (_proxy != null)
            {
                try_player_login(_proxy, account,
code, anonymous_code, rsp);
            }
            else
            {
```

```
                rsp.err((int)em_error.server_busy);
            }
        }

        private async void
Login_Module_on_player_login_dy(string code, string
anonymous_code)
        {
            var uuid =
Hub.Hub._gates.current_client_uuid;
            var rsp = login_Module.rsp as
login_player_login_dy_rsp;

            try
            {
                var session = await
dy.dySdk.code2Session(Login.DyAppID,
Login.DyAppSecret, code, anonymous_code);

                string account = null;
                if
(string.IsNullOrEmpty(anonymous_code))
                {
                    account = session.openid;
                }
                else
                {
                    account =
session.anonymous_openid;
                }

                Log.Log.trace("on_player_login_dy
begin! player account:{0}, uuid:{1}", account, uuid);
                var lock_key =
RedisHelp.BuildPlayerSvrCacheLockKey(account);
                var token = $"lock_{account}";
                try
                {
                    await
Login._redis_handle.Lock(lock_key, token, 1000);
```

```
                    var key =
RedisHelp.BuildPlayerSvrCacheKey(account);
                    var _player_proxy_name = await
Login._redis_handle.GetStrData(key);
                    if
(string.IsNullOrEmpty(_player_proxy_name))
                    {

random_player_svr_rsp(account, session.openid,
session.anonymous_openid, rsp);
                    }
                    else
                    {
                        var _proxy =
Login._player_proxy_mng.get_player(_player_proxy_name
);
                        if (_proxy != null)
                        {
                            await
Login._redis_handle.Expire(key,
RedisHelp.PlayerSvrInfoCacheTimeout);
                            try_player_login(_proxy,
account, session.openid, session.anonymous_openid,
rsp);
                        }
                        else
                        {

random_player_svr_rsp(account, session.openid,
session.anonymous_openid, rsp);
                        }
                    }

                    var gate_key =
RedisHelp.BuildPlayerGateCacheKey(account);
                    await
Login._redis_handle.SetStrData(gate_key,
Hub.Hub._gates.get_client_gate_name(uuid),
RedisHelp.PlayerSvrInfoCacheTimeout);
                }
                catch (System.Exception ex)
```

```
                {
                    Log.Log.err($"{ex}");
                    rsp.err((int)em_error.db_error);
                }
                finally
                {
                    await
Login._redis_handle.UnLock(lock_key, token);
                }
            }
            catch (dy.code2SessionEx ex)
            {
                Log.Log.err($"code2SessionEx:{ex}");

rsp.err((int)em_error.login_dy_faild);
            }
            catch (System.Exception ex)
            {
                Log.Log.err($"{ex}");
                rsp.err((int)em_error.db_error);
            }
        }

        private void try_player_login(player_proxy
_proxy, string account, string code,
login_player_login_wx_rsp rsp)
        {
            Log.Log.trace("try_player_login
_proxy.name:{0}", _proxy.name);

            var key =
RedisHelp.BuildPlayerSvrCacheKey(account);
            Login._redis_handle.SetStrData(key,
_proxy.name, RedisHelp.PlayerSvrInfoCacheTimeout);

            _proxy.player_login(code,
"").callBack((token, is_new) => {
                rsp.rsp(_proxy.name, token, is_new);
            }, (err) => {
                rsp.err(err);
            }).timeout(10000, () => {
```

```
                rsp.err((int)em_error.timeout);
            });
        }

        private async void
random_player_svr_rsp(string account, string code,
login_player_login_wx_rsp rsp)
        {
            var _proxy = await
Login._player_proxy_mng.random_idle_player();
            if (_proxy != null)
            {
                try_player_login(_proxy, account,
code, rsp);
            }
            else
            {
                rsp.err((int)em_error.server_busy);
            }
        }

        private async void
Login_Module_on_player_login_wx(string code)
        {
            var uuid =
Hub.Hub._gates.current_client_uuid;
            var rsp = login_Module.rsp as
login_player_login_wx_rsp;

            try
            {
                var session = await
wx.wxSdk.code2Session(Login.WxAppID,
Login.WxAppSecret, code);
                string account = session.openid;

                Log.Log.trace("on_player_login_wx
begin! player account:{0}, uuid:{1}", account, uuid);
                var lock_key =
RedisHelp.BuildPlayerSvrCacheLockKey(account);
                var token = $"lock_{account}";
```

```
                try
                {
                    await
Login._redis_handle.Lock(lock_key, token, 1000);

                    var key =
RedisHelp.BuildPlayerSvrCacheKey(account);
                    var _player_proxy_name = await
Login._redis_handle.GetStrData(key);
                    if
(string.IsNullOrEmpty(_player_proxy_name))
                    {

random_player_svr_rsp(account, session.openid, rsp);
                    }
                    else
                    {
                        var _proxy =
Login._player_proxy_mng.get_player(_player_proxy_name
);
                        if (_proxy != null)
                        {
                            await
Login._redis_handle.Expire(key,
RedisHelp.PlayerSvrInfoCacheTimeout);
                            try_player_login(_proxy,
account, session.openid, rsp);
                        }
                        else
                        {

random_player_svr_rsp(account, session.openid, rsp);
                        }
                    }

                    var gate_key =
RedisHelp.BuildPlayerGateCacheKey(account);
                    await
Login._redis_handle.SetStrData(gate_key,
Hub.Hub._gates.get_client_gate_name(uuid),
RedisHelp.PlayerSvrInfoCacheTimeout);
```

```
                }
                catch (System.Exception ex)
                {
                    Log.Log.err($"{ex}");
                    rsp.err((int)em_error.db_error);
                }
                finally
                {
                    await
Login._redis_handle.UnLock(lock_key, token);
                }
            }
            catch (System.Exception ex)
            {
                Log.Log.err($"{ex}");
                rsp.err((int)em_error.db_error);
            }
        }
    }
}

using Abelkhan;
using System.Collections.Generic;
using System.Threading.Tasks;

namespace Login
{
    public class player_proxy
    {
        private readonly login_player_caller
_login_player_caller;
        private readonly Hub.HubProxy _proxy;

        public string name
        {
            get
            {
                return _proxy.name;
            }
        }
```

```
        public player_proxy(login_player_caller
caller, Hub.HubProxy proxy)
        {
            _login_player_caller = caller;
            _proxy = proxy;
        }

        public login_player_player_login_cb
player_login(string code, string anonymous_code)
        {
            return
_login_player_caller.get_hub(_proxy.name).player_logi
n(code, anonymous_code);
        }

        public login_player_player_login_no_token_cb
player_login_no_token(string account)
        {
            return
_login_player_caller.get_hub(_proxy.name).player_logi
n_no_token(account);
        }
    }

    public class player_proxy_mng
    {
        private readonly login_player_caller
_login_player_caller = new ();
        private readonly Dictionary<string,
player_proxy> player_proxys = new ();

        public player_proxy_mng()
        {
        }

        public void reg_player_proxy(Hub.HubProxy
_proxy)
        {
            player_proxys[_proxy.name] = new
player_proxy(_login_player_caller, _proxy);
        }
```

```
        public player_proxy get_player(string
player_hub_name)
        {
            player_proxys.TryGetValue(player_hub_name, out
player_proxy proxy);
            return proxy;
        }

        public async Task<player_proxy>
random_idle_player()
        {
            var idle_player_proxys = new
List<player_proxy>();
            foreach (var p in player_proxys)
            {
                var info = await
Login._redis_handle.GetData<svr_info>(RedisHelp.Build
PlayerSvrInfoCacheKey(p.Key));
                if (info?.tick_time < 100)
                {
                    idle_player_proxys.Add(p.Value);
                }
            }

            if (idle_player_proxys.Count > 0)
            {
                uint count =
(uint)idle_player_proxys.Count;
                return
idle_player_proxys[(int)Hub.Hub.randmon_uint(count)];
            }

            return null;
        }
    }
}

using Abelkhan;
```

```
namespace Login
{
    class Login
    {
        public static string DyAppID;
        public static string DyAppSecret;
        public static string WxAppID;
        public static string WxAppSecret;
        public static RedisHandle _redis_handle;
        public static player_proxy_mng
_player_proxy_mng = new ();

        static void Main(string[] args)
        {
            var _hub = new Hub.Hub(args[0], args[1],
"login", "fixed");

            if (!
Hub.Hub._root_config.has_key("redis_for_mq_pwd"))
            {
                _redis_handle = new
RedisHandle(Hub.Hub._root_config.get_value_string("re
dis_for_cache"), string.Empty);
            }
            else
            {
                _redis_handle = new
RedisHandle(Hub.Hub._root_config.get_value_string("re
dis_for_cache"),
Hub.Hub._root_config.get_value_string("redis_for_mq_p
wd"));
            }

            HttpClientWrapper.Init();

            DyAppID =
Hub.Hub._config.get_value_string("DyAppID");
            DyAppSecret =
Hub.Hub._config.get_value_string("DyAppSecret");
            WxAppID =
Hub.Hub._config.get_value_string("WxAppID");
```

```
            WxAppSecret =
Hub.Hub._config.get_value_string("WxAppSecret");

            var _client_msg_handle = new
client_msg_handle();

            _hub.on_hubproxy += on_hubproxy;
            _hub.on_hubproxy_reconn += on_hubproxy;

            _hub.onCloseServer += () => {
                _hub.closeSvr();
            };

            Log.Log.trace("login start ok");

            _hub.run().Wait();
        }

        private static void on_hubproxy(Hub.HubProxy
_proxy)
        {
            if (_proxy.type == "player")
            {

_player_proxy_mng.reg_player_proxy(_proxy);
            }
        }
    }
}

using Abelkhan;
using Amazon.Util.Internal;
using battle_shop;
using config;
using Service;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;

namespace Match
```

```
{
    public class battle_player
    {
        public long LastActiveTime =
Timerservice.Tick;

        public string PlayerHubName {
            set; get;
        }

        private battle_client_caller caller;
        public battle_client_caller
BattleClientCaller
        {
            get
            {
                return caller;
            }
        }

        private string clientUUID;
        public string ClientUUID
        {
            set
            {
                clientUUID = value;
            }
            get
            {
                return clientUUID;
            }
        }

        private battle_shop_player
battle_Shop_Player;
        public battle_shop_player BattleShopPlayer
        {
            set
            {
                battle_Shop_Player = value;
            }
```

```
            get
            {
                return battle_Shop_Player;
            }
        }

        public BattleMod mod;

        public battle_player(BattleMod _mod, string
_clientUUID, battle_client_caller _caller, List<int>
roleList, UserInformation info)
        {
            mod = _mod;
            clientUUID = _clientUUID;
            caller = _caller;

            battle_Shop_Player = new
battle_shop_player(_clientUUID, _caller, roleList,
info);
            refresh(baseStage());
        }

        public void battle_failed_back()
        {
            if (BattleShopPlayer.BattleData.faild <=
0)
            {
                BattleShopPlayer.BattleData.faild =
1;
            }

            BattleShopPlayer.BattleData.coin = 10;

            BattleShopPlayer.ShopData =
BattleShopPlayer.refresh(baseStage());
        }

        public int baseStage()
        {
            //var _base =
(BattleShopPlayer.BattleData.round + 1) / 2;
```

```
            var _base =
BattleShopPlayer.BattleData.round;
            return _base < 6 ? _base : 6;
        }

        private void _reset()
        {
            foreach (var r in
BattleShopPlayer.BattleData.RoleList)
            {
                if (r != null)
                {
                    r.TempHP = 0;
                    r.TempAttack = 0;
                    r.TempAdditionBuffer.Clear();
                }
            }
        }

        public void start_round(int stage, int
addCoin)
        {
            BattleShopPlayer.BattleData.coin = 10 +
BattleShopPlayer.bankCoin + addCoin;
            BattleShopPlayer.bankCoin = 0;
            BattleShopPlayer.Coin =
BattleShopPlayer.BattleData.coin;

            _reset();
            BattleShopPlayer.ShopData =
BattleShopPlayer.refresh(stage);

            BattleShopPlayer.evs.Add(new shop_event()
            {
                ev = EMRoleShopEvent.start_round
            });
        }

        public void end_round()
        {
            BattleShopPlayer.evs.Add(new shop_event()
```

```
            {
                ev = EMRoleShopEvent.end_round
            });
            BattleShopPlayer.do_skill(baseStage());
        }

        public void refresh(int stage)
        {
            BattleShopPlayer.ShopData =
BattleShopPlayer.refresh(stage);
        }

        private bool add_role(int role_index, int
shop_index)
        {
            var r =
BattleShopPlayer.add_role(role_index, shop_index);
            if (r != null)
            {
                var player_proxy =
Match._player_proxy_mng.get_player(PlayerHubName);

player_proxy.buy_role(BattleShopPlayer.BattleData.Use
r.UserGuid, r);

                return true;
            }

            return false;
        }

        private em_error buy_role(int index, int
role_index)
        {
            var r =
BattleShopPlayer.BattleData.RoleList[role_index];
            var s =
BattleShopPlayer.ShopData.SaleRoleList[index];

            if (s == null)
            {
```

```
                return em_error.db_error;
            }

            if (BattleShopPlayer.BattleData.coin <
s.Price)
            {
                return em_error.no_enough_coin;
            }
            BattleShopPlayer.BattleData.coin -=
s.Price;

            if (r == null)
            {
                if (!add_role(role_index, index))
                {
                    return em_error.db_error;
                }
            }
            else
            {
                var err =
BattleShopPlayer.merge_role(index, role_index);
                if (err != em_error.success)
                {
                    return err;
                }

                var player_proxy =
Match._player_proxy_mng.get_player(PlayerHubName);

player_proxy.buy_role(BattleShopPlayer.BattleData.Use
r.UserGuid, r);
            }

            return em_error.success;
        }

        private em_error buy_equip(ShopProp p, int
index, int role_index)
        {
            var err = BattleShopPlayer.buy_equip(p,
```

```
index, role_index);
            if (err != em_error.success)
            {
                return err;
            }

            var player_proxy =
Match._player_proxy_mng.get_player(PlayerHubName);
            player_proxy.buy_equip(BattleShopPlayer.BattleData.User.UserGuid, p.PropID);

            return em_error.success;
        }

        public em_error buy(ShopIndex shop_index, int index, int role_index)
        {
            if (shop_index == ShopIndex.Role)
            {
                var result = buy_role(index, role_index);
                if (result != em_error.success)
                {
                    return result;
                }
            }
            else if (shop_index == ShopIndex.Prop)
            {
                var p = BattleShopPlayer.ShopData.SalePropList[index];
                if (p == null)
                {
                    return em_error.db_error;
                }

                if (BattleShopPlayer.BattleData.coin < p.Price)
                {
                    return em_error.no_enough_coin;
                }
```

```
                BattleShopPlayer.BattleData.coin -=
p.Price;

                if (p.PropID >=
config.Config.FoodIDMin && p.PropID <=
config.Config.FoodIDMax)
                {
                    var result =
BattleShopPlayer.buy_food(p, index, role_index,
baseStage());
                    if (result != em_error.success)
                    {
                        return result;
                    }
                }
                else if (p.PropID >=
config.Config.EquipIDMin && p.PropID <=
config.Config.EquipIDMax)
                {
                    var result = buy_equip(p, index,
role_index);
                    if (result != em_error.success)
                    {
                        return result;
                    }
                }
                else
                {
                    return em_error.db_error;
                }

                BattleShopPlayer.ShopData.SalePropList[index] = null;
            }
            else
            {
                return em_error.db_error;
            }

            return em_error.success;
        }
```

```
    }

    public class battle_mng
    {
        private Dictionary<string, battle_player>
battles = new();

        private battle_client_caller _caller;

        public battle_mng()
        {
            _caller = new battle_client_caller();

            Hub.Hub._timer.addticktime(5 * 60 * 1000,
tick_clear_timeout_player);
        }

        private void tick_clear_timeout_player(long
tick_time)
        {
            List<battle_player> timeout_battle_player
= new();
            foreach (var it in battles)
            {
                if ((it.Value.LastActiveTime + 30 *
60 * 1000) < Timerservice.Tick)
                {
                    timeout_battle_player.Add(it.Value);
                }
            }
            foreach (var _player in
timeout_battle_player)
            {
                battles.Remove(_player.ClientUUID);
            }

            Hub.Hub._timer.addticktime(5 * 60 * 1000,
tick_clear_timeout_player);
        }
```

```
        public battle_player
add_player_to_battle(BattleMod mod, string
clientUUID, List<int> roleList, UserInformation
user_info)
        {
            var _player = new battle_player(mod,
clientUUID, _caller, roleList, user_info);
            battles[clientUUID] = _player;
            return _player;
        }

        public bool change_player_uuid(string
old_client_uuid, string new_client_uuid)
        {
            if (battles.Remove(old_client_uuid, out
var _player))
            {
                _player.ClientUUID = new_client_uuid;
                battles[new_client_uuid] = _player;
            }

            return true;
        }

        public battle_player get_battle_player(string
clientUUID)
        {
            battles.TryGetValue(clientUUID, out var
_player);
            _player.LastActiveTime =
Timerservice.Tick;
            return _player;
        }
    }
}
using Abelkhan;
using Amazon.Runtime.Internal.Util;
using Amazon.SecurityToken.Model;
using battle_shop;
using InfluxData.Net.Common.Helpers;
```

```
using InfluxData.Net.InfluxDb.Models.Responses;
using MsgPack;
using MsgPack.Serialization;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;

namespace Match
{
    class client_msg_handle
    {
        private plan_module plan_Module = new
plan_module();
        private battle_revive_module
battle_revive_Module = new battle_revive_module();
        private peak_strength_module
peak_Strength_Module = new peak_strength_module();
        private gm_module gm_Module = new
gm_module();

        public client_msg_handle()
        {
            plan_Module.on_buy += Plan_Module_on_buy;
            plan_Module.on_sale_role +=
Plan_Module_on_sale_role;
            plan_Module.on_move +=
Plan_Module_on_move;
            plan_Module.on_refresh +=
Plan_Module_on_refresh;
            plan_Module.on_start_round1 +=
Plan_Module_on_start_round1;
            plan_Module.on_confirm_round_victory +=
Plan_Module_on_confirm_round_victory;
            plan_Module.on_freeze +=
Plan_Module_on_freeze;
            plan_Module.on_start_round +=
Plan_Module_on_start_round;
            plan_Module.on_get_battle_data +=
Plan_Module_on_get_battle_data;
            plan_Module.on_end_round +=
```

```
Plan_Module_on_end_round;

            
battle_revive_Module.on_battle_failed_back +=
Battle_revive_Module_on_battle_failed_back;

            
peak_Strength_Module.on_start_peak_strength +=
Peak_Strength_Module_on_start_peak_strength;
            
peak_Strength_Module.on_confirm_peak_strength_victory
+=
Peak_Strength_Module_on_confirm_peak_strength_victory
;

            gm_Module.on_set_formation +=
Gm_Module_on_set_formation;
        }

        private void
Battle_revive_Module_on_battle_failed_back()
        {
            var rsp = battle_revive_Module.rsp as
battle_revive_battle_failed_back_rsp;
            var uuid =
Hub.Hub._gates.current_client_uuid;

            try
            {
                var _player =
Match.battle_Mng.get_battle_player(uuid);
                _player.battle_failed_back();
                
rsp.rsp(_player.BattleShopPlayer.BattleData,
_player.BattleShopPlayer.ShopData);
            }
            catch (System.Exception ex)
            {
                
Log.Log.err("Battle_revive_Module_on_battle_failed_ba
ck error:{0}", ex);
```

```
                rsp.err((int)em_error.db_error);
            }
        }

        private void Plan_Module_on_end_round()
        {
            var rsp = plan_Module.rsp as
plan_end_round_rsp;
            var uuid =
Hub.Hub._gates.current_client_uuid;

            try
            {
                var _player =
Match.battle_Mng.get_battle_player(uuid);
                _player.end_round();
                rsp.rsp();
            }
            catch (System.Exception ex)
            {
                Log.Log.err("Plan_Module_on_end_round
error:{0}", ex);
                rsp.err((int)em_error.db_error);
            }
        }

        private async void
Peak_Strength_Module_on_confirm_peak_strength_victory
(BattleVictory is_victory, int addCoin)
        {
            var rsp = peak_Strength_Module.rsp as
peak_strength_confirm_peak_strength_victory_rsp;
            var uuid =
Hub.Hub._gates.current_client_uuid;
            // to do

            try
            {
                var _player =
Match.peak_strength_mng.get_battle_player(uuid);
                var formation = await
```

```
Match._redis_handle.GetData<UserBattleData>(RedisHelp
.BuildPlayerPeakStrengthFormationCache(_player.GUID))
;
                var player_proxy =
Match._player_proxy_mng.get_player(_player.PlayerHubN
ame);

player_proxy.peak_strength_victory(is_victory,
formation).callBack(async (userRankInfo) =>
                {

Log.Log.trace("Peak_Strength_Module_on_confirm_peak_s
trength_victory peak_strength_victory begin!");

                    /*var battle_data = await
Match._redis_handle.GetData<UserBattleData>(RedisHelp
.BuildPlayerPeakStrengthFormationCache(_player.GUID))
;
                    userRankInfo.battle_data = new
UserSharedData()
                    {
                        User = battle_data.User,
                        round = battle_data.round,
                        victory =
battle_data.victory,
                        faild = 5 -
battle_data.faild,
                    };
                    using var st =
MemoryStreamPool.mstMgr.GetStream();
                    var _serializer =
MessagePackSerializer.Get<MessagePackObjectDictionary
>();
                    _serializer.Pack(st,
UserRankInfo.UserRankInfo_to_protcol(userRankInfo));

                    var r = new rank_item
                    {
                        guid = _player.GUID,
                        score = userRankInfo.score,
                        item = st.ToArray()
```

```
                    };
                    Match._rank_proxy.update_rank_item(r);*/
                }, (err) =>
                {
                    Log.Log.err("peak_strength_victory err:{0}", err);
                    rsp.err();
                }).timeout(1500, () =>
                {
                    Log.Log.err("peak_strength_victory timeout!");
                    rsp.err();
                });

                rsp.rsp();
            }
            catch (System.Exception ex)
            {
                Log.Log.err("Peak_Strength_Module_on_del_peak_strength_formation error:{0}", ex);
                rsp.err();
            }
        }

        private async void Peak_Strength_Module_on_start_peak_strength()
        {
            var rsp = peak_Strength_Module.rsp as peak_strength_start_peak_strength_rsp;
            var uuid = Hub.Hub._gates.current_client_uuid;
            // to do

            try
            {
                var _player = Match.peak_strength_mng.get_battle_player(uuid);
                var formation = await Match._redis_handle.GetData<UserBattleData>(RedisHelp
```

```
.BuildPlayerPeakStrengthFormationCache(_player.GUID))
;
                var target = await
Match._redis_handle.RandomList<UserBattleData>(RedisH
elp.BuildPeakStrengthCache());
                rsp.rsp(formation, target);
            }
            catch (System.Exception ex)
            {
                Log.Log.err("Peak_Strength_Module_on_start_peak_stren
gth error:{0}", ex);
                rsp.err((int)em_error.db_error);
            }
        }

        private void Plan_Module_on_get_battle_data()
        {
            var rsp = plan_Module.rsp as
plan_get_battle_data_rsp;
            var uuid =
Hub.Hub._gates.current_client_uuid;

            try
            {
                var _player =
Match.battle_Mng.get_battle_player(uuid);

                rsp.rsp(_player.BattleShopPlayer.BattleData,
_player.BattleShopPlayer.ShopData,
_player.BattleShopPlayer.check_fetters());
            }
            catch (System.Exception ex)
            {
                Log.Log.err("Plan_Module_on_get_battle_data error:
{0}", ex);
                rsp.err();
            }
        }
```

```
        private void Plan_Module_on_freeze(ShopIndex
shop_index, int index, bool is_freeze)
        {
            var rsp = plan_Module.rsp as
plan_freeze_rsp;
            var uuid =
Hub.Hub._gates.current_client_uuid;

            try
            {
                var _player =
Match.battle_Mng.get_battle_player(uuid);
                _player.BattleShopPlayer.freeze(shop_index, index,
is_freeze);

                rsp.rsp(_player.BattleShopPlayer.ShopData);
            }
            catch (System.Exception ex)
            {
                Log.Log.err("Plan_Module_on_freeze
error:{0}", ex);
                rsp.err((int)em_error.db_error);
            }
        }

        private void Plan_Module_on_move(int
role_index1, int role_index2)
        {
            var rsp = plan_Module.rsp as
plan_move_rsp;
            var uuid =
Hub.Hub._gates.current_client_uuid;

            try
            {
                var _player =
Match.battle_Mng.get_battle_player(uuid);
                
```

```
_player.BattleShopPlayer.move(role_index1,
role_index2);

rsp.rsp(_player.BattleShopPlayer.BattleData);
            }
            catch (System.Exception ex)
            {
                Log.Log.err("Plan_Module_on_move
error:{0}", ex);
                rsp.err((int)em_error.db_error);
            }
        }

        private async void
Plan_Module_on_confirm_round_victory(BattleVictory
is_victory, int addCoin, List<Role> roles)
        {
            var rsp = plan_Module.rsp as
plan_confirm_round_victory_rsp;
            var uuid =
Hub.Hub._gates.current_client_uuid;

            try
            {
                var _player =
Match.battle_Mng.get_battle_player(uuid);

                var len = await
Match._redis_handle.PushList(RedisHelp.BuildAutoChess
BattleCache(_player.BattleShopPlayer.BattleData.round
), _player.BattleShopPlayer.BattleData);
                if (len > 1100)
                {
                    var
Match._redis_handle.PopList(RedisHelp.BuildAutoChessB
attleCache(_player.BattleShopPlayer.BattleData.round)
, 100);
                }

```

```
_player.BattleShopPlayer.lastBattleResults =
is_victory;

_player.BattleShopPlayer.BattleData.stage =
(_player.BattleShopPlayer.BattleData.round + 1) / 2;

                if (is_victory ==
BattleVictory.victory)
                {

_player.BattleShopPlayer.BattleData.victory++;
                }
                else if (is_victory ==
BattleVictory.faild)
                {
                    if
(_player.BattleShopPlayer.BattleData.faild > 0)
                    {

_player.BattleShopPlayer.BattleData.faild--;
                    }
                }

                var formation =
_player.BattleShopPlayer.BattleData;
                if
(_player.BattleShopPlayer.BattleData.round <= 32)
                {
                    await
Match._redis_handle.PushList(RedisHelp.BuildPeakStren
gthCache(), _player.BattleShopPlayer.BattleData);
                    await
Match._redis_handle.SetData(RedisHelp.BuildPlayerPeak
StrengthFormationCache(_player.BattleShopPlayer.Battl
eData.User.UserGuid), formation);
                }

                var player_proxy =
Match._player_proxy_mng.get_player(_player.PlayerHubN
ame);
                if
```

```
(_player.BattleShopPlayer.BattleData.victory >= 8)
                {
                    _player.BattleClientCaller.get_client(_player.ClientUID).battle_victory(_player.mod, true);
                    player_proxy.battle_victory(_player.mod, true, _player.BattleShopPlayer.BattleData, _player.BattleShopPlayer.UnlockRoles);

                    player_proxy.peak_strength_victory(is_victory, formation).callBack(async (userRankInfo) =>
                    {
                        Log.Log.trace("Plan_Module_on_confirm_round_victory peak_strength_victory begin! userRankInfo:{0}", userRankInfo);

                        userRankInfo.score += _player.BattleShopPlayer.BattleData.victory;
                        userRankInfo.battle_data = new UserSharedData()
                        {
                            User = _player.BattleShopPlayer.BattleData.User,
                            round = _player.BattleShopPlayer.BattleData.round,
                            victory = _player.BattleShopPlayer.BattleData.victory,
                            faild = 5 - _player.BattleShopPlayer.BattleData.faild,
                        };

                        using var st = MemoryStreamPool.mstMgr.GetStream();
                        var _serializer = MessagePackSerializer.Get<MessagePackObjectDictionary>();
                        _serializer.Pack(st, UserRankInfo.UserRankInfo_to_protcol(userRankInfo));
```

```
                        var r = new rank_item
                        {
                            guid =
_player.BattleShopPlayer.BattleData.User.UserGuid,
                            score =
userRankInfo.score,
                            item = st.ToArray()
                        };

Match._rank_proxy.update_rank_item(r);
                    }, (err) =>
                    {

Log.Log.err("confirm_round_victory err:{0}", err);
                        rsp.err();
                    }).timeout(1500, () =>
                    {

Log.Log.err("confirm_round_victory timeout!");
                        rsp.err();
                    });
                }
                else
                {
                    if
(_player.BattleShopPlayer.BattleData.faild <= 0)
                    {

_player.BattleClientCaller.get_client(_player.ClientU
UID).battle_victory(_player.mod, false);

player_proxy.battle_victory(_player.mod, false,
_player.BattleShopPlayer.BattleData,
_player.BattleShopPlayer.UnlockRoles);
                    }
                    else
                    {

_player.start_round(_player.baseStage(), addCoin);

```

```
_player.BattleShopPlayer.do_skill(_player.baseStage()
);

                                
_player.BattleClientCaller.get_client(_player.ClientU
UID).battle_plan_refresh(_player.BattleShopPlayer.Bat
tleData, _player.BattleShopPlayer.ShopData,
_player.BattleShopPlayer.check_fetters());
                    }
                }

                
_player.BattleShopPlayer.BattleData.round++;
                    rsp.rsp();
                }
                catch (System.Exception ex)
                {
                
Log.Log.err("Plan_Module_on_confirm_round_victory
error:{0}", ex);
                    rsp.err();
                }
            }

            private int countRoleList(List<Role>
roleList)
            {
                var count = 0;
                foreach(var r in roleList)
                {
                    if (r != null)
                    {
                        count++;
                    }
                }
                return count;
            }

            private int baseCount(int round)
            {
                int count;
```

```
            if (round <= 1)
            {
                count = 3;
            }
            else if (round <= 2)
            {
                count = 4;
            }
            else if (round <= 3)
            {
                count = 5;
            }
            else
            {
                count = 6;
            }

            return count;
        }

        private async Task<UserBattleData>
getCacheBattleData(int round)
        {
            var count = baseCount(round);

            var cache =
RedisHelp.BuildAutoChessBattleCache(round);
            var target = await
Match._redis_handle.RandomList<UserBattleData>(cache)
;
            if (target == null)
            {
                return null;
            }

            for(var i = 0; i < 3; i++)
            {
                if (countRoleList(target.RoleList) <
count)
                {
                    break;
```

```
                }
                target = await
Match._redis_handle.RandomList<UserBattleData>(cache)
;
            }

            return target;
        }

        private async void
Plan_Module_on_start_round1()
        {

Log.Log.trace("Plan_Module_on_start_round1 begin!");

            var rsp = plan_Module.rsp as
plan_start_round1_rsp;
            var uuid =
Hub.Hub._gates.current_client_uuid;

            try
            {
                var _player =
Match.battle_Mng.get_battle_player(uuid);

                var target = await
getCacheBattleData(_player.BattleShopPlayer.BattleDat
a.round);
                if (target == null)
                {
                    target =
getRandomBattleData(_player.BattleShopPlayer.BattleDa
ta.round);
                }

Log.Log.trace("Plan_Module_on_start_round1 round:{1}
target:{0}", target.ToJson(),
_player.BattleShopPlayer.BattleData.round);

rsp.rsp(_player.BattleShopPlayer.BattleData, target);
```

```
            }
            catch (System.Exception ex)
            {
                Log.Log.err("Plan_Module_on_start_round1 error:{0}",
ex);
                rsp.err((int)em_error.db_error);
            }
        }

        private List<RoleSetUp> selfSetUp;
        private List<RoleSetUp> targetSetUp;
        private void
Gm_Module_on_set_formation(List<RoleSetUp> self,
List<RoleSetUp> target)
        {
            var rsp = gm_Module.rsp as
gm_set_formation_rsp;

            selfSetUp = self;
            targetSetUp = target;

            rsp.rsp();
        }

        private UserBattleData
getRandomBattleData(int round, List<RoleSetUp> setUp)
        {
            var data = new UserBattleData();
            data.User = new UserInformation();
            data.RoleList = new List<Role>();

            var count = (round+2) > 6 ? 6 :
(round+2);
            var stage = (count/3) > 1 ? (count/3) :
1;

            while (data.RoleList.Count < count)
            {
                config.RoleConfig rolec;
                int level;
```

```
                if (setUp != null && setUp.Count > 0)
                {
                    var r = setUp[data.RoleList.Count];
                    rolec = config.Config.RoleConfigs[r.RoleID];
                    level = r.Level;
                }
                else
                {
                    var index = RandomHelper.RandomInt(config.Config.RoleConfigs.Values.Count);
                    rolec = config.Config.RoleConfigs.Values.ElementAt(index);

                    if (rolec.Stage > 1 && rolec.Stage > stage)
                    {
                        continue;
                    }

                    level = RandomHelper.RandomInt(3) + 1;
                    if (count < 6)
                    {
                        level = 1;
                    }
                }

                var role = new Role()
                {
                    RoleID = rolec.Id,
                    Level = level,
                    HP = rolec.Hp + 2 * (level-1),
                    Attack = rolec.Attack + 2 * (level - 1),
                    TempAdditionBuffer = new List<int>(),
                    additionBuffer = new List<int>(),
                };
```

```
                var indexBuffer =
RandomHelper.RandomInt(config.Config.BufferConfigs.Va
lues.Count);
                var bufferc =
config.Config.BufferConfigs.Values.ElementAt(indexBuf
fer);
                role.additionBuffer.Add(bufferc.Id);

                data.RoleList.Add(role);
            }

            return data;
        }

        private UserBattleData
getRandomBattleData(int round)
        {
            var data = new UserBattleData();
            data.User = new UserInformation();
            data.RoleList = new List<Role>();

            var count = (round + 2) > 6 ? 6 : (round
+ 2);
            var stage = (count / 3) > 1 ? (count /
3) : 1;

            while (data.RoleList.Count < count)
            {
                var index =
RandomHelper.RandomInt(config.Config.RoleConfigs.Valu
es.Count);
                var rolec =
config.Config.RoleConfigs.Values.ElementAt(index);

                Log.Log.trace("getRandomBattleData
rolec.Stage:{0} stage:{1} ", rolec.Stage, stage);
                if (rolec.Stage > 1 && rolec.Stage >
stage)
                {
                    continue;
```

```
                }

                int level = RandomHelper.RandomInt(3)
+ 1;
                if (count < 6)
                {
                    level = 1;
                }

                var role = new Role()
                {
                    RoleID = rolec.Id,
                    Level = level,
                    HP = rolec.Hp + 2 * (level - 1),
                    Attack = rolec.Attack + 2 *
(level - 1),
                    TempAdditionBuffer = new
List<int>(),
                    additionBuffer = new List<int>(),
                };

                var indexBuffer =
RandomHelper.RandomInt(config.Config.BufferConfigs.Va
lues.Count);
                var bufferc =
config.Config.BufferConfigs.Values.ElementAt(indexBuf
fer);
                role.additionBuffer.Add(bufferc.Id);

                data.RoleList.Add(role);
            }

            return data;
        }

        private void Plan_Module_on_start_round()
        {
            var rsp = plan_Module.rsp as
plan_start_round_rsp;
            var uuid =
Hub.Hub._gates.current_client_uuid;
```

```
            try
            {
                var self = getRandomBattleData(3, selfSetUp);
                var target = getRandomBattleData(3, targetSetUp);
                rsp.rsp(self, target);
            }
            catch(System.Exception ex)
            {
                Log.Log.err("Plan_Module_on_start_round error:{0}", ex);
                rsp.err((int)em_error.db_error);
            }
        }

        private void Plan_Module_on_refresh()
        {
            var rsp = plan_Module.rsp as plan_refresh_rsp;
            var uuid = Hub.Hub._gates.current_client_uuid;

            try
            {
                var self = Match.battle_Mng.get_battle_player(uuid);
                if (self.BattleShopPlayer.BattleData.coin <= 0)
                {
                    rsp.err((int)em_error.no_enough_coin);
                }
                else
                {
                    self.BattleShopPlayer.BattleData.coin--;

                    
```

```
self.BattleShopPlayer.refresh(self.baseStage());

self.BattleShopPlayer.do_skill(self.baseStage());

rsp.rsp(self.BattleShopPlayer.ShopData,
self.BattleShopPlayer.BattleData);
                }
            }
            catch(System.Exception ex)
            {
                Log.Log.err("Plan_Module_on_refresh
error:{0}", ex);
                rsp.err((int)em_error.db_error);
            }
        }

        private void Plan_Module_on_sale_role(int
index)
        {
            var rsp = plan_Module.rsp as
plan_sale_role_rsp;
            var uuid =
Hub.Hub._gates.current_client_uuid;

            try
            {
                var self =
Match.battle_Mng.get_battle_player(uuid);
                var stage = self.baseStage();
                if
(self.BattleShopPlayer.sale_role(index, stage))
                {

rsp.rsp(self.BattleShopPlayer.BattleData);
                }
                else
                {

rsp.err((int)em_error.no_role_in_index_to_sale);
                }
```

```
            }
            catch (System.Exception ex)
            {
                Log.Log.err("Plan_Module_on_sale_role
error:{0}", ex);
                rsp.err((int)em_error.db_error);
            }
        }

        private void Plan_Module_on_buy(ShopIndex
shop_index, int index, int role_index)
        {
            var rsp = plan_Module.rsp as
plan_buy_rsp;
            var uuid =
Hub.Hub._gates.current_client_uuid;

            try
            {
                var self =
Match.battle_Mng.get_battle_player(uuid);
                var err = self.buy(shop_index, index,
role_index);
                if (err == em_error.success)
                {
                    self.BattleShopPlayer.do_skill(self.baseStage());
                    rsp.rsp(self.BattleShopPlayer.BattleData,
self.BattleShopPlayer.ShopData);
                }
                else
                {
                    rsp.err((int)err);
                }
            }
            catch (System.Exception ex)
            {
                Log.Log.err("Plan_Module_on_buy
error:{0}", ex);
                rsp.err((int)em_error.db_error);
```

```
            }
        }


    }
}

using Abelkhan;
using Amazon.Util.Internal;
using config;
using InfluxData.Net.Common.Helpers;
using Service;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;

// to do
namespace Match
{
    public class peak_strength_player
    {
        public long LastActiveTime =
Timerservice.Tick;

        private long guid;
        public long GUID
        {
            get
            {
                return guid;
            }
        }

        private string clientUUID;
        public string ClientUUID
        {
            set
            {
                clientUUID = value;
            }
```

```
            get
            {
                return clientUUID;
            }
        }

        private string player_hub_name;
        public string PlayerHubName
        {
            get
            {
                return player_hub_name;
            }
        }

        public peak_strength_player(string
_player_hub_name, string _clientUUID, long _guid)
        {
            player_hub_name = _player_hub_name;
            clientUUID = _clientUUID;
            guid = _guid;
        }
    }

    public class peak_strength_mng
    {
        private Dictionary<string,
peak_strength_player> peak_strength_battles = new();

        public peak_strength_mng()
        {
            Hub.Hub._timer.addticktime(5 * 60 * 1000,
tick_clear_timeout_player);
        }

        private void tick_clear_timeout_player(long
tick_time)
        {
            List<peak_strength_player>
timeout_battle_player = new();
            foreach (var it in peak_strength_battles)
```

```
            {
                if ((it.Value.LastActiveTime + 30 *
60 * 1000) < Timerservice.Tick)
                {
                    timeout_battle_player.Add(it.Value);
                }
                else
                {
                    it.Value.LastActiveTime =
Timerservice.Tick;
                }
            }
            foreach (var _player in
timeout_battle_player)
            {
                peak_strength_battles.Remove(_player.ClientUUID);
            }

            Hub.Hub._timer.addticktime(5 * 60 * 1000,
tick_clear_timeout_player);
        }

        public peak_strength_player
add_player_to_battle(string player_hub_name, string
clientUUID, long guid)
        {
            Log.Log.trace("peak_strength_mng
add_player_to_battle player_hub_name:{0} clientUUID:
{1} guid:{2}", player_hub_name, clientUUID, guid);
            var _player = new
peak_strength_player(player_hub_name, clientUUID,
guid);
            peak_strength_battles[clientUUID] =
_player;
            return _player;
        }

        public bool change_player_uuid(string
old_client_uuid, string new_client_uuid)
```

```
        {
            if
(peak_strength_battles.Remove(old_client_uuid, out
var _player))
            {
                _player.ClientUUID = new_client_uuid;

peak_strength_battles[new_client_uuid] = _player;
            }

            return true;
        }

        public peak_strength_player
get_battle_player(string clientUUID)
        {
            Log.Log.trace("peak_strength_mng
get_battle_player peak_strength_battles:{0}
clientUUID:{1}", peak_strength_battles.ToJson(),
clientUUID);
            if
(peak_strength_battles.TryGetValue(clientUUID, out
var _player))
            {
                _player.LastActiveTime =
Timerservice.Tick;
            }
            return _player;
        }
    }
}

using Abelkhan;
using System;
using System.Collections.Generic;

namespace Match
{
    class player_msg_handle
    {
        private player_match_module
```

```
player_match_Module = new player_match_module();

        public player_msg_handle()
        {
            player_match_Module.on_start_battle +=
Player_match_Module_on_start_battle;

player_match_Module.on_start_peak_strength +=
Player_match_Module_on_start_peak_strength;
            player_match_Module.on_reconnect +=
Player_match_Module_on_reconnect;
        }

        private async void
Player_match_Module_on_start_peak_strength(string
clientUUID, long guid)
        {
            var rsp = player_match_Module.rsp as
player_match_start_peak_strength_rsp;

            // to do
            try
            {
                var date = System.DateTime.Now;
                int daysUntilSaturday =
((int)DayOfWeek.Saturday - (int)(date.DayOfWeek ==
DayOfWeek.Sunday ? 7 : (int)date.DayOfWeek));
                DateTime nextSaturday =
date.AddDays(daysUntilSaturday);
                DateTime nextSaturday1730 = new
DateTime(nextSaturday.Year, nextSaturday.Month,
nextSaturday.Day, 17, 30, 0);
                if (System.DateTime.Now >
nextSaturday1730)
                {

rsp.err((int)em_error.peak_strength_end);
                }

                var _player =
Match.peak_strength_mng.add_player_to_battle(Hub.Hub.
```

```
_hubs.current_hubproxy.name, clientUUID, guid);
                var _battleData = await
Match._redis_handle.GetData<UserBattleData>(RedisHelp
.BuildPlayerPeakStrengthFormationCache(_player.GUID))
;
                if (_battleData != null)
                {
                    rsp.rsp(_battleData);
                }
                else
                {
rsp.err((int)em_error.peak_strength_no_data);
                }
            }
            catch (System.Exception e)
            {
Log.Log.err("Player_match_Module_on_start_battle
error:{0}", e);
                rsp.err((int)em_error.db_error);
            }
        }

        private void
Player_match_Module_on_reconnect(string
old_client_uuid, string new_client_uuid)
        {
            var rsp = player_match_Module.rsp as
player_match_reconnect_rsp;

            try
            {
rsp.rsp(Match.battle_Mng.change_player_uuid(old_clien
t_uuid, new_client_uuid) &&
Match.peak_strength_mng.change_player_uuid(old_client
_uuid, new_client_uuid));
            }
            catch (System.Exception e)
```

```
            {
                Log.Log.err("Player_match_Module_on_start_battle
error:{0}", e);
                rsp.err();
            }
        }

        private void
Player_match_Module_on_start_battle(BattleMod mod,
string clientUUID, List<int> roleList,
UserInformation user_info)
        {
            var rsp = player_match_Module.rsp as
player_match_start_battle_rsp;

            try
            {
                var _player =
Match.battle_Mng.add_player_to_battle(mod,
clientUUID, roleList, user_info);
                Match.peak_strength_mng.add_player_to_battle(Hub.Hub.
_hubs.current_hubproxy.name, clientUUID,
user_info.UserGuid);
                _player.PlayerHubName =
Hub.Hub._hubs.current_hubproxy.name;
                rsp.rsp(_player.BattleShopPlayer.BattleData,
_player.BattleShopPlayer.ShopData);
            }
            catch (System.Exception e)
            {
                Log.Log.err("Player_match_Module_on_start_battle
error:{0}", e);
                rsp.err((int)em_error.db_error);
            }
        }
    }
}
```

```
using Abelkhan;
using System.Collections.Generic;
using System.Threading.Tasks;

namespace Match
{
    public class player_proxy
    {
        private readonly match_player_caller
_match_player_caller;
        private readonly Hub.HubProxy _proxy;

        public string name
        {
            get
            {
                return _proxy.name;
            }
        }

        public player_proxy(match_player_caller
caller, Hub.HubProxy proxy)
        {
            _match_player_caller = caller;
            _proxy = proxy;
        }

        public void battle_victory(BattleMod mod,
bool is_victory, UserBattleData user, List<Role>
roles)
        {
            
_match_player_caller.get_hub(_proxy.name).battle_vict
ory(mod, is_victory, user, roles);
        }

        public match_player_peak_strength_victory_cb
peak_strength_victory(BattleVictory is_victory,
UserBattleData user)
        {
```

```
            return
_match_player_caller.get_hub(_proxy.name).peak_streng
th_victory(is_victory, user);
        }

        public void buy_role(long guid, Role
roleInfo)
        {
            return
_match_player_caller.get_hub(_proxy.name).buy_role(gu
id, roleInfo);
        }

        public void buy_equip(long guid, int equipId)
        {
            return
_match_player_caller.get_hub(_proxy.name).buy_equip(g
uid, equipId);
        }
    }

    public class player_proxy_mng
    {
        private readonly match_player_caller
_match_player_caller = new ();
        private readonly Dictionary<string,
player_proxy> player_proxys = new ();

        public player_proxy_mng()
        {
        }

        public void reg_player_proxy(Hub.HubProxy
_proxy)
        {
            player_proxys[_proxy.name] = new
player_proxy(_match_player_caller, _proxy);
        }

        public player_proxy get_player(string
player_hub_name)
```

```
        {
            player_proxys.TryGetValue(player_hub_name, out
player_proxy proxy);
            return proxy;
        }
    }
}

using Abelkhan;

namespace Match
{
    class Match
    {
        public static RedisHandle _redis_handle;
        public static player_proxy_mng
_player_proxy_mng = new ();
        public static rank_proxy _rank_proxy;
        public static battle_mng battle_Mng = new ();
        public static peak_strength_mng
peak_strength_mng = new ();

        static void Main(string[] args)
        {
            var _hub = new Hub.Hub(args[0], args[1],
"match", "fixed");

            if (!
Hub.Hub._root_config.has_key("redis_for_mq_pwd"))
            {
                _redis_handle = new
RedisHandle(Hub.Hub._root_config.get_value_string("re
dis_for_cache"), string.Empty);
            }
            else
            {
                _redis_handle = new
RedisHandle(Hub.Hub._root_config.get_value_string("re
dis_for_cache"),
Hub.Hub._root_config.get_value_string("redis_for_mq_p
```

```
wd"));
            }

            config.Config.Load(Hub.Hub._config.get_value_string("excel_json_config"));

            var _player_msg_handle = new player_msg_handle();
            var _client_msg_handle = new client_msg_handle();

            _hub.on_hubproxy += on_hubproxy;
            _hub.on_hubproxy_reconn += on_hubproxy;

            _hub.onCloseServer += () => {
                _hub.closeSvr();
            };

            Log.Log.trace("login start ok");

            _hub.run().Wait();
        }

        private static void on_hubproxy(Hub.HubProxy _proxy)
        {
            if (_proxy.type == "player")
            {
                _player_proxy_mng.reg_player_proxy(_proxy);
            }
            else if (_proxy.type == "rank")
            {
                _rank_proxy = new rank_proxy(_proxy);
            }
        }
    }
}

using Abelkhan;
```

```
using System.Collections.Generic;
using System.Threading.Tasks;

namespace Match
{
    public class rank_proxy
    {
        private readonly  rank_svr_service_caller
_caller;
        private readonly Hub.HubProxy _proxy;

        public string name
        {
            get
            {
                return _proxy.name;
            }
        }

        public rank_proxy(Hub.HubProxy proxy)
        {
            _caller = new rank_svr_service_caller();
            _proxy = proxy;
        }

        public void update_rank_item(rank_item item)
        {
            
_caller.get_hub(_proxy.name).update_rank_item(Constan
t.Constant.player_db_guid_rank, item);
        }
    }
}

using Abelkhan;
using avatar;
using battle_shop;
using config;
using
Microsoft.AspNetCore.Authorization.Infrastructure;
using Microsoft.Extensions.Logging;
```

```
using MongoDB.Bson;
using OfflineMsg;
using Service;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;

namespace Player
{
    public class LoginException : System.Exception
    {
        public string ErrorInfo;
        public LoginException(string err) : base(err)
        {
            ErrorInfo = err;
        }
    }

    public class GetPlayerException :
System.Exception
    {
        public string ErrorInfo;
        public GetPlayerException(string err) :
base(err)
        {
            ErrorInfo = err;
        }
    }

    public class GetGuidFromDBError :
System.Exception
    {
        public string error;

        public GetGuidFromDBError(string err) :
base(err)
        {
            error = err;
        }
    }
```

```
    public class PlayerInfo : IHostingData
    {
        private UserData info;
        private long lastTickStrengthTime;

        public battle_shop_player battleShopPlayer;
        public PVELevelConfig pveLevelCfg;
        public List<Role> RogueRoleList = new();
        public bool isInitRound = false;
        public bool isPVEQuestEvent;

        public class TmpPVEData
        {
            public battle_shop_player
TmpBattleShopPlayer;
            public PVELevelConfig TmpPVELevelCfg;
            public int TmpPVELevelIndex;
            public bool isInitRound = false;
            public bool isQuestEvent;
        }
        public TmpPVEData tmpPVEData;
        public bool isTmp = false;

        public bool oldPVEQuestEvent;

        public static string Type()
        {
            return "player_info";
        }

        private static UserWeekAchievement
NewUserWeekAchievement()
        {
            return new UserWeekAchievement()
            {
                wAchievData = new
List<AchievementData>(),
                battleInfo = new
List<BattleInfo>(),       // 周成就 战绩记录
                timeout =
```

```
Timerservice.WeekEndTimetmp(),  // 周成就超时时间
            };
        }

        public static IHostingData Create()
        {
            return new PlayerInfo()
            {
                info = new UserData()
                {
                    User = new UserInformation()
                    {
                        UserName = string.Empty,
                        UserGuid = 0,
                    },
                    Achiev = new UserAchievement()
                    {
                        achievData = new
List<AchievementData>(),
                        battleInfo = new
List<BattleInfo>(),// 战绩记录
                    },
                    wAchiev =
NewUserWeekAchievement(),
                    Strength = 100,
                    gold = 100,
                    diamond = 10,
                    score = 0,
                    quest = 10001,
                    bag = new Abelkhan.Bag()
{ ItemList = new() },
                    RoleList = new List<Role>(),
                    GuideSteps = new (),
                },
                isPVEQuestEvent = false,
                lastTickStrengthTime =
Timerservice.Tick
            };
        }

        public static IHostingData Load(BsonDocument
```

```
data)
        {
            var info = new PlayerInfo()
            {
                info = new UserData()
                {
                    User = new UserInformation()
                    {
                        UserName = string.Empty,
                        UserGuid = 0,
                    },
                    Achiev = new UserAchievement()
                    {
                        achievData = new
List<AchievementData>(),
                        battleInfo = new
List<BattleInfo>(),// 战绩记录
                    },
                    wAchiev =
NewUserWeekAchievement(),
                    bag = new Abelkhan.Bag(),
                    Strength = 0,
                    gold = 0,
                    diamond = 0,
                    quest = 0,
                    score = 0,
                    RoleList = new (),
                    GuideSteps = new(),
                },
            };

            var user =
data.GetValue("User").AsBsonDocument;
            info.info.User =
MongoDB.Bson.Serialization.BsonSerializer.Deserialize
<UserInformation>(user);

            if (data.Contains("Achiev"))
            {
                var achiev =
data.GetValue("Achiev").AsBsonDocument;
```

```
                info.info.Achiev =
MongoDB.Bson.Serialization.BsonSerializer.Deserialize
<UserAchievement>(achiev);
                info.CheckAchievementStatus();
            }

            if (data.Contains("wAchiev"))
            {
                var wAchiev =
data.GetValue("wAchiev").AsBsonDocument;
                info.info.wAchiev =
MongoDB.Bson.Serialization.BsonSerializer.Deserialize
<UserWeekAchievement>(wAchiev);
                info.CheckWeekAchievement();
            }

            info.info.Strength =
data.GetValue("Strength").AsInt32;

            foreach (var role in
data.GetValue("RoleList").AsBsonArray)
            {
                var r =
MongoDB.Bson.Serialization.BsonSerializer.Deserialize
<Role>(role.AsBsonDocument);
                info.info.RoleList.Add(r);
            }

            var itemList = new List<RoleCardInfo>();
            foreach (var item in
data.GetValue("bag").AsBsonArray)
            {
                var roleID =
item.AsBsonDocument.GetValue("roleID").AsInt32;
                var Number =
item.AsBsonDocument.GetValue("Number").AsInt32;
                var isTatter =
item.AsBsonDocument.GetValue("isTatter").AsBoolean;

                itemList.Add(new RoleCardInfo() {
                    roleID= roleID,
```

```
                    Number = Number,
                    isTatter = isTatter
                });
            }
            info.info.bag.ItemList = itemList;

            if (data.Contains("guideSteps"))
            {
                var guideSteps =
data.GetValue("guideSteps").AsBsonArray;
                foreach (var g in guideSteps)
                {
                    
info.info.GuideSteps.Add((GuideStep)g.AsInt32);
                }
            }

            if (data.Contains("gold"))
            {
                info.info.gold =
data.GetValue("gold").AsInt32;
            }
            else
            {
                info.info.gold = 100;
            }

            if (data.Contains("diamond"))
            {
                info.info.diamond =
data.GetValue("diamond").AsInt32;
            }
            else
            {
                info.info.diamond = 10;
            }

            if (data.Contains("quest"))
            {
                info.info.quest =
data.GetValue("quest").AsInt32;
```

```
            }
            else
            {
                info.info.quest = 10001;
            }

            if (data.Contains("isPVEQuestEvent"))
            {
                info.isPVEQuestEvent =
data.GetValue("isPVEQuestEvent").AsBoolean;
            }
            else
            {
                info.isPVEQuestEvent = false;
            }

            if (data.Contains("RogueRoleList"))
            {
                var rogueRoleList =
data.GetValue("RogueRoleList").AsBsonArray;
                foreach (var r in rogueRoleList)
                {
                    info.RogueRoleList.Add(MongoDB.Bson.Serialization.Bso
nSerializer.Deserialize<Role>(r.ToBsonDocument()));
                }
            }

            if (data.Contains("PVELevelIndex"))
            {
                info.info.PVELevelIndex =
data.GetValue("PVELevelIndex").AsInt32;
            }

            if (data.Contains("score"))
            {
                info.info.score =
data.GetValue("score").AsInt32;
            }
            else
            {
```

```
                info.info.score = 0;
            }

            if
(data.Contains("lastTickStrengthTime"))
            {
                info.lastTickStrengthTime =
data.GetValue("lastTickStrengthTime").AsInt64;
            }
            else
            {
                info.lastTickStrengthTime =
Timerservice.Tick;
            }

            if (data.Contains("isTmp"))
            {
                info.isTmp =
data.GetValue("isTmp").AsBoolean;
            }

            if (data.Contains("BattleData"))
            {
                var BattleData =
data.GetValue("BattleData").AsBsonDocument;
                info.battleShopPlayer = new
battle_shop_player(null, null,
info.BattleRoleGroup(), info.info.User);
                info.battleShopPlayer.BattleData =
MongoDB.Bson.Serialization.BsonSerializer.Deserialize
<UserBattleData>(BattleData);
            }

            info.tmpPVEData = new();
            if (data.Contains("TmpBattleData"))
            {
                var BattleData =
data.GetValue("TmpBattleData").AsBsonDocument;
                info.tmpPVEData.TmpBattleShopPlayer =
new battle_shop_player(null, null,
info.BattleRoleGroup(), info.info.User);
```

```
                info.tmpPVEData.TmpBattleShopPlayer.BattleData =
MongoDB.Bson.Serialization.BsonSerializer.Deserialize
<UserBattleData>(BattleData);
            }
            if (data.Contains("TmpPVELevelIndex"))
            {
                info.tmpPVEData.TmpPVELevelIndex =
data.GetValue("TmpPVELevelIndex").AsInt32;
            }
            if (data.Contains("TmpPVEIsQuestEvent"))
            {
                info.tmpPVEData.isQuestEvent =
data.GetValue("TmpPVEIsQuestEvent").AsBoolean;
            }

            return info;
        }

        public BsonDocument Store()
        {
            var roleList = new BsonArray();
            foreach(var id in info.RoleList)
            {
                roleList.Add(id.ToBsonDocument());
            }

            var rogueRoleList = new BsonArray();
            foreach (var r in RogueRoleList)
            {
                rogueRoleList.Add(r.ToBsonDocument());
            }

            var itemList = new BsonArray();
            foreach (var item in info.bag.ItemList)
            {
                var i = new BsonDocument
{ { "roleID", item.roleID }, { "Number",
item.Number }, { "isTatter", item.isTatter } };
                itemList.Add(i);
```

```
            }

            var tmpGuideSteps = new BsonArray();
            if (info.GuideSteps != null)
            {
                foreach (var g in info.GuideSteps)
                {
                    tmpGuideSteps.Add(g);
                }
            }

            var doc = new BsonDocument
            {
                { "User", 
info.User.ToBsonDocument() },
                { "Achiev", 
info.Achiev.ToBsonDocument() },
                { "wAchiev", 
info.wAchiev.ToBsonDocument() },
                { "Strength", info.Strength },
                { "gold", info.gold },
                { "diamond", info.diamond },
                { "quest", info.quest },
                { "score", info.score },
                { "isQuestEvent", isPVEQuestEvent },
                { "RoleList", roleList },
                { "bag", itemList },
                { "guideSteps", tmpGuideSteps },
                { "lastTickStrengthTime", 
lastTickStrengthTime },
                { "PVELevelIndex", 
info.PVELevelIndex },
                {  "isTmp", isTmp },
                { "RogueRoleList", rogueRoleList }
            };

            if (battleShopPlayer != null)
            {
                doc.Add("BattleData", 
battleShopPlayer.BattleData.ToBsonDocument());
            }
```

```
            if (tmpPVEData != null)
            {
                if (tmpPVEData.TmpBattleShopPlayer != null)
                {
                    doc.Add("TmpBattleData", tmpPVEData.TmpBattleShopPlayer.BattleData.ToBsonDocument());
                }
                doc.Add("TmpPVELevelIndex", tmpPVEData.TmpPVELevelIndex);
            }

            return doc;
        }

        public UserData Info()
        {
            return info;
        }

        public void AddRogueRole(List<Role> role)
        {
            RogueRoleList.AddRange(role);
        }

        public bool CheckNotFinishPVELevel()
        {
            return config.Config.PVELevelConfigs.ContainsKey(info.quest);
        }

        public List<int> BattleRoleGroup()
        {
            return config.Config.RoleConfigs.Keys.ToList();
        }

        public void UpdateGuideStep(GuideStep step)
```

```
        {
            if (!info.GuideSteps.Contains(step))
            {
                info.GuideSteps.Add(step);
            }
        }

        public void AddRoleList(List<Role> roles)
        {
            info.RoleList.AddRange(roles);
        }

        public void CheckRank(BattleMod mod, bool
is_victory)
        {
            if (info.rankTimeTmp <
Timerservice.WeekEndTimetmp())
            {
                info.score = 0;
                info.rank = UserRank.BlackIron;
            }
            info.rankTimeTmp = Timerservice.Tick;

            if (mod == BattleMod.RankBattle)
            {
                if (is_victory)
                {
                    info.score += 1;
                }
                else
                {
                    info.score -= 1;
                    if (info.score < 0)
                    {
                        info.score = 0;
                    }
                }

                if (info.score < 5)
                {
                    info.rank = UserRank.BlackIron;
```

```
                }
                else if (info.score < 10)
                {
                    info.rank = UserRank.Bronze;
                }
                else if (info.score < 25)
                {
                    info.rank = UserRank.Silver;
                }
                else if (info.score < 30)
                {
                    info.rank = UserRank.Gold;
                }
                else if (info.score < 35)
                {
                    info.rank = UserRank.Diamond;
                }
                else if (info.score < 40)
                {
                    info.rank = UserRank.Master;
                }
                else
                {
                    info.rank = UserRank.King;
                }
            }
        }

        public void AddStrength(int _strength)
        {
            info.Strength += _strength;
        }

        public int CostStrength(int strength)
        {
            if ((info.Strength - strength) < 0)
            {
                return
(int)em_error.no_enough_strength;
            }
            info.Strength -= strength;
```

```
            return (int)em_error.success;
        }

        public bool CheckStrength()
        {
            if (info.Strength >= 100)
            {
                return false;
            }

            bool s_change = false;
            while (true)
            {
                if ((lastTickStrengthTime + 2 * 60 *
1000) > Timerservice.Tick)
                {
                    break;
                }

                lastTickStrengthTime += 2 * 60 *
1000;
                ++info.Strength;
                s_change = true;

                if (info.Strength >= 100)
                {
                    lastTickStrengthTime =
Timerservice.Tick;
                    break;
                }
            }

            return s_change;
        }

        public AchievementData
GetAchievementData(Achievement enumAchieve)
        {
            foreach(var a in info.Achiev.achievData)
            {
                if (a.emAchievement == enumAchieve)
```

```
                {
                    return a;
                }
            }

            var data = new AchievementData()
            {
                emAchievement = enumAchieve,
                status = 
AchievementAwardStatus.EMNotComplete,
                count = 0,
            };
            info.Achiev.achievData.Add(data);

            return data;
        }

        private void CheckAchievementStatus()
        {
            foreach (var info in 
info.Achiev.battleInfo)
            {
                CheckAchievement(info);
            }
        }

        private void CheckWeekAchievement()
        {
            foreach (var info in 
info.wAchiev.battleInfo)
            {
                CheckWeekAchievement(info);
            }

            foreach (var a in 
info.wAchiev.wAchievData)
            {
                if 
(config.Config.TaskConfigs.TryGetValue(Enum.GetName(t
ypeof(Achievement), a.emAchievement), out var task))
                {
```

```
                    if (a.count >= task.Value &&
a.status != AchievementAwardStatus.EMRecv)
                    {
                        a.status =
AchievementAwardStatus.EMComplete;
                    }
                }
            }
        }

        private bool CheckSuccessiveFiveVictory()
        {
            var data =
GetAchievementData(Achievement.EMSuccessiveFiveVictor
y);
            if (data.status ==
AchievementAwardStatus.EMRecv)
            {
                return false;
            }

            var check = true;
            for(int i = info.Achiev.battleInfo.Count
- 1; i < 0 && i < (info.Achiev.battleInfo.Count - 5);
--i)
            {
                var battleInfo =
info.Achiev.battleInfo[i];
                if (battleInfo.mod ==
BattleMod.Battle || battleInfo.mod ==
BattleMod.RankBattle)
                {
                    if (battleInfo.isVictory !=
BattleVictory.victory)
                    {
                        check = false;
                        break;
                    }
                }
            }
```

```
            if (check)
            {
                data.status =
AchievementAwardStatus.EMComplete;
            }

            return check;
        }

        private bool CheckFullLevelVictory(BattleInfo
battleInfo)
        {
            var data =
GetAchievementData(Achievement.EMFullLevelVictory);
            if (data.status ==
AchievementAwardStatus.EMRecv)
            {
                return false;
            }

            var check = true;
            if (battleInfo.RoleList == null)
            {
                check = false;
            }
            else
            {
                foreach (var r in
battleInfo.RoleList)
                {
                    if (r != null && r.Level < 3)
                    {
                        check = false;
                        break;
                    }
                }
            }

            if (check)
            {
                data.status =
```

```
AchievementAwardStatus.EMComplete;
            }

            return check;
        }

        private bool
CheckMachinistlVictory(BattleInfo battleInfo)
        {
            var data =
GetAchievementData(Achievement.EMMachinistlVictory);
            if (data.status ==
AchievementAwardStatus.EMRecv)
            {
                return false;
            }

            var check = true;
            if (battleInfo.RoleList == null)
            {
                check = false;
            }
            else
            {
                foreach (var r in
battleInfo.RoleList)
                {
                    if (r != null &&
r.FettersSkillID.fetters_id != 6)
                    {
                        check = false;
                        break;
                    }
                }
            }

            if (check)
            {
                data.status =
AchievementAwardStatus.EMComplete;
            }
```

```
            return check;
        }

        private bool
CheckFullAttributesVictory(BattleInfo battleInfo)
        {
            var data =
GetAchievementData(Achievement.EMFullAttributesVictor
y);
            if (data.status ==
AchievementAwardStatus.EMRecv)
            {
                return false;
            }

            var check = true;
            if (battleInfo.RoleList == null)
            {
                check = false;
            }
            else
            {
                foreach (var r in
battleInfo.RoleList)
                {
                    if (r != null && r.Attack >= 50
&& r.HP >= 50)
                    {
                        check = false;
                        break;
                    }
                }
            }

            if (check)
            {
                data.status =
AchievementAwardStatus.EMComplete;
            }
```

```
            return check;
        }

        private bool
CheckNoneEquipmentVictory(BattleInfo battleInfo)
        {
            var data =
GetAchievementData(Achievement.EMNoneEquipmentVictory
);
            if (data.status ==
AchievementAwardStatus.EMRecv)
            {
                return false;
            }

            var check = true;
            if (battleInfo.RoleList == null)
            {
                check = false;
            }
            else
            {
                foreach (var r in
battleInfo.RoleList)
                {
                    if (r != null && r.equipID != 0)
                    {
                        check = false;
                        break;
                    }
                }
            }

            if (check)
            {
                data.status =
AchievementAwardStatus.EMComplete;
            }

            return check;
        }
```

```
        public bool CheckAchievement(BattleInfo
battleInfo)
        {
            bool check = false;
            if (CheckSuccessiveFiveVictory())
            {
                check = true;
            }
            if (CheckFullLevelVictory(battleInfo))
            {
                check = true;
            }
            if (battleInfo.isStreakVictory)
            {
                var data =
GetAchievementData(Achievement.EMStreakVictory);
                if (data.status !=
AchievementAwardStatus.EMRecv)
                {
                    data.status =
AchievementAwardStatus.EMComplete;
                    check = true;
                }
            }
            if
(config.Config.TaskConfigs.TryGetValue(Enum.GetName(t
ypeof(Achievement), Achievement.EMFiveHundredGame),
out var task))
            {
                if (info.Achiev.battleInfo.Count >=
task.Value)
                {
                    var data =
GetAchievementData(Achievement.EMFiveHundredGame);
                    if (data.status !=
AchievementAwardStatus.EMRecv)
                    {
                        data.status =
AchievementAwardStatus.EMComplete;
                        check = true;
```

```
                    }
                }
            }
            if (CheckMachinistlVictory(battleInfo))
            {
                check = true;
            }
            if
(CheckFullAttributesVictory(battleInfo))
            {
                check = true;
            }
            if
(CheckNoneEquipmentVictory(battleInfo))
            {
                check = true;
            }
            if
(config.Config.TaskConfigs.TryGetValue(Enum.GetName(t
ypeof(Achievement), Achievement.EMNotGivenAllYet),
out var task1))
            {
                if (battleInfo.RoleList == null)
                {
                    check = false;
                }
                else
                {
                    var count = 0;
                    foreach(var r in
battleInfo.RoleList)
                    {
                        if (r != null)
                        {
                            count++;
                        }
                    }
                    if (count < task1.Value)
                    {
                        var data =
GetAchievementData(Achievement.EMNotGivenAllYet);
```

```
                        if (data.status !=
AchievementAwardStatus.EMRecv)
                        {
                            data.status =
AchievementAwardStatus.EMComplete;
                            check = true;
                        }
                    }
                }
            }

            return check;
        }

        public bool AddCheckAchievement(BattleInfo
battleInfo)
        {
            info.Achiev.battleInfo.Add(battleInfo);

            return CheckAchievement(battleInfo);
        }

        public AchievementData
GetWeekAchievementData(Achievement enumAchieve)
        {
            foreach (var a in
info.wAchiev.wAchievData)
            {
                if (a.emAchievement == enumAchieve)
                {
                    return a;
                }
            }

            var data = new AchievementData()
            {
                emAchievement = enumAchieve,
                status =
AchievementAwardStatus.EMNotComplete,
                count = 0,
            };
```

```
            info.wAchiev.wAchievData.Add(data);

            return data;
        }

        private bool CheckFullLevelRole(BattleInfo
battleInfo)
        {
            if (battleInfo.RoleList != null)
            {
                foreach (var role in
battleInfo.RoleList)
                {
                    if (role != null && role.Level >=
3)
                    {
                        return true;
                    }
                }
            }

            return false;
        }

        public bool CheckKillRole(Role roleInfo)
        {
            RefreshWeekAchiev();

            var check = false;
            var a =
GetWeekAchievementData(Achievement.EMWeekTotalAnnihil
ation);
            if (a.status !=
AchievementAwardStatus.EMRecv)
            {
                a.count++;
                if
(config.Config.TaskConfigs.TryGetValue(Enum.GetName(t
ypeof(Achievement),
Achievement.EMWeekTotalAnnihilation), out var task))
                {
```

```
                    if (a.count >= task.Value)
                    {
                        a.status =
AchievementAwardStatus.EMComplete;
                        check = true;
                    }
                }
            }

            if (roleInfo.FettersSkillID != null)
            {
                switch
(roleInfo.FettersSkillID.fetters_id)
                {
                    case 9:
                        {
                            a =
GetWeekAchievementData(Achievement.EMWeekWizardAnnihi
lation);
                            if (a.status !=
AchievementAwardStatus.EMRecv)
                            {
                                a.count++;
                                if
(config.Config.TaskConfigs.TryGetValue(Enum.GetName(t
ypeof(Achievement),
Achievement.EMWeekWizardAnnihilation), out var task))
                                {
                                    if (a.count >=
task.Value)
                                    {
                                        a.status =
AchievementAwardStatus.EMComplete;
                                        check = true;
                                    }
                                }
                            }
                        }
                        break;

                    case 2:
```

```
                    {
                        a = GetWeekAchievementData(Achievement.EMWeekBerserkerAnnihilation);
                        if (a.status != AchievementAwardStatus.EMRecv)
                        {
                            a.count++;
                            if (config.Config.TaskConfigs.TryGetValue(Enum.GetName(typeof(Achievement), Achievement.EMWeekBerserkerAnnihilation), out var task))
                            {
                                if (a.count >= task.Value)
                                {
                                    a.status = AchievementAwardStatus.EMComplete;
                                    check = true;
                                }
                            }
                        }
                    }
                    break;

                case 3:
                    {
                        a = GetWeekAchievementData(Achievement.EMWeekCorsairAnnihilation);
                        if (a.status != AchievementAwardStatus.EMRecv)
                        {
                            if (config.Config.TaskConfigs.TryGetValue(Enum.GetName(typeof(Achievement), Achievement.EMWeekCorsairAnnihilation), out var task))
                            {
                                a.count++;
```

```
                                        if (a.count >= task.Value)
                                        {
                                            a.status = AchievementAwardStatus.EMComplete;
                                            check = true;
                                        }
                                    }
                                }
                            }
                            break;
                        }
                    }

                    return check;
                }

                public bool CheckBuyRole(Role roleInfo)
                {
                    RefreshWeekAchiev();

                    var check = false;
                    if (config.Config.SkillConfigs.TryGetValue(roleInfo.SkillID, out var skill))
                    {
                        if (skill.EffectTime == 3)
                        {
                            var a = GetWeekAchievementData(Achievement.EMWeekBuyTenBeforeRound);
                            if (a.status != AchievementAwardStatus.EMRecv)
                            {
                                a.count++;
                                if (config.Config.TaskConfigs.TryGetValue(Enum.GetName(typeof(Achievement), Achievement.EMWeekBuyTenBeforeRound), out var task))
                                {
                                    if (a.count >=
```

```
task.Value)
                                {
                                    a.status =
AchievementAwardStatus.EMComplete;
                                    check = true;
                                }
                            }
                        }
                    }
                else if (skill.EffectTime == 18)
                {
                    var a =
GetWeekAchievementData(Achievement.EMWeekBuyBeHurted)
;
                    if (a.status !=
AchievementAwardStatus.EMRecv)
                    {
                        a.count++;
                        if
(config.Config.TaskConfigs.TryGetValue(Enum.GetName(t
ypeof(Achievement), Achievement.EMWeekBuyBeHurted),
out var task))
                        {
                            if (a.count >=
task.Value)
                            {
                                a.status =
AchievementAwardStatus.EMComplete;
                                check = true;
                            }
                        }
                    }
                }
                else if (skill.EffectTime == 8)
                {
                    var a =
GetWeekAchievementData(Achievement.EMWeekBuyBeDead);
                    if (a.status !=
AchievementAwardStatus.EMRecv)
                    {
                        a.count++;
```

```
                        if
(config.Config.TaskConfigs.TryGetValue(Enum.GetName(t
ypeof(Achievement), Achievement.EMWeekBuyBeDead), out
var task))
                        {
                            if (a.count >=
task.Value)
                            {
                                a.status =
AchievementAwardStatus.EMComplete;
                                check = true;
                            }
                        }
                    }
                }

                return check;
            }

            public bool CheckBuyEquip(int equipID)
            {
                RefreshWeekAchiev();

                var a =
GetWeekAchievementData(Achievement.EMWeekBuyTenEquip)
;
                if (a.status !=
AchievementAwardStatus.EMRecv)
                {
                    a.count++;
                    if
(config.Config.TaskConfigs.TryGetValue(Enum.GetName(t
ypeof(Achievement), Achievement.EMWeekBuyTenEquip),
out var task))
                    {
                        if (a.count >= task.Value)
                        {
                            a.status =
AchievementAwardStatus.EMComplete;
                            return true;
```

```
                    }
                }
            }

            return false;
        }

        private bool CheckWeekAchievement(BattleInfo
battleInfo)
        {
            var check = false;
            if (battleInfo.isVictory ==
BattleVictory.victory)
            {
                var a =
GetWeekAchievementData(Achievement.EMWeekOneGameVicto
ry);
                if (a.status !=
AchievementAwardStatus.EMRecv)
                {
                    a.status =
AchievementAwardStatus.EMComplete;
                    check = true;
                }
            }
            if (CheckFullLevelRole(battleInfo))
            {
                var a =
GetWeekAchievementData(Achievement.EMWeekOneFullLevel
Role);
                if (a.status !=
AchievementAwardStatus.EMRecv)
                {
                    a.status =
AchievementAwardStatus.EMComplete;
                    check = true;
                }
            }

            return check;
        }
```

```
        public bool
AddCheckWeekAchievement(BattleInfo battleInfo)
        {
            RefreshWeekAchiev();
            info.wAchiev.battleInfo.Add(battleInfo);

            return CheckWeekAchievement(battleInfo);
        }

        private void RefreshWeekAchiev()
        {
            if (info.wAchiev != null &&
info.wAchiev.timeout < Timerservice.Tick)
            {
                info.wAchiev =
NewUserWeekAchievement();
            }
        }

        public bool
CheckGetAchievementReward(Achievement achievement)
        {
            RefreshWeekAchiev();

            switch (achievement)
            {
                case
Achievement.EMSuccessiveFiveVictory:
                case Achievement.EMFullLevelVictory:
                case Achievement.EMStreakVictory:
                case Achievement.EMFiveHundredGame:
                case Achievement.EMMachinistlVictory:
                case
Achievement.EMPeakStrengthVictory:
                case Achievement.EMGold25:
                case
Achievement.EMFullAttributesVictory:
                case
Achievement.EMNoneEquipmentVictory:
                    {
```

```
                            var a =
GetAchievementData(achievement);
                            if (a.status ==
AchievementAwardStatus.EMComplete)
                            {
                                a.status =
AchievementAwardStatus.EMRecv;
                                return true;
                            }
                        }
                        break;

                    case
Achievement.EMWeekOneGameVictory:
                    case Achievement.EMWeekOpenCardPack:
                    case
Achievement.EMWeekTotalAnnihilation:
                    case
Achievement.EMWeekWizardAnnihilation:
                    case
Achievement.EMWeekBerserkerAnnihilation:
                    case
Achievement.EMWeekCorsairAnnihilation:
                    case
Achievement.EMWeekBuyTenBeforeRound:
                    case Achievement.EMWeekBuyBeHurted:
                    case Achievement.EMWeekBuyBeDead:
                    case Achievement.EMWeekBuyTenEquip:
                    case
Achievement.EMWeekOneFullLevelRole:
                        {
                            var a =
GetWeekAchievementData(achievement);
                            if (a.status ==
AchievementAwardStatus.EMComplete)
                            {
                                a.status =
AchievementAwardStatus.EMRecv;
                                return true;
                            }
                        }
```

```
                    break;
                }

                return false;
            }

            public bool refresh(int stage)
            {
                var BattleShopPlayerImpl =
battleShopPlayer;
                if (tmpPVEData != null &&
tmpPVEData.TmpBattleShopPlayer != null)
                {
                    BattleShopPlayerImpl =
tmpPVEData.TmpBattleShopPlayer;
                }

                if (BattleShopPlayerImpl.BattleData.coin
<= 0)
                {
                    return false;
                }

                BattleShopPlayerImpl.BattleData.coin--;
                BattleShopPlayerImpl.ShopData =
BattleShopPlayerImpl.refresh(stage);

                return true;
            }

            public int GetStage()
            {
                var PVELevelCfgImpl = pveLevelCfg;
                var PVELevelIndex = info.PVELevelIndex;
                if (tmpPVEData != null &&
tmpPVEData.TmpPVELevelCfg != null)
                {
                    PVELevelCfgImpl =
tmpPVEData.TmpPVELevelCfg;
                    PVELevelIndex =
tmpPVEData.TmpPVELevelIndex;
```

```
            }

            Log.Log.trace("GetStage PVELevelCfg:{0}
PVELevelIndex:{1}", PVELevelCfgImpl.ToJson(),
PVELevelIndex);
            if
(config.Config.PVERoundConfigs.TryGetValue(PVELevelCf
gImpl.Level[PVELevelIndex], out var cfg))
            {
                return cfg.Stage;
            }

            return 1;
        }

        public void ClearCurrPVEState()
        {
            Log.Log.trace("ClearPVEState");
            battleShopPlayer = null;
            pveLevelCfg = null;
            info.PVELevelIndex = 0;
        }

        public void ClearTmpPVEState()
        {
            Log.Log.trace("ClearTmpPVEState");
            tmpPVEData = null;
        }

        public battle_shop_player
battle_failed_back()
        {
            if (isTmp)
            {
                if (tmpPVEData != null &&
tmpPVEData.TmpBattleShopPlayer != null)
                {

tmpPVEData.TmpBattleShopPlayer.BattleData.faild = 1;
                    StartPVERound(0);
                }
```

```
                return
tmpPVEData.TmpBattleShopPlayer;
            }
            else
            {
                battleShopPlayer.BattleData.faild =
1;
                StartPVERound(0);

                return battleShopPlayer;
            }
        }

        public void StartPVERound(int addCoin)
        {
            var BattleShopPlayerImpl =
battleShopPlayer;
            var PVELevelCfg = pveLevelCfg;
            if (isTmp && tmpPVEData != null &&
tmpPVEData.TmpBattleShopPlayer != null)
            {
                BattleShopPlayerImpl =
tmpPVEData.TmpBattleShopPlayer;
                PVELevelCfg =
tmpPVEData.TmpPVELevelCfg;
            }

            if (info.PVELevelIndex >=
PVELevelCfg.Level.Count)
            {
                info.PVELevelIndex = 0;
            }

            foreach (var r in
BattleShopPlayerImpl.BattleData.RoleList)
            {
                if (r != null)
                {
                    r.TempHP = 0;
                    r.TempAttack = 0;
```

```
                    r.TempAdditionBuffer.Clear();
                }
            }

            if
(BattleShopPlayerImpl.ShopData.SaleRoleList[5] ==
null)
            {
                BattleShopPlayerImpl.ShopData =
BattleShopPlayerImpl.refresh(GetStage());
            }

            if
(config.Config.PVERoundConfigs.TryGetValue(PVELevelCf
g.Level[info.PVELevelIndex], out var cfg))
            {
                BattleShopPlayerImpl.BattleData.coin
= cfg.Gold + BattleShopPlayerImpl.bankCoin + addCoin;
                BattleShopPlayerImpl.bankCoin = 0;
                BattleShopPlayerImpl.Coin =
BattleShopPlayerImpl.BattleData.coin;
            }

            BattleShopPlayerImpl.evs.Add(new
shop_event()
            {
                ev = EMRoleShopEvent.start_round
            });

            
BattleShopPlayerImpl.do_skill(GetStage());
        }

        public void EndPVERound()
        {
            var BattleShopPlayerImpl =
battleShopPlayer;
            if (tmpPVEData != null &&
tmpPVEData.TmpBattleShopPlayer != null)
            {
                BattleShopPlayerImpl =
```

```
tmpPVEData.TmpBattleShopPlayer;
            }

            BattleShopPlayerImpl.evs.Add(new
shop_event()
            {
                ev = EMRoleShopEvent.end_round
            });

            BattleShopPlayerImpl.do_skill(GetStage());
        }

        private void ResetTmpBattleShop()
        {
            isTmp = false;
        }

        public Tuple<em_error, List<QuestRogueRole>>
StartQuestReady(string _clientUUID,
battle_client_caller battleClientCaller)
        {
            ResetTmpBattleShop();

            if (battleShopPlayer == null)
            {
                battleShopPlayer = new
battle_shop_player(_clientUUID, battleClientCaller,
BattleRoleGroup(), info.User);
            }
            else
            {
                battleShopPlayer.ClientUUID =
_clientUUID;
                battleShopPlayer.BattleClientCaller =
battleClientCaller;
            }

            if
(config.Config.PVELevelConfigs.TryGetValue(info.quest
, out var cfg))
```

```
            {
                Log.Log.trace("StartQuestReady
PVELevelConfigs TryGetValue quest:{0}", info.quest);

                if (cfg.Level.Count <= 0)
                {
                    return
Tuple.Create(em_error.last_quest, new
List<QuestRogueRole>());
                }

                pveLevelCfg = cfg;
                if (!isInitRound)
                {
                    battleShopPlayer.BattleData.faild
= pveLevelCfg.Hp;
                    StartPVERound(0);
                    isInitRound = true;
                }
                if
((battleShopPlayer.ShopData.SaleRoleList.Count == 0
||
                    battleShopPlayer.ShopData.SalePropList.Count == 0) &&
                    battleShopPlayer.ShopData.SaleRoleList[5] == null)
                {
                    battleShopPlayer.refresh(GetStage());
                }

                Log.Log.trace("PVELevelConfigs
TryGetValue isQuestEvent:{0}", isPVEQuestEvent);
                if (!isPVEQuestEvent)
                {
                    Log.Log.trace("PVELevelConfigs
TryGetValue info.PVELevelIndex:{0}",
info.PVELevelIndex);
                    if (pveLevelCfg.Level.Count <=
info.PVELevelIndex)
                    {
```

```
                        return
Tuple.Create(em_error.last_quest, new
List<QuestRogueRole>());
                    }

                    if (!
config.Config.PVERoundConfigs.TryGetValue(pveLevelCfg
.Level[info.PVELevelIndex], out var roundcfg))
                    {
                        return
Tuple.Create(em_error.success, new
List<QuestRogueRole>());
                    }

                    var roles = new
List<QuestRogueRole>();
                    if (RogueRoleList.Count >= 3)
                    {
                        while (roles.Count < 3)
                        {
                        break1:
                            var index =
RandomHelper.RandomInt(RogueRoleList.Count);
                            var r =
RogueRoleList[index];
                            foreach (var _role in
roles)
                            {
                                if (_role.index ==
index)
                                {
                                    goto break1;
                                }
                            }

                            var _roleRougue = new
QuestRogueRole()
                            {
                                r = r,
                                index = index
                            };
```

```
                                roles.Add(_roleRougue);
                            }
                        }
                        else
                        {

                            for (int index = 0; index < roundcfg.EventID.Count; index++)
                            {
                                var eventId = roundcfg.EventID[index];
                                if (config.Config.PVEEventConfigs.TryGetValue(eventId, out var ecfg))
                                {
                                    if (config.Config.RoleConfigs.TryGetValue(ecfg.RoleID, out RoleConfig rcfg))
                                    {
                                        var r = new Role();
                                        r.RoleID = ecfg.RoleID;
                                        r.Level = ecfg.RoleLevel;
                                        r.SkillID = rcfg.SkillID;
                                        r.HP = ecfg.RoleHP;
                                        r.Attack = ecfg.RoleAttack;
                                        r.equipID = ecfg.RoleEquip;

                                        var _roleRougue = new QuestRogueRole()
                                        {
                                            r = r,
                                            index = index
                                        };
```

```
roles.Add(_roleRougue);
                                    }
                                }
                            }
                        }
                    return
Tuple.Create(em_error.success, roles);
                }
                return Tuple.Create(em_error.success,
new List<QuestRogueRole>());
            }

            return
Tuple.Create(em_error.not_exist_quest, new
List<QuestRogueRole>());
        }

        public Tuple<em_error, List<QuestRogueRole>>
StartQuestReady1(int quest, string _clientUUID,
battle_client_caller battleClientCaller)
        {
            if (tmpPVEData == null)
            {
                tmpPVEData = new TmpPVEData();
            }
            if (tmpPVEData.TmpBattleShopPlayer ==
null || tmpPVEData.TmpPVELevelCfg == null || quest !=
tmpPVEData.TmpPVELevelCfg.ID)
            {
                tmpPVEData.TmpBattleShopPlayer = new
battle_shop_player(_clientUUID, battleClientCaller,
BattleRoleGroup(), info.User);
            }
            else
            {
                tmpPVEData.TmpBattleShopPlayer.ClientUUID =
_clientUUID;
                tmpPVEData.TmpBattleShopPlayer.BattleClientCaller =
battleClientCaller;
```

```
            }
            isTmp = true;

            if
(config.Config.PVELevelConfigs.TryGetValue(quest, out
var cfg))
            {
                Log.Log.trace("StartQuestReady1
PVELevelConfigs TryGetValue quest:{0}", quest);

                if (cfg.Level.Count <= 0)
                {
                    return
Tuple.Create(em_error.last_quest, new
List<QuestRogueRole>());
                }

                tmpPVEData.TmpPVELevelCfg = cfg;
                if (!tmpPVEData.isInitRound)
                {
                    tmpPVEData.TmpBattleShopPlayer.BattleData.faild =
tmpPVEData.TmpPVELevelCfg.Hp;
                    StartPVERound(0);
                    tmpPVEData.isInitRound = true;
                }
                if
((tmpPVEData.TmpBattleShopPlayer.ShopData.SaleRoleLis
t.Count == 0 ||
                    tmpPVEData.TmpBattleShopPlayer.ShopData.SalePropList.
Count == 0) &&
                    tmpPVEData.TmpBattleShopPlayer.ShopData.SaleRoleList[
5] == null)
                {
                    tmpPVEData.TmpBattleShopPlayer.refresh(GetStage());
                }

                Log.Log.trace("PVELevelConfigs
```

```
TryGetValue isQuestEvent:{0}",
tmpPVEData.isQuestEvent);
                if (!tmpPVEData.isQuestEvent)
                {
                    Log.Log.trace("PVELevelConfigs
TryGetValue info.PVELevelIndex:{0}",
tmpPVEData.TmpPVELevelIndex);
                    if
(tmpPVEData.TmpPVELevelCfg.Level.Count <=
tmpPVEData.TmpPVELevelIndex)
                    {
                        return
Tuple.Create(em_error.last_quest, new
List<QuestRogueRole>());
                    }

                    if (!
config.Config.PVERoundConfigs.TryGetValue(tmpPVEData.
TmpPVELevelCfg.Level[tmpPVEData.TmpPVELevelIndex],
out var roundcfg))
                    {
                        return
Tuple.Create(em_error.success, new
List<QuestRogueRole>());
                    }

                    var roles = new
List<QuestRogueRole>();
                    if (RogueRoleList.Count >= 3)
                    {
                        while (roles.Count < 3)
                        {
                        break1:
                            var index =
RandomHelper.RandomInt(RogueRoleList.Count);
                            var r =
RogueRoleList[index];
                            foreach (var _role in
roles)
                            {
                                if (_role.index ==
```

```
index)
                                    {
                                        goto break1;
                                    }
                                }

                                var _roleRougue = new
QuestRogueRole()
                                {
                                    r = r,
                                    index = index
                                };
                                roles.Add(_roleRougue);
                            }
                        }
                        else
                        {
                            for (int index = 0; index <
roundcfg.EventID.Count; index++)
                            {
                                var eventId =
roundcfg.EventID[index];
                                if
(config.Config.PVEEventConfigs.TryGetValue(eventId,
out var ecfg))
                                {
                                    if
(config.Config.RoleConfigs.TryGetValue(ecfg.RoleID,
out RoleConfig rcfg))
                                    {
                                        var r = new
Role();
                                        r.RoleID =
ecfg.RoleID;
                                        r.Level =
ecfg.RoleLevel;
                                        r.SkillID =
rcfg.SkillID;
                                        r.HP =
ecfg.RoleHP;
                                        r.Attack =
```

```
ecfg.RoleAttack;
                                        r.equipID =
ecfg.RoleEquip;

                                        var _roleRougue =
new QuestRogueRole()
                                        {
                                            r = r,
                                            index = index
                                        };

roles.Add(_roleRougue);
                                    }
                                }
                            }
                        }
                    return
Tuple.Create(em_error.success, roles);
                }
                return Tuple.Create(em_error.success,
new List<QuestRogueRole>());
            }

            return
Tuple.Create(em_error.not_exist_quest, new
List<QuestRogueRole>());
        }

        public void StartQuestShop(int index)
        {
            Log.Log.trace("StartQuestShop index:{0}",
index);
            if (index < 0)
            {
                return;
            }

            var r = new ShopRole();
            if (RogueRoleList.Count >= 3)
            {
                var role = RogueRoleList[index];
```

```
                r.RoleID = role.RoleID;
                r.Level = role.Level;
                r.SkillID = role.SkillID;
                r.HP = role.HP;
                r.Attack = role.Attack;
                r.equipID = role.equipID;
            }
            else
            {
                var PVELevelCfgImpl = pveLevelCfg;
                var PVELevelIndex =
info.PVELevelIndex;
                if (isTmp && tmpPVEData != null &&
tmpPVEData.TmpBattleShopPlayer != null)
                {
                    PVELevelCfgImpl =
tmpPVEData.TmpPVELevelCfg;
                    PVELevelIndex =
tmpPVEData.TmpPVELevelIndex;
                }

                if
(config.Config.PVERoundConfigs.TryGetValue(PVELevelCf
gImpl.Level[PVELevelIndex], out var roundcfg))
                {
                    var eventId =
roundcfg.EventID[index];
                    if
(config.Config.PVEEventConfigs.TryGetValue(eventId,
out var ecfg))
                    {
                        if
(config.Config.RoleConfigs.TryGetValue(ecfg.RoleID,
out RoleConfig rcfg))
                        {
                            r.RoleID = ecfg.RoleID;
                            r.Level = ecfg.RoleLevel;
                            r.SkillID = rcfg.SkillID;
                            r.HP = ecfg.RoleHP;
                            r.Attack =
ecfg.RoleAttack;
```

```
                                r.equipID =
ecfg.RoleEquip;
                            }
                        }
                    }
                }

            if (isTmp && tmpPVEData != null &&
tmpPVEData.TmpBattleShopPlayer != null)
            {
                
tmpPVEData.TmpBattleShopPlayer.ShopData.SaleRoleList[
5] = r;
                tmpPVEData.isQuestEvent = true;
            }
            else
            {
                
battleShopPlayer.ShopData.SaleRoleList[5] = r;
                isPVEQuestEvent = true;
            }
        }


        public UserBattleData StartQuestBattle()
        {
            var PVELevelCfgImpl = pveLevelCfg;
            var PVELevelIndex = info.PVELevelIndex;
            if (isTmp && tmpPVEData != null &&
tmpPVEData.TmpBattleShopPlayer != null)
            {
                PVELevelCfgImpl =
tmpPVEData.TmpPVELevelCfg;
                PVELevelIndex =
tmpPVEData.TmpPVELevelIndex;
            }

            var target = new UserBattleData();
            target.RoleList = new List<Role>();

            if
```

```
(config.Config.PVERoundConfigs.TryGetValue(PVELevelCf
gImpl.Level[PVELevelIndex], out var cfg))
            {
                foreach(var rInfo in cfg.Enemys)
                {
                    if
(config.Config.RoleConfigs.TryGetValue(rInfo.RoleID,
out RoleConfig rcfg))
                    {
                        var r = new Role();

                        r.RoleID = rInfo.RoleID;
                        r.Level = rInfo.RoleLevel;
                        r.SkillID = rcfg.SkillID;
                        r.HP = rInfo.RoleHP;
                        r.Attack = rInfo.RoleAttack;
                        r.TempHP = 0;
                        r.TempAttack = 0;
                        r.TempAdditionBuffer = new();
                        r.FettersSkillID = new
Fetters()
                        {
                            fetters_id =
rcfg.Fetters,
                            fetters_level = 0,
                            number = 1
                        };
                        r.equipID = rInfo.RoleEquip;

                        target.RoleList.Add(r);
                    }
                }

                target.FettersList =
battle_shop_player.check_fetters(target.RoleList);
            }

            return target;
        }

        private bool add_role(string ClientUUID, int
```

```
role_index, int index)
        {
            var BattleShopPlayerImpl =
battleShopPlayer;
            if (isTmp && tmpPVEData != null &&
tmpPVEData.TmpBattleShopPlayer != null)
            {
                BattleShopPlayerImpl =
tmpPVEData.TmpBattleShopPlayer;
            }

            var r =
BattleShopPlayerImpl.add_role(role_index, index);
            if (r != null)
            {
                if (CheckBuyRole(r))
                {
                    client_mng.PlayerClientCaller.get_client(ClientUUID).
achievement_complete(info.Achiev, info.wAchiev);
                }

                return true;
            }

            return false;
        }

        private em_error buy_role(bool isTmp, string
ClientUUID, int index, int role_index)
        {
            var BattleShopPlayerImpl =
battleShopPlayer;
            if (isTmp && tmpPVEData != null &&
tmpPVEData.TmpBattleShopPlayer != null)
            {
                BattleShopPlayerImpl =
tmpPVEData.TmpBattleShopPlayer;
            }

            var r =
```

```
BattleShopPlayerImpl.BattleData.RoleList[role_index];
            var s =
BattleShopPlayerImpl.ShopData.SaleRoleList[index];

            if (s == null)
            {
                return em_error.db_error;
            }

            if (BattleShopPlayerImpl.BattleData.coin
< s.Price)
            {
                return em_error.no_enough_coin;
            }
            BattleShopPlayerImpl.BattleData.coin -=
s.Price;

            if (r == null)
            {
                if (!add_role(ClientUUID, role_index,
index))
                {
                    return em_error.db_error;
                }
            }
            else
            {
                var err =
BattleShopPlayerImpl.merge_role(index, role_index);
                if (err != em_error.success)
                {
                    return err;
                }

                if (CheckBuyRole(r))
                {

client_mng.PlayerClientCaller.get_client(ClientUUID).
achievement_complete(info.Achiev, info.wAchiev);
                }
            }
```

```
            return em_error.success;
        }

        private em_error buy_equip(string ClientUUID,
ShopProp p, int index, int role_index)
        {
            var BattleShopPlayerImpl =
battleShopPlayer;
            if (isTmp && tmpPVEData != null &&
tmpPVEData.TmpBattleShopPlayer != null)
            {
                BattleShopPlayerImpl =
tmpPVEData.TmpBattleShopPlayer;
            }

            var err =
BattleShopPlayerImpl.buy_equip(p, index, role_index);
            if (err != em_error.success)
            {
                return err;
            }

            if (CheckBuyEquip(p.PropID))
            {
                client_mng.PlayerClientCaller.get_client(ClientUUID).
achievement_complete(info.Achiev, info.wAchiev);
            }

            return em_error.success;
        }

        public em_error buy(string ClientUUID,
ShopIndex shop_index, int index, int role_index)
        {
            var BattleShopPlayerImpl =
battleShopPlayer;
            if (isTmp && tmpPVEData != null &&
tmpPVEData.TmpBattleShopPlayer != null)
            {
```

```
                BattleShopPlayerImpl =
tmpPVEData.TmpBattleShopPlayer;
            }

            if (shop_index == ShopIndex.Role)
            {
                var result = buy_role(isTmp,
ClientUUID, index, role_index);
                if (result != em_error.success)
                {
                    return result;
                }
            }
            else if (shop_index == ShopIndex.Prop)
            {
                var p =
BattleShopPlayerImpl.ShopData.SalePropList[index];
                if (p == null)
                {
                    return em_error.db_error;
                }

                if
(BattleShopPlayerImpl.BattleData.coin < p.Price)
                {
                    return em_error.no_enough_coin;
                }
                BattleShopPlayerImpl.BattleData.coin
-= p.Price;

                if (p.PropID >=
config.Config.FoodIDMin && p.PropID <=
config.Config.FoodIDMax)
                {
                    var result =
BattleShopPlayerImpl.buy_food(p, index, role_index,
GetStage());
                    if (result != em_error.success)
                    {
                        return result;
                    }
```

```
                }
                else if (p.PropID >=
config.Config.EquipIDMin && p.PropID <=
config.Config.EquipIDMax)
                {
                    var result =
buy_equip(ClientUUID, p, index, role_index);
                    if (result != em_error.success)
                    {
                        return result;
                    }
                }
                else
                {
                    return em_error.db_error;
                }

                BattleShopPlayerImpl.ShopData.SalePropList[index] =
null;
            }
            else
            {
                return em_error.db_error;
            }

            return em_error.success;
        }
    }

    public static class AvatarExtensions
    {
        public static PlayerInfo PlayerInfo(this
Avatar avatar)
        {
            var _data =
avatar.get_real_hosting_data<PlayerInfo>();
            return _data.Data;
        }
    }
```

```
    public class client_mng
    {
        private readonly Dictionary<string,
avatar.Avatar> client_token_dict = new ();
        private readonly Dictionary<string, string>
client_sdk_uuid = new ();

        private static readonly player_client_caller
player_Client_Caller = new();
        public static player_client_caller
PlayerClientCaller
        {
            get
            {
                return player_Client_Caller;
            }
        }

        private static readonly battle_client_caller
battleClientCaller = new();
        public static battle_client_caller
BattleClientCaller
        {
            get
            {
                return battleClientCaller;
            }
        }

        private readonly AvatarMgr avatarMgr;

        public int Count
        {
            get
            {
                return avatarMgr.Count;
            }
        }

        public static async void
forward_offline_msg(OfflineMsgMgr.offline_msg
```

```
_offline_msg)
        {
            if (await
Player.offline_Msg_Mng.send_offline_msg(_offline_msg)
)
            {
                var player_guid =
long.Parse(_offline_msg.player_guid);
                var player_svr_key =
RedisHelp.BuildPlayerGuidCacheKey(player_guid);
                string player_hub_name = await
Player._redis_handle.GetStrData(player_svr_key);
                if
(string.IsNullOrEmpty(player_hub_name))
                {
                    return;
                }
                if (player_hub_name != Hub.Hub.name)
                {
                    var player_proxy =
Player.player_Proxy_Mng.get_player_proxy(player_hub_n
ame);
                    player_proxy.player_have_offline_msg(player_guid);
                }
                else
                {
                    var target_role =
Player.client_Mng.guid_get_client_proxy(player_guid);
                    if (target_role != null)
                    {
                        await
Player.offline_Msg_Mng.process_offline_msg(_offline_m
sg.player_guid);
                    }
                }
            }
        }

        public client_mng()
        {
```

```
            avatarMgr = new();
            avatarMgr.init();
            avatarMgr.init_data_db_opt((opt) =>
            {
                opt.DBName =
Constant.Constant.player_db_name;
                opt.DBCollection =
Constant.Constant.player_db_collection;
                opt.GuidCollection =
Constant.Constant.player_db_guid_collection;
                opt.StoreDelayTime = 10 * 60 * 1000;
            });
            avatarMgr.add_hosting<PlayerInfo>();

            bag.BagModdule.init(30, avatarMgr);
        }

        public async Task<Tuple<string, bool>>
token_player_login(string sdk_uuid, string
anonymous_sdk_uuid = "")
        {
            var token = Guid.NewGuid().ToString();

            avatar.Avatar _avatar = null;
            if (!
string.IsNullOrEmpty(anonymous_sdk_uuid))
            {
                _avatar = await
avatarMgr.load(anonymous_sdk_uuid);
                if (!string.IsNullOrEmpty(sdk_uuid))
                {
                    if (!await
_avatar.transfer(sdk_uuid))
                    {
                        Log.Log.err("_avatar.transfer
faild anonymous_sdk_uuid:{0, }sdk_uuid:{1}",
anonymous_sdk_uuid, sdk_uuid);
                    }
                }
            }
            else
```

```
            {
                _avatar = await
avatarMgr.load(sdk_uuid);
            }

            if (_avatar != null)
            {

                var player_svr_key =
RedisHelp.BuildPlayerGuidCacheKey(_avatar.Guid);
                await
Player._redis_handle.SetStrData(player_svr_key,
Hub.Hub.name, RedisHelp.PlayerSvrInfoCacheTimeout);

                _avatar.onDestory += () =>
                {
                    var uuid_key =
RedisHelp.BuildPlayerSDKUUIDCacheKey(_avatar.ClientUU
ID);
                    Player._redis_handle.DelData(uuid_key);

                    var player_key =
RedisHelp.BuildPlayerSvrCacheKey(_avatar.SDKUUID);
                    Player._redis_handle.DelData(player_key);

                    var player_guid_key =
RedisHelp.BuildPlayerGuidCacheKey(_avatar.Guid);
                    Player._redis_handle.DelData(player_guid_key);

                    var gate_key =
RedisHelp.BuildPlayerGateCacheKey(_avatar.SDKUUID);
                    Player._redis_handle.DelData(gate_key);
                };

                client_token_dict[token] = _avatar;

                return Tuple.Create(token, false);
```

```
            }
            else
            {
                client_sdk_uuid[token] = sdk_uuid;
            }

            return Tuple.Create(token, true);
        }

        public avatar.Avatar
token_get_client_proxy(string uuid, string token)
        {
            if (!client_token_dict.Remove(token, out
avatar.Avatar _avatar))
            {
                throw new LoginException($"invaild
token:{token}");
            }

            if (!
string.IsNullOrEmpty(_avatar.ClientUUID))
            {
                Hub.Hub._gates.disconnect_client(_avatar.ClientUUID,
"others login!");
            }

            avatarMgr.bind_avatar(_avatar, uuid);

            var uuid_key =
RedisHelp.BuildPlayerSDKUUIDCacheKey(uuid);
            Player._redis_handle.SetStrData(uuid_key,
_avatar.SDKUUID,
RedisHelp.PlayerSvrInfoCacheTimeout);

            return _avatar;
        }

        public async Task<avatar.Avatar>
create_player(string uuid, string token, string name,
string nick_name, string avatar)
```

```
        {
            if (client_sdk_uuid.Remove(token, out var
sdk_uuid))
            {
                var _avatar = await
avatarMgr.create_avatar(sdk_uuid);
                avatarMgr.bind_avatar(_avatar, uuid);
                var info =
_avatar.get_real_hosting_data<PlayerInfo>();
                info.Data.Info().User.UserName =
nick_name;
                info.Data.Info().User.UserGuid =
_avatar.Guid;
                info.Data.Info().User.Avatar =
avatar;
                _avatar.set_dirty();

                return _avatar;
            }
            return null;
        }

        public async Task<Avatar>
uuid_get_client_proxy(string uuid)
        {
            var _avatar = avatarMgr.get_avatar(uuid);
            if (_avatar != null)
            {
                return _avatar;
            }


            var uuid_key =
RedisHelp.BuildPlayerSDKUUIDCacheKey(uuid);
            var sdk_uuid = await
Player._redis_handle.GetStrData(uuid_key);
            await
Player._redis_handle.Expire(uuid_key,
RedisHelp.PlayerSvrInfoCacheTimeout);

            if (!string.IsNullOrEmpty(sdk_uuid))
```

```
            {
                var gate_key =
RedisHelp.BuildPlayerGateCacheKey(sdk_uuid);
                var gate_name = await
Player._redis_handle.GetStrData(gate_key);
                await
Player._redis_handle.Expire(gate_key,
RedisHelp.PlayerSvrInfoCacheTimeout);

                _avatar = await
avatarMgr.load_or_create(sdk_uuid, uuid);
                Hub.Hub._gates.client_seep(uuid,
gate_name);

                return _avatar;
            }

            return null;
        }

        public Avatar
sdk_uuid_get_client_proxy(string sdk_uuid)
        {
            var _avatar =
avatarMgr.get_target_avatar(sdk_uuid);
            return _avatar;
        }

        public Avatar guid_get_client_proxy(long
guid)
        {
            var _avatar =
avatarMgr.get_target_avatar(guid);
            return _avatar;
        }
    }
}

using Abelkhan;
using avatar;
using battle_shop;
```

```
using config;
using InfluxData.Net.InfluxDb.Models.Responses;
using Microsoft.Extensions.Logging;
using MongoDB.Bson;
using MsgPack;
using MsgPack.Serialization;
using StackExchange.Redis;
using System;
using System.Collections.Generic;
using System.Xml.Linq;

namespace Player
{
    public class client_msg_handle
    {
        private readonly player_login_module
player_login_Module;
        private readonly player_battle_module
player_battle_Module;
        private readonly player_shop_module
player_shop_Module;
        private readonly player_quest_module
player_quest_Module;
        private readonly battle_revive_module
battle_Revive_Module;
        private plan_module plan_Module;

        public client_msg_handle()
        {
            plan_Module = new plan_module();
            plan_Module.on_buy +=
Plan_Module_on_buy; ;
            plan_Module.on_sale_role +=
Plan_Module_on_sale_role; ;
            plan_Module.on_move +=
Plan_Module_on_move; ;
            plan_Module.on_refresh +=
Plan_Module_on_refresh;
            plan_Module.on_freeze +=
Plan_Module_on_freeze;
            plan_Module.on_get_battle_data +=
```

```
Plan_Module_on_get_battle_data;
            plan_Module.on_end_round +=
Plan_Module_on_end_round;

            battle_Revive_Module = new
battle_revive_module();

battle_Revive_Module.on_battle_failed_back +=
Battle_Revive_Module_on_battle_failed_back;

            player_login_Module = new();
            player_login_Module.on_player_login +=
Login_Player_Module_on_player_login;
            player_login_Module.on_create_role +=
Player_login_Module_on_create_role;
            player_login_Module.on_reconnect +=
Player_login_Module_on_reconnect;
            player_login_Module.on_guide_step +=
Player_login_Module_on_guide_step;

            player_battle_Module = new();
            player_battle_Module.on_start_battle +=
Player_battle_Module_on_start_battle;

player_battle_Module.on_start_peak_strength +=
Player_battle_Module_on_start_peak_strength;

player_battle_Module.on_achievement_gold25 +=
Player_battle_Module_on_achievement_gold25;
            player_battle_Module.on_kill_role +=
Player_battle_Module_on_kill_role;
            player_battle_Module.on_check_achievement
+= Player_battle_Module_on_check_achievement;

            player_quest_Module = new();

player_quest_Module.on_start_quest_shop_ready +=
Player_quest_Module_on_start_quest_shop_ready;
            player_quest_Module.on_select_quest_event
+= Player_quest_Module_on_select_quest_event;
            player_quest_Module.on_start_quest_battle
```

```
+= Player_quest_Module_on_start_quest_battle;
            player_quest_Module.on_confirm_quest_victory +=
Player_quest_Module_on_confirm_quest_victory;
            player_quest_Module.on_get_quest_shop_data +=
Player_quest_Module_on_get_quest_shop_data;
            player_quest_Module.on_check_finish_pve_level +=
Player_quest_Module_on_check_finish_pve_level;
            player_quest_Module.on_start_quest_shop_ready1 +=
Player_quest_Module_on_start_quest_shop_ready1;

            player_shop_Module = new();
            //player_shop_Module.on_buy_card_packet
+= Player_shop_Module_on_buy_card_packet;
            //player_shop_Module.on_buy_card_merge +=
Player_shop_Module_on_buy_card_merge;
            //player_shop_Module.on_edit_role_group
+= Player_shop_Module_on_edit_role_group;
            player_shop_Module.on_get_user_data +=
Player_shop_Module_on_get_user_data;
        }

        private async void
Battle_Revive_Module_on_battle_failed_back()
        {
            var rsp = battle_Revive_Module.rsp as
battle_revive_battle_failed_back_rsp;
            var uuid =
Hub.Hub._gates.current_client_uuid;

            try
            {
                var _avatar = await
Player.client_Mng.uuid_get_client_proxy(uuid);
                if (_avatar != null)
                {
                    var _data =
_avatar.get_real_hosting_data<PlayerInfo>();
```

```
                    Log.Log.trace("battleShopPlayer:
{0}", _data.Data.battleShopPlayer.ToJson());

                    var battleShopPlayer =
_data.Data.battle_failed_back();
                    if (_data.Data.isTmp &&
_data.Data.tmpPVEData != null)
                    {

_data.Data.tmpPVEData.isQuestEvent =
_data.Data.oldPVEQuestEvent;
                    }
                    else
                    {
                        _data.Data.isPVEQuestEvent =
_data.Data.oldPVEQuestEvent;
                    }

rsp.rsp(battleShopPlayer.BattleData,
battleShopPlayer.ShopData);
                }
            }
            catch (System.Exception ex)
            {

Log.Log.err("Battle_Revive_Module_on_battle_failed_ba
ck error:{0}", ex);
                rsp.err((int)em_error.db_error);
            }
        }

        private async void
Player_quest_Module_on_start_quest_shop_ready1(int
quest)
        {
            Log.Log.trace("on_start_quest_ready1
begin!");

            try
            {
                var rsp = player_quest_Module.rsp as
```

```
player_quest_start_quest_shop_ready1_rsp;
                var uuid =
Hub.Hub._gates.current_client_uuid;
                var _avatar = await
Player.client_Mng.uuid_get_client_proxy(uuid);
                if (_avatar != null)
                {
                    var _data =
_avatar.get_real_hosting_data<PlayerInfo>();
                    var (err, eventIdList) =
_data.Data.StartQuestReady1(quest, uuid,
client_mng.BattleClientCaller);

Log.Log.trace("on_start_quest_ready1 quest:{2} err:
{0} eventId List:{1}", err, eventIdList.ToJson(),
quest);
                    if (err == em_error.success)
                    {

Log.Log.trace("on_start_quest_ready1 eventId List:{0}
_data.Data.tmpPVEData.TmpBattleShopPlayer:{1}",
eventIdList.ToJson(),
_data.Data.tmpPVEData.TmpBattleShopPlayer.ToJson());

rsp.rsp(_data.Data.tmpPVEData.TmpBattleShopPlayer.Bat
tleData,
_data.Data.tmpPVEData.TmpBattleShopPlayer.ShopData,
eventIdList);
                    }
                    else
                    {
                        rsp.err((int)err);
                    }
                }
            }
            catch (System.Exception ex)
            {

Log.Log.err($"Player_quest_Module_on_start_quest_read
y1 err:{ex}");
            }
```

```
            Log.Log.trace("on_start_quest_ready1
end!");
        }

        private async void
Player_quest_Module_on_check_finish_pve_level()
        {
            var rsp = player_quest_Module.rsp as
player_quest_check_finish_pve_level_rsp;
            var uuid =
Hub.Hub._gates.current_client_uuid;

            try
            {
                var _avatar = await
Player.client_Mng.uuid_get_client_proxy(uuid);
                if (_avatar != null)
                {
                    var _data =
_avatar.get_real_hosting_data<PlayerInfo>();
                    var is_not_finish_pve_level =
_data.Data.CheckNotFinishPVELevel();
                    rsp.rsp(!
is_not_finish_pve_level);
                }
            }
            catch (System.Exception ex)
            {

Log.Log.err("Player_quest_Module_on_check_finish_pve_
level error:{0}", ex);
                rsp.err((int)em_error.db_error);
            }
        }

        private async void Plan_Module_on_end_round()
        {
            var rsp = plan_Module.rsp as
plan_end_round_rsp;
            var uuid =
```

```
Hub.Hub._gates.current_client_uuid;

            try
            {
                var _avatar = await
Player.client_Mng.uuid_get_client_proxy(uuid);
                var _data =
_avatar.get_real_hosting_data<PlayerInfo>();
                _data.Data.EndPVERound();
                rsp.rsp();
            }
            catch (System.Exception ex)
            {
                Log.Log.err("Plan_Module_on_end_round
error:{0}", ex);
                rsp.err((int)em_error.db_error);
            }
        }

        private async void
Plan_Module_on_get_battle_data()
        {
            var rsp = plan_Module.rsp as
plan_get_battle_data_rsp;
            var uuid =
Hub.Hub._gates.current_client_uuid;

            try
            {
                var _avatar = await
Player.client_Mng.uuid_get_client_proxy(uuid);
                var _data =
_avatar.get_real_hosting_data<PlayerInfo>();

                var BattleShopPlayer =
_data.Data.battleShopPlayer;
                if (_data.Data.tmpPVEData != null &&
_data.Data.tmpPVEData.TmpBattleShopPlayer != null)
                {
                    BattleShopPlayer =
_data.Data.tmpPVEData.TmpBattleShopPlayer;
```

```
                }
                rsp.rsp(BattleShopPlayer.BattleData,
BattleShopPlayer.ShopData,
BattleShopPlayer.check_fetters());
            }
            catch (System.Exception ex)
            {

Log.Log.err("Plan_Module_on_get_battle_data error:
{0}", ex);
                rsp.err();
            }
        }

        private async void
Plan_Module_on_freeze(ShopIndex shop_index, int
index, bool is_freeze)
        {
            var rsp = plan_Module.rsp as
plan_freeze_rsp;
            var uuid =
Hub.Hub._gates.current_client_uuid;

            try
            {
                var _avatar = await
Player.client_Mng.uuid_get_client_proxy(uuid);
                var _data =
_avatar.get_real_hosting_data<PlayerInfo>();

                var BattleShopPlayer =
_data.Data.battleShopPlayer;
                if (_data.Data.tmpPVEData != null &&
_data.Data.tmpPVEData.TmpBattleShopPlayer != null)
                {
                    BattleShopPlayer =
_data.Data.tmpPVEData.TmpBattleShopPlayer;
                }
                BattleShopPlayer.freeze(shop_index,
index, is_freeze);
                rsp.rsp(BattleShopPlayer.ShopData);
```

```
            }
            catch (System.Exception ex)
            {
                Log.Log.err("Plan_Module_on_freeze
error:{0}", ex);
                rsp.err((int)em_error.db_error);
            }
        }

        private async void
Player_quest_Module_on_get_quest_shop_data()
        {
            Log.Log.trace("on_get_quest_shop_data
begin!");

            try
            {
                var rsp = player_quest_Module.rsp as
player_quest_get_quest_shop_data_rsp;
                var uuid =
Hub.Hub._gates.current_client_uuid;
                var _avatar = await
Player.client_Mng.uuid_get_client_proxy(uuid);
                if (_avatar != null)
                {
                    var _data =
_avatar.get_real_hosting_data<PlayerInfo>();

                    var BattleShopPlayer =
_data.Data.battleShopPlayer;
                    var PVELevelCfgImpl =
_data.Data.pveLevelCfg;
                    var PVELevelIndexImpl =
_data.Data.Info().PVELevelIndex;
                    if (_data.Data.isTmp &&
_data.Data.tmpPVEData != null &&
_data.Data.tmpPVEData.TmpBattleShopPlayer != null)
                    {
                        BattleShopPlayer =
_data.Data.tmpPVEData.TmpBattleShopPlayer;
                        PVELevelCfgImpl =
```

```
_data.Data.tmpPVEData.TmpPVELevelCfg;
                        PVELevelIndexImpl =
_data.Data.tmpPVEData.TmpPVELevelIndex;
                    }
                    if (PVELevelIndexImpl <
PVELevelCfgImpl.Level.Count)
                    {
                        var roles = new
List<QuestRogueRole>();
                        if
(_data.Data.RogueRoleList.Count >= 3)
                        {
                            while (roles.Count < 3)
                            {
                            break1:
                                var index =
RandomHelper.RandomInt(_data.Data.RogueRoleList.Count
);
                                var r =
_data.Data.RogueRoleList[index];
                                foreach (var _role in
roles)
                                {
                                    if (_role.index
== index)
                                    {
                                        goto break1;
                                    }
                                }

                                var _roleRougue = new
QuestRogueRole()
                                {
                                    r = r,
                                    index = index
                                };

roles.Add(_roleRougue);
                            }
                        }
                        else
```

```
                    {
                        if
(config.Config.PVERoundConfigs.TryGetValue(PVELevelCf
gImpl.Level[PVELevelIndexImpl], out var roundcfg))
                        {
                            for (int index = 0;
index < roundcfg.EventID.Count; index++)
                            {
                                var eventId =
roundcfg.EventID[index];
                                if
(config.Config.PVEEventConfigs.TryGetValue(eventId,
out var ecfg))
                                {
                                    if
(config.Config.RoleConfigs.TryGetValue(ecfg.RoleID,
out RoleConfig rcfg))
                                    {
                                        var r =
new Abelkhan.Role();
                                        r.RoleID
= ecfg.RoleID;
                                        r.Level =
ecfg.RoleLevel;
                                        r.SkillID
= rcfg.SkillID;
                                        r.HP =
ecfg.RoleHP;
                                        r.Attack
= ecfg.RoleAttack;
                                        r.equipID
= ecfg.RoleEquip;

                                        var
_roleRougue = new QuestRogueRole()
                                        {
                                            r =
r,
                                            index
= index
                                        };
```

```
roles.Add(_roleRougue);
                                    }
                                }
                            }
                        }
                    }
rsp.rsp(BattleShopPlayer.BattleData,
BattleShopPlayer.ShopData, roles);
                }
                else
                {
rsp.err((int)em_error.not_exist_quest_level);
                }
            }
        }
        catch (System.Exception ex)
        {
Log.Log.err($"Player_quest_Module_on_get_quest_shop_d
ata err:{ex}");
        }

        Log.Log.trace("on_get_quest_shop_data
end!");
    }

    private async void
Player_quest_Module_on_confirm_quest_victory(BattleVi
ctory is_victory, int addCoin)
    {
        Log.Log.trace("on_confirm_quest_victory
begin!");

        try
        {
            var rsp = player_quest_Module.rsp as
player_quest_confirm_quest_victory_rsp;
            var uuid =
```

```
Hub.Hub._gates.current_client_uuid;

                var _avatar = await
Player.client_Mng.uuid_get_client_proxy(uuid);
                if (_avatar != null)
                {
                    var _data =
_avatar.get_real_hosting_data<PlayerInfo>();

                    var BattleShopPlayer =
_data.Data.battleShopPlayer;
                    var pveLevelCfg =
_data.Data.pveLevelCfg;
                    var PVELevelIndex =
_data.Data.Info().PVELevelIndex;
                    if (_data.Data.isTmp &&
_data.Data.tmpPVEData != null &&
_data.Data.tmpPVEData.TmpBattleShopPlayer != null)
                    {
                        BattleShopPlayer =
_data.Data.tmpPVEData.TmpBattleShopPlayer;
                        pveLevelCfg =
_data.Data.tmpPVEData.TmpPVELevelCfg;
                        PVELevelIndex =
_data.Data.tmpPVEData.TmpPVELevelIndex;
                    }

                    if (BattleShopPlayer == null)
                    {
                        return;
                    }

                    if (_data.Data.isTmp &&
_data.Data.tmpPVEData != null)
                    {

_data.Data.tmpPVEData.isInitRound = false;
                    }
                    else
                    {
                        _data.Data.isInitRound =
```

```
false;
                    }

                    BattleShopPlayer.lastBattleResults = is_victory;
                    if (is_victory ==
BattleVictory.faild)
                    {
                        if
(BattleShopPlayer.BattleData.faild > 0)
                        {
                            BattleShopPlayer.BattleData.faild--;
                        }
                    }
                    else if (is_victory ==
BattleVictory.victory)
                    {
                        BattleShopPlayer.BattleData.victory++;
                        PVELevelIndex++;
                    }

                    if
(BattleShopPlayer.BattleData.faild > 0)
                    {
                        if (PVELevelIndex >=
pveLevelCfg.Level.Count)
                        {
                            _data.Data.AddRogueRole(BattleShopPlayer.UnlockRoles)
;
                            PVELevelIndex = 0;

                            if (!_data.Data.isTmp ||
_data.Data.tmpPVEData == null ||
_data.Data.tmpPVEData.TmpBattleShopPlayer == null)
                            {
                                _data.Data.Info().quest++;
                            }
```

```
Log.Log.trace("on_confirm_quest_victory quest:{0}",
_data.Data.Info().quest);
                                if (_data.Data.isTmp &&
_data.Data.tmpPVEData != null)
                                {
rsp.rsp(em_quest_state.finish_tmp_level);
_data.Data.ClearTmpPVEState();
                                }
                                else
                                {
                                    if
(_data.Data.CheckNotFinishPVELevel())
                                    {
rsp.rsp(em_quest_state.next_quest);
_data.Data.oldPVEQuestEvent =
_data.Data.isPVEQuestEvent;
_data.Data.isPVEQuestEvent = false;
                                    }
                                    else
                                    {
rsp.rsp(em_quest_state.finish_pve_level);
                                    }
                                }

                                var rank_Info = new
UserRankInfo
                                {
                                    nick_name =
_data.Data.Info().User.UserName,
                                    avatar =
_data.Data.Info().User.Avatar,
                                    score =
_data.Data.Info().score +
```

```
BattleShopPlayer.BattleData.victory,
                                    };

                                    var battleInfo = new
BattleInfo
                                    {
                                        mod =
BattleMod.PeakStrength,
                                        isVictory =
is_victory,
                                        isStreakVictory =
false,
                                        RoleList =
_data.Data.Info().RoleList,
                                    };
                                    rank_Info.battle_data =
new UserSharedData()
                                    {
                                        User =
BattleShopPlayer.BattleData.User,
                                        round =
BattleShopPlayer.BattleData.round + 1,
                                        victory =
BattleShopPlayer.BattleData.victory,
                                        faild =
pveLevelCfg.Hp - BattleShopPlayer.BattleData.faild,
                                    };

                                    using var st =
MemoryStreamPool.mstMgr.GetStream();
                                    var _serializer =
MessagePackSerializer.Get<MessagePackObjectDictionary
>();
                                    _serializer.Pack(st,
UserRankInfo.UserRankInfo_to_protcol(rank_Info));
                                    var r = new rank_item
                                    {
                                        guid =
_data.Data.Info().User.UserGuid,
                                        score =
rank_Info.score,
```

```
                                    item = st.ToArray()
                                };

Player._rank_caller.get_hub("rank").update_rank_item(
Constant.Constant.player_db_guid_rank, r);
                            }
                            else
                            {

rsp.rsp(em_quest_state.next_level);

_data.Data.StartPVERound(addCoin);
                            }
                        }
                        else
                        {
                            if (_data.Data.isTmp &&
_data.Data.tmpPVEData != null)
                            {

_data.Data.oldPVEQuestEvent =
_data.Data.tmpPVEData.isQuestEvent;

_data.Data.tmpPVEData.isQuestEvent = false;
                            }
                            else
                            {

_data.Data.oldPVEQuestEvent =
_data.Data.isPVEQuestEvent;

_data.Data.isPVEQuestEvent = false;
                            }


rsp.rsp(em_quest_state.failed);
                        }

                        if (_data.Data.isTmp &&
_data.Data.tmpPVEData != null &&
_data.Data.tmpPVEData.TmpBattleShopPlayer != null)
```

```
                    {
                    _data.Data.tmpPVEData.TmpPVELevelIndex =
PVELevelIndex;
                    }
                    else
                    {
                    _data.Data.Info().PVELevelIndex = PVELevelIndex;
                    }

                    BattleShopPlayer.BattleData.round++;
                }
            }
            catch (System.Exception ex)
            {
                Log.Log.err("Player_quest_Module_on_confirm_quest_vic
tory error:{0}", ex);
            }

            Log.Log.trace("on_confirm_quest_victory
end!");
        }

        private async void Plan_Module_on_refresh()
        {
            Log.Log.trace("on_refresh begin!");

            try
            {
                var rsp = plan_Module.rsp as
plan_refresh_rsp;
                var uuid =
Hub.Hub._gates.current_client_uuid;

                var _avatar = await
Player.client_Mng.uuid_get_client_proxy(uuid);
                if (_avatar != null)
                {
```

```
                    var _data =
_avatar.get_real_hosting_data<PlayerInfo>();
                    if (!
_data.Data.refresh(_data.Data.GetStage()))
                    {
                        rsp.err((int)em_error.no_enough_coin);
                    }
                    else
                    {
                        var BattleShopPlayer =
_data.Data.battleShopPlayer;
                        if (_data.Data.isTmp &&
_data.Data.tmpPVEData != null &&
_data.Data.tmpPVEData.TmpBattleShopPlayer != null)
                        {
                            BattleShopPlayer =
_data.Data.tmpPVEData.TmpBattleShopPlayer;
                        }
                        BattleShopPlayer.do_skill(_data.Data.GetStage());
                        rsp.rsp(BattleShopPlayer.ShopData,
BattleShopPlayer.BattleData);
                    }
                }
            }
            catch (System.Exception ex)
            {
                Log.Log.err("Plan_Module_on_move
error:{0}", ex);
            }

            Log.Log.trace("Plan_Module_on_refresh
end!");
        }

        private async void Plan_Module_on_move(int
role_index1, int role_index2)
        {
            Log.Log.trace("on_move begin!");
```

```
            try
            {
                var rsp = plan_Module.rsp as
plan_move_rsp;
                var uuid =
Hub.Hub._gates.current_client_uuid;

                var _avatar = await
Player.client_Mng.uuid_get_client_proxy(uuid);
                if (_avatar != null)
                {
                    var _data =
_avatar.get_real_hosting_data<PlayerInfo>();

                    var BattleShopPlayer =
_data.Data.battleShopPlayer;
                    if (_data.Data.isTmp &&
_data.Data.tmpPVEData != null &&
_data.Data.tmpPVEData.TmpBattleShopPlayer != null)
                    {
                        BattleShopPlayer =
_data.Data.tmpPVEData.TmpBattleShopPlayer;
                    }

BattleShopPlayer.move(role_index1, role_index2);

rsp.rsp(BattleShopPlayer.BattleData);
                }
            }
            catch (System.Exception ex)
            {
                Log.Log.err("Plan_Module_on_move
error:{0}", ex);
            }

            Log.Log.trace("on_move end!");
        }

        private async void
Plan_Module_on_sale_role(int index)
```

```
        {
            Log.Log.trace("on_sale_role begin!");

            try
            {
                var rsp = plan_Module.rsp as
plan_sale_role_rsp;
                var uuid =
Hub.Hub._gates.current_client_uuid;

                var _avatar = await
Player.client_Mng.uuid_get_client_proxy(uuid);
                if (_avatar != null)
                {
                    var _data =
_avatar.get_real_hosting_data<PlayerInfo>();
                    var stage =
_data.Data.GetStage();

                    var BattleShopPlayer =
_data.Data.battleShopPlayer;
                    if (_data.Data.isTmp &&
_data.Data.tmpPVEData != null &&
_data.Data.tmpPVEData.TmpBattleShopPlayer != null)
                    {
                        BattleShopPlayer =
_data.Data.tmpPVEData.TmpBattleShopPlayer;
                    }
                    if
(BattleShopPlayer.sale_role(index, stage))
                    {

rsp.rsp(BattleShopPlayer.BattleData);
                    }
                    else
                    {

rsp.err((int)em_error.no_role_in_index_to_sale);
                    }
                }
```

```
            }
            catch (System.Exception ex)
            {
                Log.Log.err($"Plan_Module_on_sale_role err:{ex}");
            }

            Log.Log.trace("on_sale_role end!");
        }

        private async void Plan_Module_on_buy(ShopIndex shop_index, int index, int role_index)
        {
            Log.Log.trace("on_buy begin!");

            try
            {
                var rsp = plan_Module.rsp as plan_buy_rsp;
                var uuid = Hub.Hub._gates.current_client_uuid;

                var _avatar = await Player.client_Mng.uuid_get_client_proxy(uuid);
                if (_avatar != null)
                {
                    var _data = _avatar.get_real_hosting_data<PlayerInfo>();
                    var err = _data.Data.buy(_avatar.ClientUUID, shop_index, index, role_index);
                    if (err != em_error.success)
                    {
                        rsp.err((int)err);
                        return;
                    }

                    var BattleShopPlayer = _data.Data.battleShopPlayer;
                    if (_data.Data.isTmp &&
```

```
_data.Data.tmpPVEData != null &&
_data.Data.tmpPVEData.TmpBattleShopPlayer != null)
                    {
                        BattleShopPlayer =
_data.Data.tmpPVEData.TmpBattleShopPlayer;
                    }

BattleShopPlayer.do_skill(_data.Data.GetStage());

rsp.rsp(BattleShopPlayer.BattleData,
BattleShopPlayer.ShopData);
                }
            }
            catch (System.Exception ex)
            {
                Log.Log.err($"Plan_Module_on_buy err:
{ex}");
            }

            Log.Log.trace("on_buy end!");
        }

        private async void
Player_quest_Module_on_start_quest_battle()
        {
            Log.Log.trace("on_start_quest_battle
begin!");

            try
            {
                var rsp = player_quest_Module.rsp as
player_quest_start_quest_battle_rsp;
                var uuid =
Hub.Hub._gates.current_client_uuid;
                var _avatar = await
Player.client_Mng.uuid_get_client_proxy(uuid);
                if (_avatar != null)
                {
                    var _data =
_avatar.get_real_hosting_data<PlayerInfo>();
                    var BattleShopPlayer =
```

```
_data.Data.battleShopPlayer;
                    if (_data.Data.isTmp &&
_data.Data.tmpPVEData != null &&
_data.Data.tmpPVEData.TmpBattleShopPlayer != null)
                    {
                        BattleShopPlayer =
_data.Data.tmpPVEData.TmpBattleShopPlayer;
                    }

rsp.rsp(BattleShopPlayer.BattleData,
_data.Data.StartQuestBattle());
                }
            }
            catch (System.Exception ex)
            {

Log.Log.err($"Player_quest_Module_on_start_quest_batt
le err:{ex}");
            }

            Log.Log.trace("on_start_quest_battle
end!");
        }

        private async void
Player_quest_Module_on_select_quest_event(int index)
        {
            Log.Log.trace("on_start_quest_shop
begin!");

            try
            {
                var rsp = player_quest_Module.rsp as
player_quest_select_quest_event_rsp;
                var uuid =
Hub.Hub._gates.current_client_uuid;
                var _avatar = await
Player.client_Mng.uuid_get_client_proxy(uuid);
                if (_avatar != null)
                {
                    var _data =
```

```
_avatar.get_real_hosting_data<PlayerInfo>();
                    _data.Data.StartQuestShop(index);
                    var BattleShopPlayer =
_data.Data.battleShopPlayer;
                    if (_data.Data.isTmp &&
_data.Data.tmpPVEData != null &&
_data.Data.tmpPVEData.TmpBattleShopPlayer != null)
                    {
                        BattleShopPlayer =
_data.Data.tmpPVEData.TmpBattleShopPlayer;
                    }

rsp.rsp(BattleShopPlayer.BattleData,
BattleShopPlayer.ShopData);
                }
            }
            catch (System.Exception ex)
            {

Log.Log.err($"Player_quest_Module_on_select_quest_eve
nt err:{ex}");
            }

            Log.Log.trace("on_start_quest_shop
end!");
        }

        private async void
Player_quest_Module_on_start_quest_shop_ready()
        {
            Log.Log.trace("on_start_quest_ready
begin!");

            try
            {
                var rsp = player_quest_Module.rsp as
player_quest_start_quest_shop_ready_rsp;
                var uuid =
Hub.Hub._gates.current_client_uuid;
                var _avatar = await
Player.client_Mng.uuid_get_client_proxy(uuid);
```

```
                if (_avatar != null)
                {
                    var _data =
_avatar.get_real_hosting_data<PlayerInfo>();
                    var (err, eventIdList) =
_data.Data.StartQuestReady(uuid,
client_mng.BattleClientCaller);

Log.Log.trace("on_start_quest_ready quest:{2} err:{0}
eventId List:{1}", err, eventIdList,
_data.Data.Info().quest);
                    if (err == em_error.success)
                    {

Log.Log.trace("on_start_quest_ready eventId List:
{0}", eventIdList.ToJson());
                        var BattleShopPlayer =
_data.Data.battleShopPlayer;
                        /*if (_data.Data.isTmp &&
_data.Data.tmpPVEData != null &&

_data.Data.tmpPVEData.TmpBattleShopPlayer != null)
                        {
                            BattleShopPlayer =
_data.Data.tmpPVEData.TmpBattleShopPlayer;
                        }*/

rsp.rsp(BattleShopPlayer.BattleData,
BattleShopPlayer.ShopData, eventIdList);
                    }
                    else
                    {
                        rsp.err((int)err);
                    }
                }
            }
            catch (System.Exception ex)
            {

Log.Log.err($"Player_quest_Module_on_start_quest_read
y err:{ex}");
```

```
            }

            Log.Log.trace("on_start_quest_ready
end!");
        }

        private async void
Player_login_Module_on_guide_step(GuideStep step)
        {
            Log.Log.trace("on_guide_step begin!");

            try
            {
                var uuid =
Hub.Hub._gates.current_client_uuid;
                var _avatar = await
Player.client_Mng.uuid_get_client_proxy(uuid);
                if (_avatar != null)
                {
                    var _data =
_avatar.get_real_hosting_data<PlayerInfo>();
                    _data.Data.UpdateGuideStep(step);
                }
            }
            catch (System.Exception ex)
            {
                Log.Log.err($"Player_login_Module_on_guide_step err:
{ex}");
            }

            Log.Log.trace("on_guide_step end!");
        }

        private async void
Player_battle_Module_on_kill_role(Abelkhan.Role
roleInfo)
        {
            Log.Log.trace("on_kill_role begin!");

            try
```

```
            {
                var uuid =
Hub.Hub._gates.current_client_uuid;
                var _avatar = await
Player.client_Mng.uuid_get_client_proxy(uuid);
                if (_avatar != null)
                {
                    var _data =
_avatar.get_real_hosting_data<PlayerInfo>();
                    if
(_data.Data.CheckKillRole(roleInfo))
                    {
                        
client_mng.PlayerClientCaller.get_client(_avatar.Clie
ntUUID).achievement_complete(_data.Data.Info().Achiev
, _data.Data.Info().wAchiev);
                    }
                }
            }
            catch (System.Exception ex)
            {
                Log.Log.err($"on_kill_role err:
{ex}");
            }

            Log.Log.trace("on_kill_role end!");
        }

        private async void
Player_battle_Module_on_check_achievement(Achievement
achievement)
        {
            Log.Log.trace("on_check_achievement
begin!");

            var rsp = player_battle_Module.rsp as
player_battle_check_achievement_rsp;
            var uuid =
Hub.Hub._gates.current_client_uuid;

            try
```

```
            {
                var _avatar = await
Player.client_Mng.uuid_get_client_proxy(uuid);
                if (_avatar != null)
                {
                    var _data =
_avatar.get_real_hosting_data<PlayerInfo>();
                    if
(_data.Data.CheckGetAchievementReward(achievement))
                    {
                        var reward = new
AchievementReward();
                        if
(config.Config.TaskConfigs.TryGetValue(Enum.GetName(t
ypeof(Achievement), achievement), out var task))
                        {
                            reward.gold =
task.RewardGold;
                        }
                        _data.Data.Info().gold +=
task.RewardGold;
                        rsp.rsp(reward);
                    }
                    else
                    {
                        rsp.err((int)em_error.not_complete_achievement);
                    }
                }
            }
            catch (System.Exception ex)
            {
                Log.Log.err($"Player_shop_Module_on_get_user_data
err:{ex}");
                rsp.err((int)em_error.db_error);
            }

            Log.Log.trace("on_check_achievement
end!");
        }
```

```
        private async void
Player_battle_Module_on_achievement_gold25()
        {
            Log.Log.trace("on_achievement_gold25
begin!");

            var uuid =
Hub.Hub._gates.current_client_uuid;

            try
            {
                var _avatar = await
Player.client_Mng.uuid_get_client_proxy(uuid);
                if (_avatar != null)
                {
                    var _data =
_avatar.get_real_hosting_data<PlayerInfo>();
                    var a =
_data.Data.GetAchievementData(Achievement.EMGold25);
                    if (a.status !=
AchievementAwardStatus.EMRecv)
                    {
                        a.status =
AchievementAwardStatus.EMComplete;

client_mng.PlayerClientCaller.get_client(_avatar.Clie
ntUUID).achievement_complete(_data.Data.Info().Achiev
, _data.Data.Info().wAchiev);
                    }
                }
            }
            catch (System.Exception ex)
            {

Log.Log.err($"Player_battle_Module_on_achievement_gol
d25 err:{ex}");
            }

            Log.Log.trace("on_achievement_gold25
end!");
```

```
        }

        private async void
Player_shop_Module_on_get_user_data()
        {
            Log.Log.trace("on_get_user_data begin!");

            var rsp = player_shop_Module.rsp as
player_shop_get_user_data_rsp;
            var uuid =
Hub.Hub._gates.current_client_uuid;

            try
            {
                var _avatar = await
Player.client_Mng.uuid_get_client_proxy(uuid);
                if (_avatar != null)
                {
                    var _data =
_avatar.get_real_hosting_data<PlayerInfo>();
                    rsp.rsp(_data.Data.Info());
                }
            }
            catch (System.Exception ex)
            {

Log.Log.err($"Player_shop_Module_on_get_user_data
err:{ex}");
                rsp.err((int)em_error.db_error);
            }
        }

        //private async void
Player_shop_Module_on_edit_role_group(RoleGroup
_group)
        //{
        //    Log.Log.trace("on_edit_role_group
begin!");

        //    var rsp = player_shop_Module.rsp as
player_shop_edit_role_group_rsp;
```

```
        //    var uuid =
Hub.Hub._gates.current_client_uuid;

        //    try
        //    {
        //        var _avatar = await
Player.client_Mng.uuid_get_client_proxy(uuid);
        //        if (_avatar != null)
        //        {
        //            var _data =
_avatar.get_real_hosting_data<PlayerInfo>();
        //            var err =
_data.Data.EditRoleGroup(_group);
        //            if (err == 0)
        //            {
        //                rsp.rsp(_data.Data.Info());
        //            }
        //            else
        //            {
        //                rsp.err((int)err);
        //            }
        //        }
        //    }
        //    catch (System.Exception ex)
        //    {
        //
Log.Log.err($"Player_shop_Module_on_edit_role_group
err:{ex}");
        //        rsp.err((int)em_error.db_error);
        //    }
        //}

        //private async void
Player_shop_Module_on_buy_card_packet()
        //{
        //    Log.Log.trace("on_buy_card_packet
begin!");

        //    var rsp = player_shop_Module.rsp as
player_shop_buy_card_packet_rsp;
        //    var uuid =
```

```
Hub.Hub._gates.current_client_uuid;

        //    try
        //    {
        //        var _avatar = await
Player.client_Mng.uuid_get_client_proxy(uuid);
        //        if (_avatar != null)
        //        {
        //            var _data =
_avatar.get_real_hosting_data<PlayerInfo>();
        //            var (err, packet) =
_data.Data.BuyCardPacket();
        //            if (err == 0)
        //            {
        //                rsp.rsp(packet,
_data.Data.Info().bag);
        //            }
        //            else
        //            {
        //                rsp.err((int)err);
        //            }
        //        }
        //    }
        //    catch (System.Exception ex)
        //    {
        //
Log.Log.err($"Player_shop_Module_on_buy_card_packet
err:{ex}");
        //        rsp.err((int)em_error.db_error);
        //    }
        //}

        //private async void
Player_shop_Module_on_buy_card_merge(int _roleID)
        //{
        //    Log.Log.trace("on_buy_card_merge
begin!");

        //    var rsp = player_shop_Module.rsp as
player_shop_buy_card_merge_rsp;
        //    var uuid =
```

```
Hub.Hub._gates.current_client_uuid;

        //    try
        //    {
        //        var _avatar = await
Player.client_Mng.uuid_get_client_proxy(uuid);
        //        if (_avatar != null)
        //        {
        //            var _data =
_avatar.get_real_hosting_data<PlayerInfo>();
        //            var err =
_data.Data.BuyCardMerge(_roleID);
        //            if (err == 0)
        //            {
        //                rsp.rsp(_roleID,
_data.Data.Info());
        //            }
        //            else
        //            {
        //                rsp.err((int)err);
        //            }
        //        }
        //    }
        //    catch (System.Exception ex)
        //    {
        //
Log.Log.err($"Player_shop_Module_on_buy_card_merge
err:{ex}");
        //        rsp.err((int)em_error.db_error);
        //    }
        //}

        private async void
Player_battle_Module_on_start_peak_strength()
        {
            Log.Log.trace("on_start_battle begin!");

            var rsp = player_battle_Module.rsp as
player_battle_start_peak_strength_rsp;
            var uuid =
Hub.Hub._gates.current_client_uuid;
```

```
            try
            {
                var _avatar = await
Player.client_Mng.uuid_get_client_proxy(uuid);
                if (_avatar != null)
                {
                    var peakStrengthID = await
Player._redis_handle.GetData<long>(RedisHelp.BuildPea
kStrengthID());
                    //if (peakStrengthID !=
_avatar.PlayerInfo().PeakStrengthID)
                    {
                    //
_avatar.PlayerInfo().PeakStrengthID = peakStrengthID;
                        Player._redis_handle.DelData(RedisHelp.BuildPlayerPea
kStrengthFormationCache(_avatar.Guid));
                    }

                    var _match =
Player.match_Proxy_Mng.get_match_proxy();
                    _match.start_peak_strength(uuid,
_avatar.Guid).callBack(async (battleData) =>
                    {
                        rsp.rsp(_match.name,
battleData);

                        var match_key =
RedisHelp.BuildPlayerMatchSvrCache(_avatar.Guid);
                        await
Player._redis_handle.SetStrData(match_key,
_match.name, RedisHelp.PlayerMatchSvrCacheTimeout);

                    }, rsp.err);
                }
            }
            catch (System.Exception ex)
            {
                Log.Log.err($"Player_battle_Module_on_start_battle:
```

```
err{ex}");
                rsp.err((int)em_error.db_error);
            }
        }

        private async void
Player_battle_Module_on_start_battle(BattleMod mod)
        {
            Log.Log.trace("on_start_battle begin!");

            var rsp = player_battle_Module.rsp as
player_battle_start_battle_rsp;
            var uuid =
Hub.Hub._gates.current_client_uuid;

            try
            {
                var _avatar = await
Player.client_Mng.uuid_get_client_proxy(uuid);
                if (_avatar != null)
                {
                    var _match =
Player.match_Proxy_Mng.get_match_proxy();
                    _match.start_battle(mod, uuid,
_avatar.PlayerInfo().BattleRoleGroup(),
_avatar.PlayerInfo().Info().User).callBack(async
(battle, shop) =>
                    {
                        rsp.rsp(_match.name, battle,
shop);

                        var match_key =
RedisHelp.BuildPlayerMatchSvrCache(_avatar.Guid);
                        await
Player._redis_handle.SetStrData(match_key,
_match.name, RedisHelp.PlayerMatchSvrCacheTimeout);

                    }, rsp.err);
                }
            }
            catch (System.Exception ex)
```

```
            {
                Log.Log.err($"Player_battle_Module_on_start_battle:
err{ex}");
                rsp.err((int)em_error.db_error);
            }
        }

        private async void
Player_login_Module_on_create_role(string token,
string name, string nick_name, string avatar)
        {
            Log.Log.trace("on_player_login begin!");

            var rsp = player_login_Module.rsp as
player_login_create_role_rsp;
            var uuid =
Hub.Hub._gates.current_client_uuid;

            try
            {
                var _avatar = await
Player.client_Mng.create_player(uuid, token, name,
nick_name, avatar);
                if (_avatar != null)
                {
                    rsp.rsp(_avatar.PlayerInfo().Info());
                }
                else
                {
                    rsp.err((int)em_error.token_out_time);
                }
            }
            catch (LoginException ex)
            {
                Log.Log.err(ex.ErrorInfo);
                Hub.Hub._gates.disconnect_client(uuid, "system
error!");
```

```
            }
            catch (System.Exception ex)
            {
                Log.Log.err($"Player_login_Module_on_create_role err:
{ex}");
                rsp.err((int)em_error.db_error);
            }
        }

        private async void
Player_login_Module_on_reconnect(long guid)
        {
            Log.Log.trace("on_reconnect begin!");

            var rsp = player_login_Module.rsp as
player_login_reconnect_rsp;
            var uuid =
Hub.Hub._gates.current_client_uuid;

            try
            {
                var _avatar =
Player.client_Mng.guid_get_client_proxy(guid);
                if (_avatar != null)
                {
                    var match_key =
RedisHelp.BuildPlayerMatchSvrCache(_avatar.Guid);
                    string match_name = await
Player._redis_handle.GetStrData(match_key);

                    match_proxy _match = null;
                    if (!
string.IsNullOrEmpty(match_name))
                    {
                        _match =
Player.match_Proxy_Mng.get_match_proxy(match_name);
                    }

                    if (_match != null)
                    {
```

```
                                    _match.reconnect(_avatar.ClientUUID, uuid).callBack((is_online) =>
                                    {
                                        if (is_online)
                                        {
                                            rsp.rsp(_avatar.PlayerInfo().Info(), match_name);
                                        }
                                        else
                                        {
                                            rsp.rsp(_avatar.PlayerInfo().Info(), "");
                                        }
                                    }, () =>
                                    {
                                        Log.Log.err("match reconnect error!");
                                    }).timeout(3000, () =>
                                    {
                                        Log.Log.err("match reconnect timeout!");
                                    });
                                }
                                else
                                {
                                    rsp.rsp(_avatar.PlayerInfo().Info(), "");
                                }
                                _avatar.ClientUUID = uuid;
                            }
                            else
                            {
                                rsp.err((int)em_error.player_offline);
                            }
                        }
                        catch (System.Exception ex)
                        {
                            Log.Log.err($"{ex}");
```

```
                rsp.err((int)em_error.db_error);
            }
        }

        private async void
Login_Player_Module_on_player_login(string token,
string name, string avatar)
        {
            Log.Log.trace("on_player_login begin!");

            var rsp = player_login_Module.rsp as
player_login_player_login_rsp;
            var uuid =
Hub.Hub._gates.current_client_uuid;

            try
            {
                var _avatar =
Player.client_Mng.token_get_client_proxy(uuid,
token);
                if (_avatar != null)
                {
                    var _data =
_avatar.get_real_hosting_data<PlayerInfo>();
                    var isNewName =
_data.Data.Info().User.UserName != name;
                    if (isNewName)
                    {

_data.Data.Info().User.UserName = name;
                    }
                    var isNewAvatar =
_data.Data.Info().User.Avatar != avatar;
                    if (isNewAvatar)
                    {
                        _data.Data.Info().User.Avatar
= avatar;
                    }

rsp.rsp(_avatar.PlayerInfo().Info());
                    await
```

```
Player.offline_Msg_Mng.process_offline_msg(_avatar.Gu
id.ToString());

                    if (isNewName || isNewAvatar)
                    {
                        var rank_Info = new
UserRankInfo
                        {
                            nick_name = name,
                            avatar = avatar,
                        };

                        using var st =
MemoryStreamPool.mstMgr.GetStream();
                        var _serializer =
MessagePackSerializer.Get<MessagePackObjectDictionary
>();
                        _serializer.Pack(st,
UserRankInfo.UserRankInfo_to_protcol(rank_Info));
                        var r = new rank_item
                        {
                            guid =
_data.Data.Info().User.UserGuid,
                            score = rank_Info.score,
                            item = st.ToArray()
                        };
                        
Player._rank_caller.get_hub("rank").update_rank_item(
Constant.Constant.player_db_guid_rank, r);
                    }
                }
                else
                {
                    
rsp.err((int)em_error.unregistered_palyer);
                }
            }
            catch (LoginException ex)
            {
                Log.Log.err(ex.ErrorInfo);
```

```
Hub.Hub._gates.disconnect_client(uuid, "login
error!");

                rsp.err((int)em_error.db_error);
            }
            catch (System.Exception ex)
            {
                Log.Log.err($"{ex}");

                rsp.err((int)em_error.db_error);
            }
        }
    }
}

using Abelkhan;
using System.Collections;

namespace Player
{
    class login_msg_handle
    {
        private readonly login_player_module
login_player_Module;

        public login_msg_handle()
        {
            login_player_Module = new
login_player_module();

            
login_player_Module.on_player_login_no_token +=
Login_player_Module_on_player_login_no_token;
            login_player_Module.on_player_login +=
Login_player_Module_on_player_login;
        }

        private async void
Login_player_Module_on_player_login_no_token(string
account)
        {
```

```
            Log.Log.trace("on_player_login_no_token
begin!");

            var rsp = login_player_Module.rsp as
login_player_player_login_no_token_rsp;

            try
            {
                var tuple = await
Player.client_Mng.token_player_login(account);
                rsp.rsp(tuple.Item1, tuple.Item2);
            }
            catch (System.Exception ex)
            {
                Log.Log.err($"{ex}");
                rsp.err((int)em_error.db_error);
            }
        }

        private async void
Login_player_Module_on_player_login(string openid,
string anonymous_openid)
        {
            Log.Log.trace("on_player_login begin!");

            var rsp = login_player_Module.rsp as
login_player_player_login_rsp;

            try
            {
                var tuple = await
Player.client_Mng.token_player_login(openid,
anonymous_openid);
                rsp.rsp(tuple.Item1, tuple.Item2);
            }
            catch (System.Exception ex)
            {
                Log.Log.err($"{ex}");
                rsp.err((int)em_error.db_error);
            }
        }
```

```
    }
}

using Abelkhan;
using System;
using System.Collections.Generic;
using System.IO;
using System.Security.Cryptography;

namespace Player
{
    class match_msg_handle
    {
        private readonly match_player_module
match_Player_Module;

        public match_msg_handle()
        {
            match_Player_Module = new ();
            match_Player_Module.on_buy_role +=
Match_Player_Module_on_buy_role;
            match_Player_Module.on_buy_equip +=
Match_Player_Module_on_buy_equip;
            match_Player_Module.on_battle_victory +=
Match_Player_Module_on_battle_victory;

match_Player_Module.on_peak_strength_victory +=
Match_Player_Module_on_peak_strength_victory;
        }

        private void
Match_Player_Module_on_buy_equip(long guid, int
equip)
        {
            Log.Log.trace("on_buy_equip begin!");

            try
            {
                var _avatar =
Player.client_Mng.guid_get_client_proxy(guid);
                var _player_info =
```

```
_avatar.get_real_hosting_data<PlayerInfo>();
                if
(_player_info.Data.CheckBuyEquip(equip))
                {
client_mng.PlayerClientCaller.get_client(_avatar.Clie
ntUUID).achievement_complete(_player_info.Data.Info()
.Achiev, _player_info.Data.Info().wAchiev);
                }
            }
            catch (System.Exception ex)
            {
                Log.Log.err($"on_buy_equip error:
{ex}");
            }

            Log.Log.trace("on_buy_equip end!");
        }

        private void
Match_Player_Module_on_buy_role(long guid, Role
roleInfo)
        {
            Log.Log.trace("on_buy_role begin!");

            try
            {
                var _avatar =
Player.client_Mng.guid_get_client_proxy(guid);
                var _player_info =
_avatar.get_real_hosting_data<PlayerInfo>();
                if
(_player_info.Data.CheckBuyRole(roleInfo))
                {
client_mng.PlayerClientCaller.get_client(_avatar.Clie
ntUUID).achievement_complete(_player_info.Data.Info()
.Achiev, _player_info.Data.Info().wAchiev);
                }
            }
            catch (System.Exception ex)
```

```
            {
                Log.Log.err($"on_buy_role error:
{ex}");
            }

            Log.Log.trace("on_buy_role end!");
        }

        private void
Match_Player_Module_on_peak_strength_victory(BattleVi
ctory is_victory, UserBattleData user)
        {
            Log.Log.trace("on_peak_strength_victory
begin!");

            var rsp = match_Player_Module.rsp as
match_player_peak_strength_victory_rsp;

            try
            {
                Log.Log.trace($"is_victory:
{is_victory}, userGuid:{user.User.UserGuid}");

                var _avatar =
Player.client_Mng.guid_get_client_proxy(user.User.Use
rGuid);
                var _player_info =
_avatar.get_real_hosting_data<PlayerInfo>();

                var battleInfo = new BattleInfo
                {
                    mod = BattleMod.PeakStrength,
                    isVictory = is_victory,
                    isStreakVictory = false,
                    RoleList = user.RoleList,
                };
                if
(_player_info.Data.AddCheckAchievement(battleInfo) ||
_player_info.Data.AddCheckWeekAchievement(battleInfo)
)
                {
```

```
client_mng.PlayerClientCaller.get_client(_avatar.Clie
ntUUID).achievement_complete(_player_info.Data.Info()
.Achiev, _player_info.Data.Info().wAchiev);
                }

                var rank_Info = new UserRankInfo
                {
                    nick_name =
_player_info.Data.Info().User.UserName,
                    avatar =
_player_info.Data.Info().User.Avatar,
                    score =
_player_info.Data.Info().score
                };
                rsp.rsp(rank_Info);
            }
            catch (System.Exception ex)
            {
                Log.Log.err($"{ex}");
                rsp.err((int)em_error.db_error);
            }
        }

        private void
Match_Player_Module_on_battle_victory(BattleMod mod,
bool is_victory, UserBattleData user, List<Role>
roles)
        {
            Log.Log.trace("on_battle_victory
begin!");

            try
            {
                Log.Log.trace($"mod:{mod},
is_victory:{is_victory}, userGuid:
{user.User.UserGuid}");
                var _avatar =
Player.client_Mng.guid_get_client_proxy(user.User.Use
rGuid);
                var _player_info =
```

```
_avatar.get_real_hosting_data<PlayerInfo>();
                var battleInfo = new BattleInfo
                {
                    mod = mod,
                    isVictory = is_victory ?
BattleVictory.victory : BattleVictory.faild,
                    isStreakVictory = is_victory &&
user.faild <= 0,
                    RoleList = user.RoleList,
                };
                if
(_player_info.Data.AddCheckAchievement(battleInfo) ||
_player_info.Data.AddCheckWeekAchievement(battleInfo)
)
                {
client_mng.PlayerClientCaller.get_client(_avatar.Clie
ntUUID).achievement_complete(_player_info.Data.Info()
.Achiev, _player_info.Data.Info().wAchiev);
                }
                _player_info.Data.CheckRank(mod,
is_victory);

_player_info.Data.AddRogueRole(roles);

client_mng.PlayerClientCaller.get_client(_avatar.Clie
ntUUID).battle_victory();
            }
            catch (System.Exception ex)
            {
                Log.Log.err($"{ex}");
            }
        }
    }
}

using Abelkhan;
using System;
using System.Collections.Generic;
```

```
using System.Linq;
using System.Threading;
using System.Threading.Tasks;

namespace Player
{
    class match_proxy
    {
        private readonly player_match_caller
player_Match_Caller;
        private readonly Hub.HubProxy _proxy;

        public string name
        {
            get { return _proxy.name; }
        }

        public match_proxy(player_match_caller
caller, Hub.HubProxy proxy)
        {
            player_Match_Caller = caller;
            _proxy = proxy;
        }

        public player_match_start_battle_cb
start_battle(BattleMod mod, string client_uuid,
List<int> role_list, UserInformation user_info)
        {
            return
player_Match_Caller.get_hub(name).start_battle(mod,
client_uuid, role_list, user_info);
        }

        public player_match_start_peak_strength_cb
start_peak_strength(string client_uuid, long guid)
        {
            return
player_Match_Caller.get_hub(name).start_peak_strength
(client_uuid, guid);
        }
```

```
        public player_match_reconnect_cb
reconnect(string old_client_uuid, string
new_client_uuid)
        {
            return
player_Match_Caller.get_hub(name).reconnect(old_clien
t_uuid, new_client_uuid);
        }
    }

    class match_proxy_mng
    {
        private readonly player_match_caller
_player_Match_Caller = new();

        private readonly Dictionary<string,
match_proxy> match_proxys = new();

        public void reg_match_proxy(Hub.HubProxy
_proxy)
        {
            if (_proxy.name != Hub.Hub.name)
            {
                match_proxys[_proxy.name] = new
match_proxy(_player_Match_Caller, _proxy);
            }
        }

        public match_proxy get_match_proxy()
        {
            var proxy = match_proxys.ToList()
[RandomHelper.RandomInt(match_proxys.Count)];
            return proxy.Value;
        }

        public match_proxy get_match_proxy(string
name)
        {
            match_proxys.TryGetValue(name, out var
proxy);
            return proxy;
```

```
        }
    }
}

using Abelkhan;

namespace Player
{
    class player_msg_handle
    {
        private readonly
player_player_offline_msg_module
player_offline_msg_Module;

        public player_msg_handle()
        {
            player_offline_msg_Module = new ();
player_offline_msg_Module.on_player_have_offline_msg
+=
Player_offline_msg_Module_on_player_have_offline_msg;
        }

        private async void
Player_offline_msg_Module_on_player_have_offline_msg(
long guid)
        {
            Log.Log.trace("on_player_have_offline_msg
begin!");

            try
            {
                await
Player.offline_Msg_Mng.process_offline_msg(guid.ToStr
ing());
            }
            catch (System.Exception ex)
            {
                Log.Log.err($"{ex}");
            }
        }
```

```
    }
}

using Abelkhan;
using System;
using System.Collections.Generic;
using System.Threading;
using System.Threading.Tasks;

namespace Player
{
    class player_proxy
    {
        private readonly
player_player_offline_msg_caller
player_Player_Offline_Msg_Caller;
        private readonly Hub.HubProxy _proxy;

        public string name
        {
            get { return _proxy.name; }
        }

        public
player_proxy(player_player_offline_msg_caller
offline_caller, Hub.HubProxy proxy)
        {
            player_Player_Offline_Msg_Caller =
offline_caller;
            _proxy = proxy;
        }

        public void player_have_offline_msg(long
guid)
        {

player_Player_Offline_Msg_Caller.get_hub(name).player
_have_offline_msg(guid);
        }
    }
```

```
    class player_proxy_mng
    {
        private readonly
player_player_offline_msg_caller
_player_offline_msg_caller = new();

        private readonly Dictionary<string,
player_proxy> player_proxys = new();

        public void reg_player_proxy(Hub.HubProxy
_proxy)
        {
            if (_proxy.name != Hub.Hub.name)
            {
                player_proxys[_proxy.name] = new
player_proxy(_player_offline_msg_caller, _proxy);
            }
        }

        public player_proxy get_player_proxy(string
player_hub_name)
        {

player_proxys.TryGetValue(player_hub_name, out
player_proxy proxy);
            return proxy;
        }
    }
}

using Abelkhan;
using Log;
using OfflineMsg;
using Service;
using System;
using System.Threading;

namespace Player
{
    class Player
    {
```

```
        public static RedisHandle _redis_handle;

        public static readonly client_mng client_Mng
= new ();
        public static readonly player_proxy_mng
player_Proxy_Mng = new ();
        public static readonly match_proxy_mng
match_Proxy_Mng = new ();
        public static rank_svr_service_caller
_rank_caller = new rank_svr_service_caller();

        public static OfflineMsgMgr offline_Msg_Mng =
new (Constant.Constant.player_db_name,
Constant.Constant.player_db_offline_msg_collection);

      static void Main(string[] args)
       {
            var _hub = new Hub.Hub(args[0], args[1],
"player", "dynamic");

            if (!
Hub.Hub._root_config.has_key("redis_for_mq_pwd"))
            {
                _redis_handle = new
RedisHandle(Hub.Hub._root_config.get_value_string("re
dis_for_cache"), string.Empty);
            }
            else
            {
                _redis_handle = new
RedisHandle(Hub.Hub._root_config.get_value_string("re
dis_for_cache"),
Hub.Hub._root_config.get_value_string("redis_for_mq_p
wd"));
            }

            
config.Config.Load(Hub.Hub._config.get_value_string("
excel_json_config"));

            _hub.set_support_take_over_svr(true);
```

```
            var _client_msg_handle = new
client_msg_handle();
            var _login_msg_handle = new
login_msg_handle();
            var _player_msg_handle = new
player_msg_handle();
            var _match_msg_handle = new
match_msg_handle();

            _hub.on_hubproxy += on_hubproxy;
            _hub.on_hubproxy_reconn += on_hubproxy;

            _hub.on_client_msg += async (uuid) =>
            {
                try
                {
                    var _client = await
client_Mng.uuid_get_client_proxy(uuid);
                    if (_client != null)
                    {
                        _client.LastActiveTime =
Timerservice.Tick;
                    }
                }
                catch (System.Exception ex)
                {
                    Log.Log.err($"{ex}");
                }
            };

            _hub.onCloseServer += () => {
                _hub.closeSvr();
            };

            
tick_set_player_svr_info(Timerservice.Tick);

            Log.Log.trace("player start ok");

            _hub.run().Wait();
```

```
        }

        private static void
tick_set_player_svr_info(long tick_time)
        {
            Player._redis_handle.SetData(RedisHelp.BuildPlayerSvr
InfoCacheKey(Hub.Hub.name), new svr_info { tick_time
= (int)Hub.Hub.tick, player_num =
client_Mng.Count });

            Hub.Hub._timer.addticktime(300000,
tick_set_player_svr_info);
        }

        private static void on_hubproxy(Hub.HubProxy
_proxy)
        {
            if (_proxy.type == "player")
            {
                player_Proxy_Mng.reg_player_proxy(_proxy);
            }
            else if (_proxy.type == "match")
            {
                match_Proxy_Mng.reg_match_proxy(_proxy);
            }
        }
    }
}

using Abelkhan;
using MsgPack.Serialization;
using OfflineMsg;
using Service;
using System;
using System.Numerics;
using System.Threading.Tasks;
using static OfflineMsg.OfflineMsgMgr;
```

```
namespace Player
{
    public class RankRewardMsg
    {
        public long guid;
        public int rank;
        public int gold;
    }

    public class rank_msg_handle
    {
        private readonly rank_player_module
rank_Player_Module;

        public rank_msg_handle()
        {
            rank_Player_Module = new ();
            rank_Player_Module.on_rank_reward +=
Rank_Player_Module_on_rank_reward;

            
Player.offline_Msg_Mng.register_offline_msg_callback(
"RankRewardMsg", do_rank_reward);
        }

        private void do_rank_reward(offline_msg msg)
        {
            using var _tmp =
MemoryStreamPool.mstMgr.GetStream();
            _tmp.Write(msg.msg, 0, msg.msg.Length);
            _tmp.Position = 0;
            var _serializer =
MessagePackSerializer.Get<RankRewardMsg>();
            var _rank_reward_msg =
_serializer.Unpack(_tmp);

            var _avatar =
Player.client_Mng.guid_get_client_proxy(long.Parse(ms
g.player_guid));
            var _data =
_avatar.get_real_hosting_data<PlayerInfo>();
```

```
            _data.Data.Info().gold +=
_rank_reward_msg.gold;

            client_mng.PlayerClientCaller.get_client(_avatar.Clie
ntUUID).rank_reward(new RankReward()
            {
                rank = _rank_reward_msg.rank,
                gold = _rank_reward_msg.gold,
                timetmp = msg.send_timetmp,
            });
            Player.offline_Msg_Mng.del_offline_msg(msg.msg_guid);
        }

        private void
Rank_Player_Module_on_rank_reward(System.Collections.
Generic.List<RankPlayer> ranks)
        {
            Log.Log.trace("on_rank_reward begin!");

            foreach (var r in ranks)
            {
                if
(config.Config.RankRewardConfigs.TryGetValue(r.rank,
out var reward))
                {
                    var gold = 0;
                    foreach (var (t, v) in
reward.Reward)
                    {
                        if (t == "Gold")
                        {
                            gold = v;
                        }
                    }

                    var _msg = new RankRewardMsg
                    {
                        guid = r.guid,
                        rank = r.rank,
```

```
                            gold = gold,
                        };

                        using var st =
MemoryStreamPool.mstMgr.GetStream();
                        var _serializer =
MessagePackSerializer.Get<RankRewardMsg>();
                        _serializer.Pack(st, _msg);

                        var _offline_msg = new
OfflineMsgMgr.offline_msg()
                        {
                            msg_guid =
Guid.NewGuid().ToString("N"),
                            player_guid =
r.guid.ToString(),
                            send_timetmp =
Timerservice.Tick,
                            msg_type = "RankRewardMsg",
                            msg = st.ToArray(),
                        };

client_mng.forward_offline_msg(_offline_msg);
                    }
                }
            }
        }
    }
```

```
using Abelkhan;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Numerics;
using System.Runtime.InteropServices;

namespace RankSvr
{
    class PlayerProxy
    {
        private Hub.HubProxy proxy;
```

```
        private rank_player_caller caller;

        public PlayerProxy(Hub.HubProxy _proxy,
rank_player_caller _caller)
        {
            proxy = _proxy;
            caller = _caller;
        }

        public void rank_reward(List<RankPlayer>
rewards)
        {

caller.get_hub(proxy.name).rank_reward(rewards);
        }
    }

    class RankSvr
    {
        private static readonly rank_player_caller
caller = new();
        private static Dictionary<string,
PlayerProxy> playerProxys = new();

        static void Main(string[] args)
        {
            var _hub = new Hub.Hub(args[0], args[1],
"rank", "fixed");

            _hub.on_hubproxy += on_hubproxy;
            _hub.on_hubproxy_reconn += on_hubproxy;

            _hub.onCloseServer += () => {
                _hub.closeSvr();
            };

            _hub.onDBProxyInit += async () =>
            {
                await
Rank.RankModule.Init(Constant.Constant.player_db_name
, Constant.Constant.player_db_guid_rank, new
```

```
List<string>()
{ Constant.Constant.player_rank_name_combat_power });
            };

            Log.Log.trace("rank svr start ok");

            _hub.run().Wait();

            Rank.RankModule.save_rank();
        }

        private static void on_hubproxy(Hub.HubProxy
_proxy)
        {
            Log.Log.trace("on_hubproxy type:{0},
name:{1}!", _proxy.type, _proxy.name);

            if (_proxy.type == "player")
            {
                playerProxys[_proxy.name] = new
PlayerProxy(_proxy, caller);
            }
        }
    }
}

/*
 * BattleDis.ts
 * author: Hotaru
 * 2023/10/12
 * 战斗展示类
 */
import {
  _decorator,
  instantiate,
  Node,
  Prefab,
  Label,
  Button,
  UITransform,
  Vec3,
  sp,
  game,
  Game,
  error,
```

```
} from "cc";
import { login } from '../../login/login';
import { Queue } from "./Queue";
import { Battle } from "../AutoChessBattle/battle";
import * as skill from "../AutoChessBattle/skill/skill_base";
import * as BattleEnums from "../AutoChessBattle/BattleEnums";
import { delay, sleep } from "../../other/sleep";
import { RoleDis } from "./RoleDis";
import { BundleManager } from "../../bundle/BundleManager";
import { hub_call_gate_reverse_reg_client_hub_rsp } from
"../../serverSDK/gate";
import { Role as rRole } from "../AutoChessBattle/role";
import { netSingleton } from "../../netDriver/netSingleton";
import * as singleton from "../../netDriver/netSingleton";
import { BattleVictory, Role } from "../../serverSDK/common";
import { Team } from "../AutoChessBattle/team";
import { GameManager } from "../../other/GameManager";
import { SendMessage } from "../../other/MessageEvent";
import { AudioManager } from "../../other/AudioManager";
import * as enums from "../../other/enums";
import * as common from "../../serverSDK/common";
import { GameMode } from "../../other/enums";
const { ccclass, property } = _decorator;

export class BattleDis {
  //摄像机
  private cameraNode: Node;
  private originCameraPos: Vec3;
  //父级对象
  public father: Node;
  //顶级面板
  public panelNode: Node;
  //战斗效果
  private battleEffectImg: Node;
  private launchSkillEffect: Node;
  //敌我开场介绍面板
  private selfBeginEffect: Node;
  private enemyBeginEffect: Node;
  //敌我队列
  public selfQueue: Queue;
  public enemyQueue: Queue;

  public BattleFailed: Node;
  public backMainBtn: Button;
  private isAdRevive: boolean = false;
  private BackMainPage: () => void = null;
  public reviveBtn: Button;

  private btnGMWin: Node;
```

```
  public rewardedVideoAd: WechatMinigame.RewardedVideoAd =
null;

  public puase: boolean = false;

  //所有需要并行执行的事件队列,分己方和敌人方，己方先执行
  private selfParallelList: Promise<void>[] = [];
  private enemyParallelList: Promise<void>[] = [];

  //战斗系统类
  public battleCentre: Battle = null;

  public constructor(battle: Battle) {
    this.battleCentre = battle;

    this.battleCentre.onPlaySound = (sound: string) => {
      AudioManager.Instance.PlaySound(sound);
    };
    this.battleCentre.onPlayerOnShot = (sound: string) => {
      AudioManager.Instance.PlayerOnShot(sound);
    };
    this.battleCentre.onKillRole = (r: Role) => {
      singleton.netSingleton.game.kill_Role_ntf(r);
    };

    this.onEvent();
  }

  public destory() {
    AudioManager.Instance.Stop();

    this.selfQueue.destroy();
    this.enemyQueue.destroy();

    this.panelNode.destroy();
  }

  /*
   * 修改start
   * author: Hotaru
   * 2024/03/07
   * 让加载更平顺
   */
  public async Start(father: Node, _callBack: (e?: () => void)
=> void) {
    try {
      console.log("battledis start");
      this.panelNode = instantiate(await
GameManager.Instance.getPanel("BattlePanel"));
      this.panelNode.setParent(GameManager.Instance.node);
```

```
      this.selfQueue =
this.panelNode.getChildByName("Self_Queue").getComponent(Queue
);
      this.enemyQueue =
this.panelNode.getChildByName("Enemy_Queue").getComponent(Queu
e);

      this.battleEffectImg =
this.panelNode.getChildByName("BattleEffectImg");
      this.battleEffectImg.active = false;

      this.selfBeginEffect =
this.panelNode.getChildByName("SelfBeginEffect");
      this.selfBeginEffect.active = false;

      this.enemyBeginEffect =
this.panelNode.getChildByName("EnemyBeginEffect");
      this.enemyBeginEffect.active = false;

      this.launchSkillEffect =
this.panelNode.getChildByName("LaunchSkillEffect");
      this.launchSkillEffect.active = false;

      this.btnGMWin =
this.panelNode.getChildByPath("BtnGMWin");
      this.BattleFailed =
this.panelNode.getChildByPath("BattleFailed");
      this.BattleFailed.active = false;
      this.backMainBtn =
this.BattleFailed.getChildByName("back").getComponent(Button);
      this.reviveBtn =
this.BattleFailed.getChildByName("revive").getComponent(Button
);
      this.registerEvent();

      let victory = this.panelNode.getChildByName("victory");
      if (victory) {
        victory.active = false;
      }

      console.log("PutRole start");
      await this.PutRole();
      console.log("PutRole end");
      this.father = father;

      AudioManager.Instance.PlaySound("Sound/sound_battle");

      _callBack(() => {
        setTimeout(this.TickBattle.bind(this), 500);
```

```
      });
    } catch (error) {
      console.error("BattleDis 下的 Start 错误 err:", error);
    }
  }

  private registerEvent() {
    this.btnGMWin.on(Button.EventType.CLICK, () => {
      if(enums.GameMode.PVP == this.battleCentre.gamemode)
      {
netSingleton.game.confirm_match_round_victory(BattleVictory.vi
ctory, this.battleCentre.addCoin);
        //
netSingleton.game.confirm_peak_strength_victory(BattleVictory.
victory, this.battleCentre.addCoin);
      }
      else
      {
netSingleton.game.confirm_quest_victory(BattleVictory.victory,
this.battleCentre.addCoin);
      }
      this.puase = true;
    });

    this.backMainBtn.node.on(Button.EventType.CLICK, () => {
      this.BattleFailed.active = false;
      this.BackMainPage();
    });

    this.reviveBtn.node.on(Button.EventType.CLICK, async () =>
{
      this.BattleFailed.active = false;
      if (this.isAdRevive) {
        // 创建激励视频广告实例，提前初始化
        if (!this.rewardedVideoAd) {
          this.rewardedVideoAd = wx.createRewardedVideoAd({
            adUnitId: 'adunit-f680ffa7a8a528b8'
          })
          this.rewardedVideoAd.onError((err) => {
            console.error("激励视频广告错误 err:", err);
            this.rewardedVideoAd = null; // 销毁广告实例，避免重复
错误
          });
          this.rewardedVideoAd.onLoad(() => {
            console.log("激励视频广告加载成功!");
          });
          this.rewardedVideoAd.onClose( async (res) => {
            if (res && res.isEnded || res === undefined) {
```

```
              // 正常播放结束，可以下发奖励
              console.log("激励视频广告正常播放结束，发放奖励!");
              this.Revive();
            } else {
              // 播放中途退出，不下发奖励
              console.log("激励视频广告播放中途退出，不发放奖励");
              this.BattleFailed.setSiblingIndex(999);
              this.BattleFailed.active = true;
            }
          });
          await this.rewardedVideoAd.load();
        }
        await this.rewardedVideoAd.show();
      } else {
        let sharedTitle = [ '一起来冒险吧!' , '我在萌萌自走棋等你
哦!' , '快来和我一起玩萌萌自走棋吧!' , '萌萌自走棋好玩又刺激，快来和我一
起玩吧!' , '萌萌自走棋等你来战!' ];
        wx.shareAppMessage({
          //imageUrl: "shared_bk.jpg",
          title: sharedTitle[Math.floor(Math.random() *
sharedTitle.length)],
        });
        this.Revive();
      }
    });
  }
```

```
  private Revive() {
    try {
      if(enums.GameMode.PVP == this.battleCentre.gamemode) {

singleton.netSingleton.game.match_revive_battle().callBack((da
ta, shop) => {

singleton.netSingleton.game.cb_start_match_battle_ready(data,
shop, null);
        }, (err) => {
          console.error("PVP 复活失败 err:", err);
        });
      } else if (enums.GameMode.PVE ==
this.battleCentre.gamemode) {

singleton.netSingleton.game.quest_revive_battle().callBack((da
ta, shop) => {

singleton.netSingleton.game.cb_start_quest_battle_ready(data,
shop, null);
        }, (err) => {
          console.error("PVE 复活失败 err:", err);
        });
```

```
      }
    }
    catch (err) {
      console.error("Revive 复活失败 err:", err);
    }
  }

  async TickBattle() {
    try {
      while (!this.battleCentre.CheckEndBattle()) {
        await this.battleCentre.TickBattle();

        while (this.puase) {
          await sleep(200);
        }
      }

      let is_victory = BattleVictory.victory;
      if (this.battleCentre.GetWinCamp() ==
BattleEnums.Camp.Self) {
        is_victory = BattleVictory.victory;
      } else if (this.battleCentre.GetWinCamp() ==
BattleEnums.Camp.Enemy) {
        is_victory = BattleVictory.faild;
      }

      let addTime = false;
      let settlement = false;
      console.log("准备战斗结算");
      if (this.battleCentre.round <= 15 && 10 ==
this.battleCentre.victory) {
        addTime = true;
      }
      if (is_victory == BattleVictory.victory || (is_victory
== BattleVictory.faild && this.battleCentre.faild - 1 > 0)) {
        let heath = is_victory == BattleVictory.victory ?
this.battleCentre.faild : this.battleCentre.faild - 1;
        settlement = true;
        this.panelNode.dispatchEvent(
          new SendMessage("OpenSettlement", true, {
            outcome: is_victory,
            GameMode: this.battleCentre.gamemode,
            addCoin: this.battleCentre.addCoin,
            hpNum: heath,
            isAddTime: addTime,
          }),
        );
      }

      if (!settlement) {
```

```
        console.log("TickBattle settlement false");

        await sleep(500);
        if(enums.GameMode.PVP == this.battleCentre.gamemode)
        {

netSingleton.game.confirm_match_round_victory(is_victory,
this.battleCentre.addCoin);
        }
        else
        {
          netSingleton.game.confirm_quest_victory(is_victory,
this.battleCentre.addCoin);
        }
        this.battleCentre.addCoin = 0;
      }

    } catch (error) {
      console.error("BattleDis 下的 TickBattle 错误 err:",
error);
    }
  }

  async SetGameVictory(is_victory: boolean, backMainPage: ()
=> void) {
    if (is_victory) {
      let msg = "游戏胜利!";
      let text = "<outline color=black width=4>" + msg + "</
outline>";
      this.panelNode.dispatchEvent(new SendMessage("ShowTip",
true, text));

      await sleep(333);
      backMainPage();
    }
    else {
      this.BackMainPage = backMainPage;
      this.BattleFailed.setSiblingIndex(999);
      this.BattleFailed.active = true;
      if (Math.random() < 0.5) {
        this.isAdRevive = true;

this.reviveBtn.node.getChildByName("Label").getComponent(Label
).string = "看广告复活一次!";
      } else {
        this.isAdRevive = false;

this.reviveBtn.node.getChildByName("Label").getComponent(Label
).string = "邀请好友复活一次!";
      }
```

```
    }
  }

  async PutRole() {
    try {
      let waits = [];

      let selfRoles =
this.battleCentre.GetSelfTeam().GetRoles();
      waits.push(this.selfQueue.InitRole(selfRoles));

      let enemyRoles =
this.battleCentre.GetEnemyTeam().GetRoles();
      waits.push(this.enemyQueue.InitRole(enemyRoles));

      await Promise.all(waits);
    } catch (error) {
      console.error("BattleDis 下的 PutRole 错误 err:", error);
    }
  }

  ShowBattleEffect() {
    this.battleEffectImg.active = true;
    this.battleEffectImg.setSiblingIndex(99);
    this.battleEffectImg
      .getComponent(sp.Skeleton)
      .setAnimation(0, "animation", false);
    delay(500, () => {
      this.battleEffectImg.active = false;
    });
  }

  private showLaunchSkillEffect(isShow: boolean = false) {
    if (!isShow) {
      return delay(0, () => {});
    }
    this.launchSkillEffect.active = true;

    this.launchSkillEffect
      .getChildByPath("BottomImg")
      .getComponent(sp.Skeleton).animation = "a2";
    this.launchSkillEffect
      .getChildByPath("RoleImg")
      .getComponent(sp.Skeleton).animation = "a";
    this.launchSkillEffect
      .getChildByPath("TopImg")
      .getComponent(sp.Skeleton).animation = "a";

    return delay(2000, () => {
      this.launchSkillEffect.active = false;
```

```
    });
  }

  private showLaunchFettersEffect() {
    return delay(2000, () => {
      this.launchSkillEffect.active = false;
    });
  }

  private shakeScreen(duration: number, magnitude: number) {
    // 获取摄像机组件
    // 保存原始位置，以便震动后可以恢复

    // 震动效果
    if (null == this.cameraNode) return;
    let elapsed = 0;
    const shake = () => {
      if (elapsed < duration) {
        // 随机确定摄像机震动的新位置
        const randomX = Math.random() * magnitude * 2 -
magnitude;
        const randomY = Math.random() * magnitude * 2 -
magnitude;
        this.cameraNode.setPosition(
          this.originCameraPos.x + randomX,
          this.originCameraPos.y + randomY,
        );

        // 更新已经过去的时间
        elapsed += 0.05;
        // 请求下一帧继续执行震动
        requestAnimationFrame(shake);
      } else {
        // 震动结束，恢复摄像机的原始位置
        this.cameraNode.setPosition(this.originCameraPos);
      }
    };

    // 开始震动
    shake();
  }
  /*
   *
   * 以下为事件响应函数
   *
   */
  //战斗开始阶段展示双方名牌
  private async CheckBeginBattle(evs: skill.Event[]) {
    try {
      for (let ev of evs) {
```

```
        //console.log("checkAttackEvent ev:", ev)
        if (BattleEnums.EventType.BattleBegin != ev.type) {
          continue;
        }

        this.selfBeginEffect.active = true;
        this.enemyBeginEffect.active = true;

        this.selfBeginEffect.setSiblingIndex(100);
        this.enemyBeginEffect.setSiblingIndex(100);

        this.selfBeginEffect
          .getChildByPath("BottomImg")
          .getComponent(sp.Skeleton).animation = "a2";
        this.selfBeginEffect
          .getChildByPath("UserName")
          .getComponent(Label).string = "己方队伍";
        this.selfBeginEffect
          .getChildByPath("TopImg")
          .getComponent(sp.Skeleton).animation = "a";

        this.enemyBeginEffect
          .getChildByPath("BottomImg")
          .getComponent(sp.Skeleton).animation = "a2";
        this.enemyBeginEffect
          .getChildByPath("UserName")
          .getComponent(Label).string = "敌方队伍";
        this.enemyBeginEffect
          .getChildByPath("TopImg")
          .getComponent(sp.Skeleton).animation = "a";

        return delay(1000, () => {
          this.selfBeginEffect.active = false;
          this.enemyBeginEffect.active = false;
        });
      }
    } catch (error) {
      console.error("BattleDis 下的 CheckBeginBattle 错误
err:", error);
    }
  }

  //普通攻击
  private async CheckAttackEvent(evs: skill.Event[]) {
    try {
      let evs_floating: skill.Event[] = [];
      let selfAttack = false;
      let enemyAttack = false;
      let selfRoleNodeRoleDis: RoleDis = null;
      let enemyRoleNodeRoleDis: RoleDis = null;
```

```
      for (let ev of evs) {
        if (
          BattleEnums.EventType.AttackInjured == ev.type ||
          BattleEnums.EventType.TransferInjured
        ) {
          evs_floating.push(ev);
        }
        if (BattleEnums.EventType.AttackInjured == ev.type) {
          if (BattleEnums.Camp.Self == ev.spellcaster.camp) {
            if (!selfAttack) {
              for (let r of ev.recipient) {
                if
(this.battleCentre.GetEnemyTeam().GetRole(r.index).CheckDead()
)
                  {}
              }
              let roleNode =
this.selfQueue.roleNodes[ev.spellcaster.index];
              if (roleNode) {
                selfRoleNodeRoleDis =
roleNode.getComponent(RoleDis);
                if (selfRoleNodeRoleDis) {
                  selfAttack = true;
                }
              }
            }
          } else if (BattleEnums.Camp.Enemy ==
ev.spellcaster.camp) {
            if (!enemyAttack) {
              let roleNode =
this.enemyQueue.roleNodes[ev.spellcaster.index];
              if (roleNode) {
                enemyRoleNodeRoleDis =
roleNode.getComponent(RoleDis);
                if (enemyRoleNodeRoleDis) {
                  enemyAttack = true;
                }
              }
            }
          }
        }
      }

      if (selfAttack && enemyAttack) {
        console.log("CheckAttackEvent begin!");
        console.log(
          "CheckAttackEvent selfRoleNodeRoleDis:",
          selfRoleNodeRoleDis,
        );
        console.log(
```

```
          "CheckAttackEvent enemyRoleNodeRoleDis:",
          enemyRoleNodeRoleDis,
        );

        let allAwait: Promise<void>[] = [];

        allAwait.push(
          selfRoleNodeRoleDis.Attack(
            this.selfQueue.readyLocation.worldPosition,
            this.selfQueue.battleLocation.worldPosition,
            BattleEnums.Camp.Self,
          ),
        );
        allAwait.push(
          enemyRoleNodeRoleDis.Attack(
            this.enemyQueue.readyLocation.worldPosition,
            this.enemyQueue.battleLocation.worldPosition,
            BattleEnums.Camp.Enemy,
          ),
        );

        await Promise.all(allAwait);
        allAwait = [];

        //震动部分
        if (null == this.cameraNode) {
          this.cameraNode =
this.panelNode.parent.getChildByName("Camera");
          if (!this.cameraNode) {
            console.error("Camera node not found,shake screen
failed");
          } else {
            this.originCameraPos =
this.cameraNode.getPosition();
          }
        }
        this.shakeScreen(0.5, 10);
        await this.OnBehurted(evs_floating);

        console.log("CheckAttackEvent end!");
      }
    } catch (error) {
      console.error("BattleDis 下的 CheckAttackEvent 错误
err:", error);
    }
  }

  //受击
  private async OnBehurted(evs: skill.Event[]) {
    try {
```

```
      let allAwait = [];
      let r: Node = null;
      for (let ev of evs) {
        if (
          BattleEnums.EventType.AttackInjured == ev.type ||
          BattleEnums.EventType.TransferInjured == ev.type
        ) {
          if (ev.is_trigger_floating) {
            continue;
          }
          ev.is_trigger_floating = true;

          if (BattleEnums.Camp.Self == ev.spellcaster.camp) {
            r =
this.enemyQueue.roleNodes[ev.recipient[0].index];
            if (r) {

allAwait.push(r.getComponent(RoleDis).BeHurted(ev));

allAwait.push(r.getComponent(RoleDis).ChangeAtt());
            }
          }
          if (BattleEnums.Camp.Enemy == ev.spellcaster.camp) {
            r =
this.selfQueue.roleNodes[ev.recipient[0].index];
            if (r) {

allAwait.push(r.getComponent(RoleDis).BeHurted(ev));

allAwait.push(r.getComponent(RoleDis).ChangeAtt());
            }
          }
        }
      }
      console.log("OnBehurted allAwait:", allAwait);
      await Promise.all(allAwait);
    } catch (error) {
      console.error("BattleDis 下的 OnBehurted 错误 err:",
error);
    }
  }

  //远程攻击技能
  private async CheckRemoteInjured(evs: skill.Event[]) {
    try {
      let allAwait = [];
      for (let ev of evs) {
        if (BattleEnums.EventType.RemoteInjured != ev.type) {
          continue;
        }
```

```
        let spList = BattleEnums.Camp.Self ==
ev.spellcaster.camp ?
          this.selfQueue : this.enemyQueue;
        if (!spList.roleNodes || spList.roleNodes.length <=
ev.spellcaster.index) {
          continue;
        }
        let self = spList.roleNodes[ev.spellcaster.index];

        if (ev.isParallel) {
          if (BattleEnums.Camp.Self == ev.spellcaster.camp) {

this.selfParallelList.push(self.getComponent(RoleDis).UseSkill
(ev));
          } else {
            this.enemyParallelList.push(
              self.getComponent(RoleDis).UseSkill(ev),
            );
          }
        } else {

allAwait.push(self.getComponent(RoleDis).UseSkill(ev));
        }
      }
      await Promise.all(allAwait);
    } catch (error) {
      console.error("BattleDis 下的 CheckRemoteInjured 错误
err:", error);
    }
  }

  //召唤技能
  private async CheckSummonEvent(evs: skill.Event[]) {
    try {
      let allAwait = [];
      for (let ev of evs) {
        if (BattleEnums.EventType.Summon == ev.type) {
          let spList =
            BattleEnums.Camp.Self == ev.spellcaster.camp
              ? this.selfQueue
              : this.enemyQueue;
          let self = spList.roleNodes[ev.spellcaster.index];
          if (ev.isParallel) {
            if (BattleEnums.Camp.Self == ev.spellcaster.camp)
{
              this.selfParallelList.push(
                self.getComponent(RoleDis).UseSkill(ev),
              );
            } else {
```

```
              this.enemyParallelList.push(
                self.getComponent(RoleDis).UseSkill(ev),
              );
            }
          } else {
allAwait.push(self.getComponent(RoleDis).UseSkill(ev));
          }
        }
      }
      await Promise.all(allAwait);
    } catch (error) {
      console.error("BattleDis 下的 CheckSummon 错误 err:",
error);
    }
  }

  /**
   * 检测队伍增益技能
   * @param evs 事件流
   */
  private async CheckAttGainEvent(evs: skill.Event[]) {
    try {
      let allAwait = [];
      for (let ev of evs) {
        if (
          BattleEnums.EventType.IntensifierProperties ==
ev.type ||
          BattleEnums.EventType.IntensifierExp == ev.type
        ) {
          if (ev.recipient.length <= 0) continue;
          console.log("检测到加属性事件");
          let spList =
            BattleEnums.Camp.Self == ev.spellcaster.camp
              ? this.selfQueue
              : this.enemyQueue;
          let self = spList.roleNodes[ev.spellcaster.index];

          if (ev.isParallel) {
            if (BattleEnums.Camp.Self == ev.spellcaster.camp)
{
              this.selfParallelList.push(
                self.getComponent(RoleDis).UseSkill(ev),
              );
            } else {
              this.enemyParallelList.push(
                self.getComponent(RoleDis).UseSkill(ev),
              );
            }
          } else {
```

```
allAwait.push(self.getComponent(RoleDis).UseSkill(ev));
          }
        }
      }
      await Promise.all(allAwait);
    } catch (error) {
      console.error("BattleDis 下的 CheckAttGainEvent 错误
err:", error);
    }
  }

  //加buff或者护盾
  private async CheckAddBuff(evs: skill.Event[]) {
    try {
      let allAwait = [];
      for (let ev of evs) {
        if (
          BattleEnums.EventType.GiveShields == ev.type ||
          BattleEnums.EventType.AddBuff == ev.type
        ) {
          let spList =
            BattleEnums.Camp.Self == ev.spellcaster.camp
              ? this.selfQueue
              : this.enemyQueue;
          let self = spList.roleNodes[ev.spellcaster.index];

allAwait.push(self.getComponent(RoleDis).UseSkill(ev));
        }
      }
      await Promise.all(allAwait);
    } catch (error) {
      console.error("BattleDis 下的 CheckAddBuff 错误 err:",
error);
    }
  }

  //换位
  private async CheckTransPosition(evs: skill.Event[]) {
    try {
      let allAwait = [];
      for (let ev of evs) {
        if (BattleEnums.EventType.ChangeLocation == ev.type) {
          let spList =
            BattleEnums.Camp.Self == ev.spellcaster.camp
              ? this.selfQueue
              : this.enemyQueue;
          let self = spList.roleNodes[ev.spellcaster.index];
```

```
allAwait.push(self.getComponent(RoleDis).UseSkill(ev));
        }
      }
      await Promise.all(allAwait);
    } catch (error) {
      console.error("BattleDis 下的 CheckTransPosition 错误
err:", error);
    }
  }

  //属性交换
  private async CheckSwapProperties(evs: skill.Event[]) {
    try {
      let allAwait = [];
      for (let ev of evs) {
        if (BattleEnums.EventType.SwapProperties == ev.type) {
          let spList =
            BattleEnums.Camp.Self == ev.spellcaster.camp
              ? this.selfQueue
              : this.enemyQueue;
          let self = spList.roleNodes[ev.spellcaster.index];

allAwait.push(self.getComponent(RoleDis).UseSkill(ev));
        }
      }
      await Promise.all(allAwait);
    } catch (error) {
      console.error("BattleDis 下的 CheckSwapProperties 错误
err:", error);
    }
  }

  //抗伤
  private async CheckSubstitute(evs: skill.Event[]) {
    try {
      let allAwait = [];
      let evs_floating: skill.Event[] = [];
      for (let ev of evs) {
        if (BattleEnums.EventType.SubstituteDamage == ev.type)
{
          let spList =
            BattleEnums.Camp.Self == ev.spellcaster.camp
              ? this.selfQueue
              : this.enemyQueue;
          let self = spList.roleNodes[ev.spellcaster.index];

allAwait.push(self.getComponent(RoleDis).UseSkill(ev));
```

```
        }
      }
      await Promise.all(allAwait);
      //await this.OnBehurted(evs_floating);
    } catch (error) {
      console.error("BattleDis 下的 CheckSubstitute 错误 err:",
error);
    }
  }

  //离场
  private async CheckExitEvent(evs: skill.Event[]) {
    try {
      let allAwait = [];
      for (let ev of evs) {
        if (BattleEnums.EventType.Exit != ev.type) {
          continue;
        }

        if (BattleEnums.Camp.Self == ev.spellcaster.camp) {

allAwait.push(this.selfQueue.RemoveRole(ev.spellcaster.index))
;
        } else if (BattleEnums.Camp.Enemy ==
ev.spellcaster.camp) {

allAwait.push(this.enemyQueue.RemoveRole(ev.spellcaster.index)
);
        }
      }
      await Promise.all(allAwait);
    } catch (error) {
      console.error("BattleDis 下的 CheckExitEvent 错误 err:",
error);
    }
  }

  onEvent() {
    this.battleCentre.on_event = async (evs) => {
      try {
        console.log("onEvent evs:", evs);

        await this.CheckBeginBattle(evs);
        await this.CheckSwapProperties(evs);
        await this.CheckTransPosition(evs);
        await this.CheckRemoteInjured(evs);
        await this.CheckAddBuff(evs);
        await this.CheckSummonEvent(evs);
        await this.CheckAttGainEvent(evs);
        if (
```

```
          this.selfParallelList.length > 0 ||
          this.enemyParallelList.length > 0
        ) {
          await Promise.all(this.selfParallelList);
          await Promise.all(this.enemyParallelList);
        }
        this.selfParallelList = [];
        this.enemyParallelList = [];
        await this.CheckAttackEvent(evs);
        await this.CheckSubstitute(evs);
        await this.CheckExitEvent(evs);
      } catch (error) {
        console.error("BattleDis 下的 on_event 错误 err:",
error);
      }
    };
  }
}

/*
 * Bullet.ts
 * author: Hotaru
 * 2023/10/11
 * 子弹展示类
 */
/*修改
 * Bullet.ts
 * author: Guanliu
 * 2023/10/18
 * 子弹轨迹改为贝塞尔曲线、计算到目标点位的距离
 */
import { _decorator, BoxCollider, Component, ITriggerEvent,
math, Node, sp, Tween, tween, Vec2, Vec3 } from 'cc';
import { loadAssets } from '../../bundle/LoadAsset';
const { ccclass, property } = _decorator;

export class BulletInfo
{
    public selfpos: Vec3 = null;
    public targetPos: Vec3 = null;
    public isGain: boolean = false;
    public style: number = 0;

    constructor(selfpos: Vec3,targetPos: Vec3,isGain:
boolean,style: number=0)
    {
        this.selfpos=selfpos;
        this.targetPos=targetPos;
        this.isGain=isGain;
        this.style=style;
```

```
    }
}

@ccclass('Bullet')
export class Bullet extends Component {

    //private target:Node;
    private targetPos:Vec3;
    private controlPoint:Vec3;
    private isInit:boolean;
    //子弹距离目标点小于这个距离就销毁
    public destoryLen:number;
    //贝塞尔曲线的参数（0~1）
    public parameter:number;

    tAttack: Tween<Node>;

    private skell:sp.Skeleton;

    start()
    {
        this.isInit=false;
    }

    protected onDestroy(): void
    {
        this.destroy();
    }

    public Init(_targetPos:Vec3 , _str:string ,
_isGain:boolean , _style:number ,
_isCopy?:boolean):Promise<void>
    {
        return new Promise<void>(async (resolve, reject) =>
        {
            this.skell=this.node.addComponent(sp.Skeleton);
            let effectStr=_str;
            if(_isCopy)
            {
                effectStr="skill_0011/Luminous sphere";
            }
            this.LoadOnConfig(effectStr).then(()=>
            {
                console.log("初始化子弹");
                this.targetPos = _targetPos;
                this.isInit = true;

                //设置旋转角度
                let dir = new Vec2(_targetPos.x -
this.node.position.x, _targetPos.y - this.node.position.y);
```

```
                let angle = dir.signAngle(new Vec2(1, 0)) * 
180 / Math.PI;
                this.node.setRotationFromEuler(new Vec3(0, 0, 
-angle));

                let 
anims=this.skell.skeletonData.getAnimsEnum();
                if(_style!=0 && _isGain)
                {
                    this.skell.setAnimation(0, 
String(anims[_style]), true);
                }

                this.tAttack = tween(this.node)
                    .to(0.5, { position: _targetPos }).call(() 
=> 
                    {
                        console.log("销毁子弹");
                        if (this.skell.skeletonData.name === 
"Luminous sphere")
                        {
                            this.skell.setAnimation(0, 
String(anims[2]), true);

this.skell.setCompleteListener((trackEntry) =>
                            {
                                this.node.destroy();
                            });
                        }
                        else
                        {
                            this.node.destroy();
                        }
                    }).call(() =>
                    {
                        resolve();
                    }).start();
            })


            // this.skell = 
this.node.getComponent(sp.Skeleton);
            // if (isGain)
            // {
            //     this.skell.enabled = false;
            //     await this.LoadOnConfig();
            // }
            // else
            // {
            //     let anims =
```

```
this.skell.skeletonData.getAnimsEnum();
            //     this.skell.setAnimation(0,
String(anims[1]), true);
            // }
            console.log("初始化子弹完成");
        });
    }

    private LoadOnConfig(_str:string)
    {
        return new Promise<void>((resolve, reject) =>
        {
            try
            {
                let path="EffectSpine/Projectiles/"+_str;
                loadAssets.LoadSkeletonData(path, (data) =>
                {
                    if (data)
                    {
                        try
                        {
                            this.skell.skeletonData = data;
                            this.skell.enabled = true;
                            let anims = data.getAnimsEnum();
                            this.skell.setAnimation(0,
String(anims[1]), true);
                            resolve();
                        }
                        catch (error)
                        {
                            console.warn(`子弹光球效果获取失败: `,
error);
                            reject();
                        }
                    }
                });
            }
            catch (error)
            {
                console.error(`Bullet 下的 LoadOnConfig 错误
err:${error}`);
                reject();
            }
        });
    }

    bezierCurve(p0: Vec3, p1: Vec3, p2: Vec3, t: number): Vec3
{
        let x = Math.pow(1 - t, 2) * p0.x + 2 * (1 - t) * t *
p1.x + Math.pow(t, 2) * p2.x;
```

```
        let y = Math.pow(1 - t, 2) * p0.y + 2 * (1 - t) * t *
p1.y + Math.pow(t, 2) * p2.y;
        return new Vec3(x, y, 0);
    }
}
```

```
import { _decorator, Component, Node, sp } from 'cc';
import * as enums from '../../other/enums';
const { ccclass, property } = _decorator;
import * as common from '../../serverSDK/common';
import { loadAssets } from '../../bundle/LoadAsset';
import { config } from '../AutoChessBattle/config/config';

@ccclass('EffectSpine')
export class EffectSpine extends Component
{
    private effectSkele:sp.Skeleton;
    private shieldSkele:sp.Skeleton;

    protected onLoad(): void
    {

this.effectSkele=this.node.getChildByPath("Effect").getCompone
nt(sp.Skeleton);

this.shieldSkele=this.node.getChildByPath("Shield").getCompone
nt(sp.Skeleton);

        this.effectSkele.node.active=false;
        this.shieldSkele.node.active=false;
    }

    private async LoadEffectData(_str:string , _style:number)
    {
        try
        {
            await loadAssets.LoadSkeletonData(_str, (data) =>
            {
                if (data)
                {
                    try
                    {
                        this.effectSkele.skeletonData = data;
                        let anims = data.getAnimsEnum();
                        this.effectSkele.setAnimation(0,
String(anims[_style]), true);
                    }
                    catch (error)
```

```
                    {
                        console.warn(`效果获取失败: ${error}`);
                    }
                }
            });
        }
        catch (error)
        {
            console.error(`EffectSpine 下的 LoadEffectData错误
err:${error}`);
        }
    }

    private async LoadBuffSpine(_buffID:number)
    {
        try
        {
            let jconfig=config.BufferConfig.get(_buffID);
            await this.LoadEffectData(jconfig.Skel , 1);
        }
        catch(error)
        {
            console.error("EffectSpine 下的 LoadBuffSpine 错
误: ",error);
        }
    }

    /**
     * 显示特效
     * @param _effect 效果类型
     * @param _isParallel 是否是并发效果
     * @param _style 样式
     * @param _buffID buff
     *
     */
    public
ShowEffect(_effect:enums.SpecialEffect ,_isParallel:boolean,
_style:number = 1 , _buffID?:number):Promise<number>
    {
        return new Promise(async (resolve , reject)=>
        {
            try
            {
                let show=false;
                switch (_effect)
                {
                    case enums.SpecialEffect.Shields:
                        {
                            this.shieldSkele.node.active=true;
                            let anims =
```

```
this.shieldSkele.skeletonData.getAnimsEnum();
                                this.shieldSkele.setAnimation(0,
String(anims[2]), true);

this.shieldSkele.setCompleteListener((trackEntry)=>
                                {
                                    if(trackEntry.animation.name
=== String(anims[2]))
                                    {

this.shieldSkele.setAnimation(0, String(anims[1]), true);
                                    }
                                });
                            }
                            break;
                        case enums.SpecialEffect.AddProperty:
                            {
                                if(!_isParallel)
                                {
                                    await
this.LoadEffectData("EffectSpine/zqsx/level up" ,_style);
                                }
                                else
                                {
                                    await
this.LoadEffectData("EffectSpine/zqsx/attribute",_style);
                                }
                                show=true;
                            }
                            break;
                        case enums.SpecialEffect.AddBuff:
                            {
                                await this.LoadBuffSpine(_buffID);
                                show=true;
                            }
                            break;
                        case enums.SpecialEffect.Summon:
                            {
                                await
this.LoadEffectData("EffectSpine/magic/magic",_style);
                                show=true
                            }
                            break;
                        case enums.SpecialEffect.Heath:
                            {
                                await
this.LoadEffectData("EffectSpine/hs/hushi",_style);
                                show=true;
                            }
                            break;
```

```
                    case enums.SpecialEffect.SwapProperties:
                        {
                            await
this.LoadEffectData("EffectSpine/jhsx/change",_style);
                            this.effectSkele.timeScale=2;
                            show=true;
                        }
                        break;
                    case enums.SpecialEffect.SubstituteDamage:
                        {
                            await
this.LoadEffectData("EffectSpine/jingji/Thorns",_style);
                            show=true;
                        }
                        break;
                }
                let ms=0;
                let interval=setInterval(()=>
                {
                    ms++;
                },1);
                if(show)
                {
                    this.effectSkele.node.active = true;
                    let anims =
this.effectSkele.skeletonData.getAnimsEnum();

this.effectSkele.setCompleteListener((trackEntry) =>
                    {
                        if (trackEntry.animation.name ===
String(anims[1]))
                        {
                            this.effectSkele.timeScale=1;
                            this.effectSkele.enabled = false;
                            clearInterval(interval);
                            resolve(ms);
                        }
                    });
                }
            }
            catch(error)
            {
                console.error("EffectSpine 下的 ShowEffect 错
误: ",error);
                reject(0);
            }
        });
    }

    public
```

```
RemoveEffect(_effect:enums.SpecialEffect):Promise<void>
    {
        return new Promise((resolve , reject)=>
            {
                try
                {
                    if (enums.SpecialEffect.Shields ==
_effect)
                    {
                        if(null == this.shieldSkele.node)
                        {

this.shieldSkele=this.node.getChildByPath("Shield").getCompone
nt(sp.Skeleton);
                        }
                        if(this.shieldSkele.node.active)
                        {
                            let anims =
this.shieldSkele.skeletonData.getAnimsEnum();
                            this.shieldSkele.setAnimation(0,
String(anims[2]), true);
                            this.shieldSkele.timeScale=-1;

this.shieldSkele.setCompleteListener((trackEntry) =>
                            {

this.shieldSkele.node.active=false;
                            });
                        }
                    }
                    resolve();
                }
                catch(error)
                {
                    console.error("EffectSpine 下的
RemoveEffect 错误: ",error);
                    reject();
                }
            });
    }
}


/*
 * Queue.ts
 * author: Hotaru
 * 2023/10/04
 * 队列展示类
 */
```

```
import { _decorator, Component, instantiate, Node, Prefab, resources, Skeleton, sp, tween, Vec2, Vec3 } from 'cc';
const { ccclass, property } = _decorator;

import * as RoleDis from '../display/RoleDis'
import { BundleManager } from '../../bundle/BundleManager';
import * as role from '../AutoChessBattle/role'
import { Battle } from '../AutoChessBattle/battle';
import { Role } from '../../serverSDK/common';
import { RoleInfo } from '../AutoChessBattle/skill/skill_base';
import { delay, sleep } from '../../other/sleep';
import * as enums from '../../other/enums';

@ccclass('Queue')
export class Queue extends Component 
{
    //站位列表
    @property([Node])
    public locationTemp:Node[]=[];

    @property(Node)
    public readyLocation:Node;
    @property(Node)
    public battleLocation:Node;
    //角色列表（站位和下标对应）
    public roleNodes:Node[] = [null, null, null, null, null, null];

    protected onDestroy(): void
    {
        this.DestroyRole();
    }

    private DestroyRole()
    {
        try
        {
            for (let i = 0; i < this.roleNodes.length; i++)
            {
                if (this.roleNodes[i])
                {
                    this.roleNodes[i].destroy();
                }
            }
        }
        catch (error)
        {
            console.log("Queue 下的 destroyRole 错误:",error);
        }
```

```
    }

    public GetRole(_index)
    {
        if(_index>0 && _index<7)
        {
            return this.roleNodes[_index];
        }
        return null;
    }
/*
 * 修改
 * author: Hotaru
 * 2023/10/31
 * 生成角色
 */
    public SpawnRole(r:role.Role):Promise<Node>
    {
        return new Promise (async (resolve)=>
        {
            try
            {
                let newNode = await
BundleManager.Instance.loadAssetsFromBundle("PartPrefabs",
"RolePrefab") as Prefab;
                let role = instantiate(newNode);
                //this.node.addChild(role);
                role.setParent(this.node.parent);

                this.roleNodes[r.index] = role;

                let roleDis =
role.getComponent(RoleDis.RoleDis);
                await roleDis.Refresh(r,true);
                resolve(role);
            }
            catch (error)
            {
                console.error("Queue 下的 SpawnRole 错误",
error);
                resolve(null);
            }
        })
    }

/*
 * 添加
 * author: Hotaru
 * 2023/10/31
 * 召唤角色
```

```
 */
/*
 * 修改
 * author: Guanliu
 * 2024/1/30
 * 召唤角色
 */
    public async
SummonRole(r:role.Role[],spellcaster:RoleInfo)
    {
        return new Promise<void>(async (resolve, reject) =>
        {
            try
            {
                for (let i = 0; i < r.length; i++)
                {
                    let role = await this.SpawnRole(r[i]) as
Node;
                    if (null == role)
                    {
                        console.warn("Queue 下的 SummonRole 读
取的 role 为空");
                        break;
                    }
                    if (-1 != r[i].index)
                    {
                        //console.log("Summon a character with
ID "+r[i].id+" at position "+r[i].index);
                        //let roleNode =
this.roleNodes[r[i].index];
                        //role.position = roleNode.position;

this.locationTemp[r[i].index].children[0].active = true;

this.locationTemp[r[i].index].children[0].getComponent(sp.Skel
eton).setCompleteListener(() =>
                        {

this.locationTemp[r[i].index].children[0].active = false;
                        })

this.locationTemp[r[i].index].children[0].getComponent(sp.Skel
eton).setAnimation(0, "animate", false);

role.setWorldPosition(this.locationTemp[r[i].index].worldPosit
ion);

role.getComponent(RoleDis.RoleDis).AttackInit();
                        //await
role.getComponent(RoleDis.RoleDis).ShiftPos(this.locationTemp[
```

```
r[i].index].worldPosition,true);
                        await
role.getComponent(RoleDis.RoleDis).OnSummon();
                    }
                    else if (-1 == r[i].index)
                    {
                        await
role.getComponent(RoleDis.RoleDis).Exit();
                    }
                }
                resolve();
                //return delay(1000,()=>{});
            }
            catch (error)
            {
                console.error("Queue 下的 SummonRole 错误",
error);
                reject();
            }
        });
    }
/*
 * 添加
 * author: Hotaru
 * 2023/10/31
 * 战斗开始初始化角色
 */
    public async InitRole(r:role.Role[])
    {
        try
        {
            let waits = [];
            for(let i = 0; i<r.length; i++)
            {
                waits.push(this.SpawnRole(r[i]));
            }

            let result = await Promise.all(waits);
            for(let i = 0; i<r.length; i++) {
                let role= result[i];
                if(null==role)
                {
                    console.warn("Queue 下的 InitRole 读取的
role 为空");
                    break;
                }
                role.worldPosition=new
Vec3(this.locationTemp[r[i].index].worldPosition);
            }
```

```
        }
        catch(error)
        {
            console.warn("Queue 下的 InitRole 错误", error);
        }
    }

    public async Shiftdis(r:role.Role[])
    {
        try
        {
            for(let i=0;i<this.roleNodes.length;i++)
            {
                //this.roleList[i].position =
this.locationTemp[i].position;
                await
this.roleNodes[i].getComponent(RoleDis.RoleDis).ShiftPos(this.
locationTemp[i].position);

this.roleNodes[i].getComponent(RoleDis.RoleDis).AttackInit();
            }
        }
       catch(err)
        {
            console.warn("Queue 下的 Shiftdis 错误:", err);
        }
    }

    public async RemoveRole(index:number)
    {
        try
        {
            console.log("RemoveRole role:", role);
            if (this.roleNodes.length <= index) {
                return;
            }

            let node = this.roleNodes[index];
            this.roleNodes[index]=null;
            if (node) {
                let c = node.getComponent(RoleDis.RoleDis);
                if (c) {
                    await c.Exit();
                }
            }
        }
        catch (err)
        {
            console.warn("Queue 下的 RemoveRole 错误:", err);
        }
```

```
    }

    /**
     * 交换角色位置
     * @param _recipient 需要交换的角色
     * @param _indexValue 目标位置
     * @returns
     */
    public async SwitchRolePos(_recipient:RoleInfo[] ,
_indexValue:number[]):Promise<void>
    {
        return new Promise(async (resolve)=>
        {
            try
            {
                let tempRole;

                let allAwait=[];
                if(_recipient[0] &&
this.roleNodes[_recipient[0].index])
                {

allAwait.push(this.roleNodes[_recipient[0].index].getComponent
(RoleDis.RoleDis).ShiftPos(this.locationTemp[_indexValue[0]].w
orldPosition));
                }
                if(_recipient[1] &&
this.roleNodes[_recipient[1].index])
                {

allAwait.push(this.roleNodes[_recipient[1].index].getComponent
(RoleDis.RoleDis).ShiftPos(this.locationTemp[_indexValue[1]].w
orldPosition));
                }

                await Promise.all(allAwait);

                tempRole = this.roleNodes[_indexValue[0]];
                this.roleNodes[_indexValue[0]] =
this.roleNodes[_indexValue[1]];
                this.roleNodes[_indexValue[1]] = tempRole;
                await sleep(500);
                resolve();
            }
            catch(err)
            {
                console.error("Queue 下的 SwitchRolePos 错误:",
err);
            }
        });
```

```
    }
}
```

```
/*
 * RoleDis.ts
 * author: Hotaru
 * 2023/10/04
 * 角色展示类
 */
import { _decorator, animation, CCInteger, TTFFont, Component,
Sprite, tween, Node, Vec3, Animation, SpriteFrame,
AnimationComponent, Prefab, instantiate, find, RichText,
settings, Tween, math, Texture2D, sp, Skeleton, Quat, color,
Event, Button, UITransform, game, director } from 'cc';
import { Role } from '../AutoChessBattle//role';
import * as BattleEnums from '../AutoChessBattle/BattleEnums';
import * as enums from '../../other/enums';
import { Battle } from '../AutoChessBattle//battle';
import * as skill from '../AutoChessBattle//skill/skill_base'
import { netDriver } from '../../netDriver/netDriver';
import { netGame } from '../../netDriver/netGame';
import { hub_call_gate_reverse_reg_client_hub_rsp } from
'../../serverSDK/gate';
import { BundleManager } from '../../bundle/BundleManager';
import { Bullet, BulletInfo } from './Bullet';
import * as singleton from '../../netDriver/netSingleton';
import { Fetters } from '../../serverSDK/common';
import { config } from '../AutoChessBattle/config/config';
import { loadAssets } from '../../bundle/LoadAsset';
import { delay, sleep } from '../../other/sleep';
import { AudioManager } from '../../other/AudioManager';
import { SendMessage } from '../../other/MessageEvent';
import * as common from '../../serverSDK/common';
import { EffectSpine } from './EffectSpine';
import { SkillDis } from './SkillDis';
import { SpEffectOnRole, spEffectObj } from '../../other/
SpEffect';
const { ccclass, property } = _decorator;

@ccclass('RoleDis')
export class RoleDis extends Component
{
    @property({
        type: CCInteger,
        displayName: "角色ID"
    })
    public RoleId: number;
```

```
    @property({
        type: Prefab,
        displayName: "子弹预制体"
    })
    public remoteNode: Prefab;

    @property({
        type: Prefab,
        displayName: "增益提示"
    })
    public intensifierTip: Prefab;
    //血量和攻击
    public Hp: number;
    public AtkNum: number;
    //等级和经验
    public Level: number;
    public Exp: number;
    //角色立绘(骨骼动画)
    public roleSprite: sp.Skeleton;
    //角色信息
    private roleInfo: Role = null;
    //生命和攻击文本
    private hpText: RichText;
    private atkText: RichText;
    private levelText: RichText;
    //提示字符
    private behurtedTextEffect: Node;
    private beHurtedText: Node;
    //受伤效果
    private bandage: Node;
    private hurtedSpine: Node;
    //攻击缓动
    private tAttack: Tween<Node> = null;
    //位移缓动
    private tShiftpos: Tween<Node> = null;
    //死亡锁
    private isDead = false;
    //初始位置
    private originalPos: Vec3;
    //id字符
    private idText: RichText;
    //字体
    private typeface: TTFFont;
    //特效效果
    private effectSpine: Node;
    //技能表现类
    private skillDis: SkillDis;
    //受伤数字设置器
    public set BeHurtedNum(value: number)
    {
```

```
        this.beHurtedText.getComponent(RichText).font =
this.typeface;
        this.beHurtedText.getComponent(RichText).string =
"<color=#ad0003><outline color=#f05856 width=4>-" + value +
"</outline></color>";
    }
    //总受伤值
    private hurtedNum: number = 0;
    //受伤缓动
    private tBeHurted: Tween<Node> = null;

    private RoleSpEffect: SpEffectOnRole;

    protected async onLoad(): Promise<void>
    {
        try
        {
            this.typeface = (await
BundleManager.Instance.loadAssetsFromBundle("Typeface",
"MAOKENASSORTEDSANS")) as TTFFont;

            //this.levelSprite =
this.node.getChildByName("LevelSprite");
            this.bandage =
this.node.getChildByName("Bandage");
            this.behurtedTextEffect =
this.node.getChildByName("BeHertedTextEffect");
            this.beHurtedText =
this.behurtedTextEffect.getChildByName("BeHurtedText");
            this.hurtedSpine =
this.node.getChildByPath("BeHurtedSpine");
            this.effectSpine =
this.node.getChildByPath("EffectSpine");

            this.bandage.active = false;
            this.behurtedTextEffect.active = false;
            this.hurtedSpine.active = false;
            this.hpText = this.node.getChildByPath("Hp/
HpText").getComponent(RichText);
            this.atkText = this.node.getChildByPath("Atk/
AtkText").getComponent(RichText);
            this.levelText = this.node.getChildByPath("Level/
LevelText").getComponent(RichText);

            this.idText =
this.node.getChildByPath("ID").getComponent(RichText);
            //this.typeface =
BundleManager.Instance.loadAssetsFromBundle("Typeface",
"MAOKENASSORTEDSANS");
            //this.hpText.font = this.atkText.font =
```

```
this.typeface;

            this.AttackInit();
        }
        catch (err)
        {
            console.warn("RoleDis 下的 onLoad 错误 err:" +
err);
        }
    }

    async start()
    {
        if (this.roleInfo)
        {
            if (this.hpText && this.atkText)
            {
                this.hpText.string = "<color=#9d0c27><outline
color=#e93552 width=4>" + this.Hp + "</outline></color>";
                this.atkText.string = "<color=#f99b08><outline
color=#fff457 width=4>" + this.AtkNum + "</outline></color>";
                this.levelText.string =
"<color=#7CFC0><outline color=#7FFF00 width=4>" + this.Level +
"</outline></color>";

                this.idText.string = "<color=#9d0c27>" +
this.roleInfo.id;
            }
        }

        if (null == this.intensifierTip)
        {
            this.intensifierTip = await
BundleManager.Instance.loadAssetsFromBundle("TextTipPrefabs",
"IntensifierTip") as Prefab;
        }

        if (null == singleton.netSingleton.battle)
        {
            this.node.getComponent(Button).enabled = false;
        }
        else
        {
            this.node.on(Button.EventType.CLICK, () =>
            {
                if (singleton.netSingleton.battle)
                {
                    singleton.netSingleton.battle.puase =
true;
                    AudioManager.Instance.PlayerOnShot("Sound/
```

```
sound_click_01");
                    this.node.dispatchEvent(new
SendMessage('OpenInfoBoard', true, { id: this.RoleId, role:
this, isBuy: true }));
                }
            })
        }

    }

    public async Refresh(roleInfo: Role, isnew?: boolean)
    {
        return new Promise<void>(async (resolve, reject) =>
        {
            try
            {
                this.roleInfo = roleInfo;
                if (isnew)
                {
                    this.RoleId = roleInfo.id;
                    let str = "Role_" + this.RoleId;
                    if (null == this.idText)
                    {
                        this.idText =
this.node.getChildByPath("ID").getComponent(RichText);
                    }
                    this.idText.string = "<color=#9d0c27>" +
this.roleInfo.id;
                    // let sf:sp.SkeletonData=await
this.LoadImg("RolesImg",str);
                    // if(sf)
                    // {
                    //
this.node.getChildByName("Sprite").getComponent(Sprite).sprite
Frame=sf;
                    //     this.roleSprite=sf;
                    // }
                    await this.LoadOnConfig();
                    this.skillDis = new SkillDis(this.node,
roleInfo.index);
                    this.RoleSpEffect = new
SpEffectOnRole(roleInfo.id, this.node);
                    await Promise.all([this.skillDis.Init(),
this.RoleSpEffect.init()]);
                }
                this.ChangeAtt();
                resolve();
            }
            catch (error)
            {
```

```
                console.error("RoleDis 下的 Refresh 错误 err:"
+ error);
                reject();
            }
        })
    }

    public GetRoleFetter(): Fetters
    {
        if (this.roleInfo) return this.roleInfo.fetter;
    }

    public GetRoleSkillID(): number
    {
        if (this.roleInfo) return this.roleInfo.skillid;
    }

    public GetRoleInfo()
    {
        if (this.roleInfo) return this.roleInfo;
    }

    public AttackInit()
    {
        this.originalPos = new Vec3(this.node.position);
    }

    /*
    * 添加
    * author: Guanliu
    * 2024/04/20
    * 为人物添加装备
    */
    public Equipping(equipId: number)
    {
        this.roleInfo.equip = equipId;
    }

    public Attack(readyLocation: Vec3, battleLocation: Vec3,
camp: BattleEnums.Camp)
    {
        return new Promise<void>((resolve, reject) =>
        {
            try
            {
                this.node.setSiblingIndex(90);
                this.tAttack = tween(this.node)
                    .to(0.15, { worldPosition:
readyLocation })
                    //这里做出蓄力效果
```

```
                    .delay(0.1)
                    .to(0.05, { worldPosition: new
Vec3(readyLocation.x + 15, readyLocation.y,
readyLocation.z) })
                    .to(0.05, { worldPosition: new
Vec3(readyLocation.x - 15, readyLocation.y,
readyLocation.z) })
                    .to(0.05, { worldPosition: new
Vec3(readyLocation.x, readyLocation.y + 15,
readyLocation.z) })
                    .to(0.05, { worldPosition: new
Vec3(readyLocation.x, readyLocation.y - 15,
readyLocation.z) })
                    //上面是蓄力效果
                    .to(0.05, { worldPosition:
battleLocation })
                    .call(() =>
                    {
                        if (BattleEnums.Camp.Self == camp)
                        {
                            
singleton.netSingleton.battle.ShowBattleEffect();
                        }
                    })
                    .call(() =>
                    {
                        this.ChangeAtt();
                        this.ResetPos(readyLocation);
                        resolve();
                    })
                    .start();
            } catch (error)
            {
                console.error("RoleDis 下的 Attack 错误 err:" +
error);
            }
        });
    }

    //异步执行将对撞角色归位，防止阻碍到后续判断
    public async ResetPos(readyLocation: Vec3)
    {
        this.tAttack = tween(this.node)
            .to(0.1, { worldPosition:
readyLocation }).start();
        return delay(100, () =>
        {
            if (this.tAttack)
            {
                this.tAttack.stop();
```

```
                this.tAttack = null;
            }
        });
    }

    public async ChangeAtt()
    {
        try
        {
            if (!this.roleInfo.getShields())
            {
                this.RoleSpEffect.RemoveBuffEffect(BattleEnums.BufferType.Shields);
            }

            if (null == this.hpText && null == this.atkText)
            {
                this.hpText = this.node.getChildByPath("Hp/HpText").getComponent(RichText);
                this.atkText = this.node.getChildByPath("Atk/AtkText").getComponent(RichText);
                this.levelText = this.node.getChildByPath("Level/LevelText").getComponent(RichText);
            }

            let _hp = Math.round(this.roleInfo.GetProperty(BattleEnums.Property.HP));
            if (this.Hp != _hp)
            {
                tween(this.hpText.node).to(0.1, { scale: new Vec3(0, 0, 0) }).call(() =>
                {
                    this.hpText.string = "<color=#9d0c27><outline color=#e93552 width=4>" + _hp + "</outline></color>";
                    this.Hp = _hp;
                }).by(0.2, { scale: new Vec3(0.5, 0.5, 0.5) }).start();

            }
            let _atk = Math.round(this.roleInfo.GetProperty(BattleEnums.Property.Attack));
            if (this.AtkNum != _atk)
            {
                tween(this.atkText.node).to(0.1, { scale: new
```

```
Vec3(0, 0, 0) }).call(() =>
                {
                    this.atkText.string =
"<color=#f99b08><outline color=#fff457 width=4>" + _atk + "</
outline></color>";
                    this.AtkNum = _atk
                }).by(0.2, { scale: new Vec3(0.5, 0.5,
0.5) }).start();
            }
            if (this.Level != this.roleInfo.level)
            {
                tween(this.levelText.node).to(0.1, { scale:
new Vec3(0, 0, 0) }).call(() =>
                {
                    this.levelText.string =
"<color=#7CFC0><outline color=#7FFF00 width=4>" +
this.roleInfo.level + "</outline></color>";
                    this.Level = this.roleInfo.level;
                }).by(0.2, { scale: new Vec3(0.5, 0.5,
0.5) }).start();
            }

            //console.log("changeAtt RoleDis.roleInfo:",
this.roleInfo);
            //console.log("changeAtt RoleDis:", this);
            return delay(100, () => { });
        }
        catch (err)
        {
            console.error("RoleDis 下的 changeAtt 错误 err:" +
err);
        }
    }

    public Exit()
    {
        try
        {

this.bandage.getComponent(Animation).on(Animation.EventType.FI
NISHED, () =>
            {
                //
singleton.netSingleton.battle.showBattleEffect(false);
                this.node.active = false;
            });

            let offset = -1000;
            if (BattleEnums.Camp.Self !=
this.roleInfo.selfCamp) offset = 1000;
```

```
            let hitAnim: Animation =
this.node.getChildByName("Sprite").getComponent(Animation);
            tween(this.node)
                .call(() =>
                {
                    // hitAnim.resume();
                    //
this.hurtedSpine.getComponent(sp.Skeleton).animation="animatio
n";
                    // this.hurtedSpine.active=true;
                    // hitAnim.play();

this.node.getChildByName("Sprite").getComponent(sp.Skeleton).c
olor = color(110, 110, 110, 255);
                    this.RoleRotate();
                })
                // .delay(0.2).call(()=>
                // {
                //     this.hurtedSpine.active=false;
                // })
                .by(0.7, { position: new
Vec3(this.node.position.x + offset, this.node.position.y +
500) }, { easing: 'quintIn' })
                .to(0.7, { position: new
Vec3(this.node.position.x + offset, this.node.position.y +
500) })
                .delay(0.2).call(() =>
                {
                    this.isDead = true;
                    this.roleInfo = null;
                    console.log("销毁角色");
                    this.node.destroy();
                })
                .start();
            return delay(200, () => { });
        }
        catch (err)
        {
            console.warn("RoleDis 下的 Exit 错误 err:" + err);
        }

    }

    private async RoleRotate()
    {
        const rotationAxis = new Vec3(0, 1, 0);
        let rotationSpeed = 4.0;
        try
        {
            while (!this.isDead)
```

```
            {
                await delay(0, () => { }); // 让出控制权，以便游戏引擎处理其他事务
                const deltaRotation = Quat.fromEuler(new Quat(), 0, rotationSpeed, 0);
                this.node.setRotation(Quat.multiply(new Quat(), this.node.rotation, deltaRotation));
            }
        } catch
        {
            console.log("角色停止旋转，人物已被销毁");
        }
    }

    //显示受伤缓动
    private ShowHurtedTween()
    {
        if (!this.tBeHurted)
        {
            let hurtedTextAnim: Animation = this.behurtedTextEffect.getComponent(Animation);
            hurtedTextAnim.on(Animation.EventType.FINISHED, () =>
            {
                hurtedTextAnim.stop();
                this.behurtedTextEffect.active = false;
                hurtedTextAnim.resume();
            }, this);
            let hitAnim: Animation = this.node.getChildByName("Sprite").getComponent(Animation);
            this.tBeHurted = tween(this.node).to(0, {}).call(() =>
            {
                hurtedTextAnim.resume();
                hitAnim.resume();
                if(this.BeHurtedNum!=0)
                {
                    this.behurtedTextEffect.active = true;
                }
                hurtedTextAnim.play();
                hitAnim.play();
            }).start();
        }
    }

    public async BeHurted(_ev: skill.Event,_style?:number)
    {
        try
        {
            if(_ev.value[0]!=0)
```

```
            {
                this.hurtedNum += _ev.value[0];
                this.BeHurtedNum = this.hurtedNum;
            }
            this.ShowHurtedTween();
            if(BattleEnums.EventType.AttackInjured == _ev.type
&& _ev.spellcaster.camp == BattleEnums.Camp.Self)
            {
                let roleConfig =
config.RoleConfig.get(this.RoleId);
                let audioString = "Sound/
sound_character_hit_MN";
                if (undefined != roleConfig.Sex && undefined !
= roleConfig.Armor)
                {
                    audioString = "Sound/sound_character_hit_"
+ roleConfig.Sex + roleConfig.Armor;
                }

AudioManager.Instance.PlayerOnShot(audioString);
            }
            if(BattleEnums.EventType.RemoteInjured ==
_ev.type)
            {

this.hurtedSpine.getComponent(sp.Skeleton).animation =
"animation";
                this.hurtedSpine.active = true;
            }

            return delay(700, () =>
            {
                this.hurtedNum = 0;
                this.tBeHurted = null;
                this.hurtedSpine.active = false;
            });
        }
        catch (err)
        {
            console.error("RoleDis 下的 BeHurted 错误 err:" +
err);
        }
    }
```

```
    public async IntensifierExp(value: number,_isColony:
boolean)
    {
        let exp = this.Exp + value;
        if (exp < 3)
        {
```

```
            this.Exp = exp;
        }
        else
        {
            this.Exp = exp % 3;
            this.Level += exp % 3;
        }
        let allAwait=[];

allAwait.push(this.RoleSpEffect.UseIntensifierEffect(_isColony
, 4));
        allAwait.push(this.ChangeAtt());

        await Promise.all(allAwait);
    }
    /**
     * -增益效果表现-
     * @param value 增益数值
     * @param _isColony 是否为群体
     * @param stack 经验值
     */
    public async Intensifier(value: number[], _isColony:
boolean, stack?: number)
    {
        try
        {
            AudioManager.Instance.PlayerOnShot("Sound/
battle_attribute_increment");
            if (stack)
            {
                this.Exp = stack % 3;
            }

            let style = 1;
            if (!_isColony)
            {
                if (0 == value[0] && value[1] != 0) style = 2;
                if (value[0] != 0 && value[1] != 0) style = 1;
                if (value[0] != 0 && 0 == value[1]) style = 5;
            }
            else
            {
                if (0 == value[0] && value[1] != 0) style = 1;
                if (value[0] != 0 && 0 == value[1]) style = 2;
                if (value[0] != 0 && value[1] != 0) style = 3;

            }

            let allAwait=[];
```

```
allAwait.push(this.RoleSpEffect.UseIntensifierEffect(_isColony
, style));
            allAwait.push(this.ChangeAtt());

            allAwait.push(new Promise<void>(async (resolve)=>
            {
                for (let i = 0; i < value.length; i++)
                {
                    let tip: Node;
                    if (value[i] != 0)
                    {
                        tip =
instantiate(this.intensifierTip);
                        if (0 == i)
                        {

tip.getChildByPath("RichText").getComponent(RichText).string =
"<color=#ad0003><outline color=#f05856 width=4>+" + value[i] +
"</outline></color>";
                        }
                        else
                        {

tip.getChildByPath("RichText").getComponent(RichText).string =
"<color=#ffa900><outline color=#ffe900 width=4>+" + value[i] +
"</outline></color>";
                        }
                        tip.setParent(this.node);
                        let anim =
tip.getComponent(Animation);
                        anim.getState('Tips').speed = 1.5;
                        anim.on(Animation.EventType.FINISHED,
(event) =>
                        {
                            tip.destroy();
                        });

                        anim.play();
                    }
                    await sleep(750);
                }
                resolve();
            }));

            await Promise.all(allAwait);
        }
        catch (err)
        {
```

```
            console.error("RoleDis 下的 Intensifier 错误 err:"
+ err);
        }

    }

    //缓动有bug暂时空置
    public async LevelUp()
    {
        try
        {
            AudioManager.Instance.PlayerOnShot("Sound/
sound_herolvup_01");
            //let str="lvl_"+_level;
            //let sf:SpriteFrame=await
this.LoadImg("LvRing",str);
            tween(this.node).to(0.1,
                {
                    //scale:new Vec3(1.1,1.1,1)
                })
                .call(() =>
                {
                    //this.Level=_level;
                    //if(sf)
                    //{
                    //
this.levelSprite.getComponent(Sprite).spriteFrame=sf;
                    //}
                })
                .delay(0.1).to(0.1,
                    {
                        //scale:new Vec3(1,1,1)
                    })
                .start();

            return delay(300, () =>
            {

            })
        }
        catch (error)
        {
            console.error("RoleDis 下的 LevelUp 错误 err:" +
error);
        }

    }

    public async ShiftPos(vec: Vec3, atkInit?: boolean)
    {
```

```
        return new Promise<void>(async (resolve, reject) =>
        {
            try
            {
                console.log(`shiftPos begin!`);
                //await
this.spEffect.CheckSkillEffect({key:"skill_0014",battleType :
null});
                await this.RoleSpEffect.CheckSkillEffect(new
spEffectObj("skill_0014", null));
                //开始缓动
                this.tShiftpos = tween(this.node).to(0.3,
{ worldPosition: vec }).call(() =>
                {
                    if (this.tShiftpos)
                    {
                        this.tShiftpos.stop();
                        this.tShiftpos = null;
                        console.log("shiftPos end!");
                    }
                    if (atkInit) this.AttackInit();
                    resolve();
                }).start();
            }
            catch (error)
            {
                console.error("RoleDis 下的 ShiftPos 错误 err:"
+ error);
                reject();
            }
        });
    }
    /**
     * -使用技能表现-
     * @param  skill 技能
     * @author: Hotaru
     * @time 2024/08/24
     */
    public async UseSkill(_ev: skill.Event)
    {
        let allAwait=[];
        allAwait.push(this.RoleSpEffect.UseSkillEffect());
        allAwait.push(this.skillDis.UseSkill(_ev));
        await Promise.all(allAwait);
    }
    /**
     * -技能发动时表现-
     * @author: Hotaru
     * @time 2024/08/24
     */
```

```
    public async OnSkill(_isFetter:boolean,_obj?:any | BulletInfo)
    {
        if(_obj && _obj as BulletInfo)
        {
            let bi:BulletInfo=_obj;
            return this.UseProjectiles(bi.selfpos,bi.targetPos,bi.isGain,_isFetter,bi.style);
        }
        else
        {
            return this.RoleSpEffect.OnSkillEffect(_isFetter);
        }

    }
    /**
     * -交换属性-
     * @param _swapType 类型
     * @author Hotaru
     * @time 2024/11/30
     */
    public async SwapProperties(_swapType: BattleEnums.SwapPropertiesType)
    {
        let allAwait=[];
        switch (_swapType)
        {
            case BattleEnums.SwapPropertiesType.AttackSwap:
            case BattleEnums.SwapPropertiesType.HpSwap:
                {
                    //await this.spEffect.CheckSkillEffect({ key: "skill_0024", battleType: _swapType });
allAwait.push(this.RoleSpEffect.CheckSkillEffect(new spEffectObj("skill_0024", _swapType)));
                }
                break;
        }

        allAwait.push(this.ChangeAtt());

        await Promise.all(allAwait);
    }
    /**
     * -接收buff表现-
     * @author Hotaru
     * @time 2024/08/24
     */
```

```
    public async ReceptionBuff(_buff: BattleEnums.BufferType)
    {
        try
        {
            let key = "";
            switch (_buff)
            {
                case BattleEnums.BufferType.Weak: key = 
"skill_0015"; break;
            }
            if (key.length > 0)
            {
                await this.RoleSpEffect.CheckSkillEffect(new 
spEffectObj(key, null));
            }
            await this.RoleSpEffect.BuffEffect(_buff);
        } catch (error)
        {
            console.error("RoleDis 下的 ReceptionBuff 错
误:",error);
        }
    }
    /**
     * -转移伤害-
     * @author Hotaru
     * @time 2024/11/30
     */
    public async DeflexionDamage()
    {
        //await 
this.spEffect.CheckSkillEffect({key:"skill_0013_1" , 
battleType : null});
        return this.RoleSpEffect.CheckSkillEffect(new 
spEffectObj("skill_0013_1", null));
    }
    /**
     * -承受伤害-
     * @author Hotaru
     * @time 2024/11/30
     */
    public async SubstituteDamage()
    {
        // await 
this.spEffect.CheckSkillEffect({key:"skill_0013_2" , 
battleType : null});
        return this.RoleSpEffect.CheckSkillEffect(new 
spEffectObj("skill_0013_2", null));
    }
    /**
     * -召唤入场效果-
```

```
     * @author Hotaru
     * @time 2024/09/04
     */
    public async OnSummon()
    {
        await this.RoleSpEffect.SummonEffect();
        this.roleSprite.node.active = true;
        this.atkText.node.active = true;
        this.hpText.node.active = true;
        this.levelText.node.active = true;
    }
    /**
     * -使用飞行物-
     * @param  _self 发射位置
     * @param _target 目标位置
     * @param _isGain 是否是增益效果
     */
    public async UseProjectiles(_self: Vec3, _target: Vec3,
_isGain: boolean , _isFetter:boolean , style?:number)
    {
        let fetterID=0;
        if(_isFetter)
        {
            fetterID = this.roleInfo.fetter.fetters_id;
        }
        return this.RoleSpEffect.ProjectilesEffect(_self,
_target, _isGain , style ,fetterID);
    }
    /**
     * -从配置文件加载-
     * @author Hotaru
     * @time 2024/03/06
     */
    private LoadOnConfig()
    {
        return new Promise<void>((resolve, reject) =>
        {
            try
            {
                let jconfig =
config.RoleConfig.get(this.RoleId);
                this.roleSprite =
this.node.getChildByPath("Sprite").getComponent(sp.Skeleton);
                loadAssets.LoadSkeletonData(jconfig.Skel,
(data) =>
                {
                    if (data)
                    {
                        try
                        {
```

```
                            this.roleSprite.skeletonData =
data;
                            let anims = data.getAnimsEnum();
                            //
this.roleSprite.animation="animation";
                            this.roleSprite.setAnimation(0,
String(anims[1]), true);
                            resolve();
                        }
                        catch (error)
                        {
                            console.warn(`角色 ${jconfig.Id} 的
动画设置失败: `, error);
                            resolve();
                        }
                    }
                });
            }
            catch (error)
            {
                console.error(`RoleDis 下的 LoadOnConfig 错误
err:${error} RoleId:${this.RoleId}`);
                reject();
            }
        })
    }
}
```

```
import { _decorator, Component, instantiate, Node, Prefab,
UITransform, Vec3 } from 'cc';
import * as skill from '../AutoChessBattle//skill/skill_base';
import * as singleton from '../../netDriver/netSingleton';
import * as BattleEnums from '../AutoChessBattle/BattleEnums';
import { Bullet, BulletInfo } from './Bullet';
import { delay } from '../../other/sleep';
import { BundleManager } from '../../bundle/BundleManager';
import { RoleDis } from './RoleDis';
import { Role as rRole } from '../AutoChessBattle/role';
import { spEffectObj } from '../../other/SpEffect';
const { ccclass, property } = _decorator;

export class SkillDis
{
    //父节点
    private parent: Node;
    //自身位置
    private index: number;
    //子弹预制体
```

```
    private remoteNode: Prefab;

    constructor(_parent: Node, _index: number)
    {
        this.parent = _parent;
        this.index = _index;
    }

    public async Init()
    {

    }

    public UseSkill(_ev: skill.Event): Promise<void>
    {
        return new Promise(async (resolve) =>
        {
            switch (_ev.type)
            {
                case BattleEnums.EventType.RemoteInjured:
                    {
                        await this.RemoteAttack(_ev);
                    }
                    break;
                case BattleEnums.EventType.IntensifierExp:
                case
BattleEnums.EventType.IntensifierProperties:
                    {
                        await this.DeliveryGain(_ev);
                    }
                    break;
                case BattleEnums.EventType.Summon:
                    {
                        await this.Summon(_ev);
                    }
                    break;
                case BattleEnums.EventType.ChangeLocation:
                    {
                        await this.TransPosition(_ev);
                    }
                    break;
                case BattleEnums.EventType.SwapProperties:
                    {
                        await this.SwapProperties(_ev);
                    }
                    break;
                case BattleEnums.EventType.AddBuff:
                case BattleEnums.EventType.GiveShields:
                    {
                        await this.AddBuff(_ev);
```

```
                    }
                    break;
                case BattleEnums.EventType.SubstituteDamage:
                    {
                        await this.SubstituteDamage(_ev);
                    }
                    break;
            }
            resolve();
        });
    }
    /**
     * -远程攻击-
     * @param  ev 事件
     * @returns 回调
     *
     * author: Hotaru
     * 2024/11/24
     */
    private async RemoteAttack(_ev: skill.Event)
    {
        return new Promise<void>(async (resolve, reject) =>
        {
            try
            {
                let spList = BattleEnums.Camp.Self ==
_ev.spellcaster.camp ?
singleton.netSingleton.battle.selfQueue :
singleton.netSingleton.battle.enemyQueue;
                let self = this.parent;
                let selfpos =
singleton.netSingleton.battle.panelNode.getComponent(UITransfo
rm).convertToNodeSpaceAR(self.getWorldPosition());

                if(_ev.objCount>=6)
                {
                    await this.RemoteAttackColony(_ev);
                    resolve();
                }

                for (let element of _ev.recipient)
                {
                    let targetList = BattleEnums.Camp.Enemy ==
element.camp ? singleton.netSingleton.battle.enemyQueue :
singleton.netSingleton.battle.selfQueue;
                    let target =
targetList.roleNodes[element.index];

                    if (self && target)
                    {
```

```
                        let targetpos =
singleton.netSingleton.battle.panelNode.getComponent(UITransfo
rm).convertToNodeSpaceAR(target.getWorldPosition());

                        await
this.parent.getComponent(RoleDis).OnSkill(_ev.isFetter,new
BulletInfo(selfpos,targetpos,false)).then(async ()=>
                        {
                            await
target.getComponent(RoleDis).BeHurted(_ev);
                            await
target.getComponent(RoleDis).ChangeAtt();
                        });
                    }
                }
                resolve();
            }
            catch (err)
            {
                console.error("SkillDis 下的 RemoteAttack 错误
err:" + err);
                reject();
            }
        })
    }
    /**
     * -群伤远程攻击-
     * @param _ev
     */
    private async RemoteAttackColony(_ev: skill.Event)
    {
        return new Promise<void>(async (resolve, reject)=>
        {
            try
            {
                let spList = BattleEnums.Camp.Self ==
_ev.spellcaster.camp ?
singleton.netSingleton.battle.selfQueue :
singleton.netSingleton.battle.enemyQueue;
                let self = this.parent;

                let allAwait=[];

                for (let element of _ev.recipient)
                {
                    let targetList = BattleEnums.Camp.Enemy ==
element.camp ? singleton.netSingleton.battle.enemyQueue :
singleton.netSingleton.battle.selfQueue;

                    let target =
```

```
targetList.roleNodes[element.index];

                    if (self && target)
                    {
                        allAwait.push(new Promise<void>(async
(resolve)=>
                        {
                            await
target.getComponent(RoleDis).BeHurted(_ev);
                            await
target.getComponent(RoleDis).ChangeAtt();
                            resolve();
                        }));
                    }
                }
                await
this.parent.getComponent(RoleDis).OnSkill(_ev.isFetter).then(a
sync ()=>
                {
                    await Promise.all(allAwait);
                    resolve();
                });
            }
            catch(err)
            {
                console.error("SkillDis 下的 RemoteAttackColony
错误 err:" + err);
                reject();
            }
        })
    }
    /**
    * -使用增益-
    * @param _ev 事件
    *
    * author: Hotaru
    * 2024/11/24
    */
    private async DeliveryGain(_ev: skill.Event)
    {
        return new Promise<void>(async (resolve, reject) =>
        {
            try
            {
                let self = this.parent;
                let allAwait=[];
                let isColony=_ev.recipient.length>1?
true:false;

                for (let element of _ev.recipient)
```

```
                {
                    if (element.index == this.index &&
_ev.recipient.length<=1)
                    {
                        await
this.parent.getComponent(RoleDis).Intensifier(_ev.value,
false);
                        break;
                    }
                    let targetList = BattleEnums.Camp.Enemy ==
element.camp ? singleton.netSingleton.battle.enemyQueue :
singleton.netSingleton.battle.selfQueue;
                    let target =
targetList.roleNodes[element.index];

                    if (self && target)
                    {
                        let selfpos =
singleton.netSingleton.battle.panelNode.getComponent(UITransfo
rm).convertToNodeSpaceAR(self.getWorldPosition());
                        let targetpos =
singleton.netSingleton.battle.panelNode.getComponent(UITransfo
rm).convertToNodeSpaceAR(target.getWorldPosition());

                        let style=_ev.value[0]?1:3;

if(BattleEnums.EventType.IntensifierExp ==_ev.type)
                        {
                            style=2;
                        }

allAwait.push(this.parent.getComponent(RoleDis).OnSkill(_ev.is
Fetter,new
BulletInfo(selfpos,targetpos,true,style)).then(async ()=>
                        {
                            switch (_ev.type)
                            {
                                case
BattleEnums.EventType.IntensifierProperties:
                                    {
                                        await
target.getComponent(RoleDis).Intensifier(_ev.value, isColony);
                                    }
                                    break;
                                case
BattleEnums.EventType.IntensifierExp:
                                    {
                                        await
target.getComponent(RoleDis).IntensifierExp(_ev.value[0],isCol
```

```
ony);
                                    }
                                    break;
                                }
                            }));
                        }
                    }
                    if(allAwait.length>0)
                    {
                        await Promise.all(allAwait);
                    }
                    resolve();
                } catch (error)
                {
                    console.error("SkillDis 下的 DeliveryGain 错误
err:" + error);
                    reject();
                }
            })
            //return delay(1200, () => { });
        }

        /**
         * -召唤-
         * @param _ev 事件
         *
         * @author: Hotaru
         * @time 2024/11/24
         */
        private Summon(_ev: skill.Event)
        {
            return new Promise<void>(async (resolve, reject) =>
            {
                try
                {
                    for (let element of _ev.recipient)
                    {
                        let tmp: rRole;
                        tmp = new rRole(null, element.index,
element.id, 1, 0, element.camp, element.properties, null, 0);
                        let targetTeam = BattleEnums.Camp.Self ==
element.camp ?
singleton.netSingleton.battle.battleCentre.GetSelfTeam() :
singleton.netSingleton.battle.battleCentre.GetEnemyTeam();
                        targetTeam.AddRole(tmp);
                        let queue = BattleEnums.Camp.Self ==
element.camp ? singleton.netSingleton.battle.selfQueue :
singleton.netSingleton.battle.enemyQueue;
                        await queue.SummonRole([tmp],
_ev.spellcaster);
```

```
                    resolve();
                }
            } catch (error)
            {
                console.error("SkillDis 下的 Summon 错误 err:"
+ error);
                reject();
            }
        })
    }
    /**
     * -换位-
     * @param _ev 事件
     *
     * @author: Hotaru
     * @time 2024/11/24
     */
    private TransPosition(_ev: skill.Event)
    {
        return new Promise<void>(async (resolve, reject) =>
        {
            try
            {
                let queue = BattleEnums.Camp.Self ==
_ev.spellcaster.camp ?
singleton.netSingleton.battle.enemyQueue :
singleton.netSingleton.battle.selfQueue;
                await queue.SwitchRolePos(_ev.recipient,
_ev.value);
                resolve();
            } catch (error)
            {
                console.error("SkillDis 下的 TransPosition 错误
err:" + error);
                reject();
            }
        })
    }
    /**
     * -交换属性-
     * @param _ev 事件
     *
     * @author: Hotaru
     * @time 2024/11/24
     */
    private SwapProperties(_ev: skill.Event)
    {
        return new Promise<void>(async (resolve, reject) =>
        {
            try
```

```
            {
                await
this.parent.getComponent(RoleDis).SwapProperties(_ev.value[0])
;
                resolve();
            } catch (error)
            {
                console.error("SkillDis 下的 SwapProperties 错
误 err:" + error);
                reject();
            }
        })
    }

    /**
     * -加buff-
     * @param _ev 事件
     *
     * @author: Hotaru
     * @time 2024/11/24
     */
    private AddBuff(_ev:skill.Event)
    {
        new Promise<void>(async (resolve, reject) =>
        {
            try
            {
                let spList = BattleEnums.Camp.Self ==
_ev.spellcaster.camp ?
singleton.netSingleton.battle.selfQueue :
singleton.netSingleton.battle.enemyQueue;

                for (let element of _ev.recipient)
                {
                    await
spList.roleNodes[element.index].getComponent(RoleDis).Receptio
nBuff(_ev.value[0]);
                }
                resolve();
            } catch (error)
            {
                console.error("SkillDis 下的 AddBuff 错误 err:"
+ error);
                reject();
            }
        })
    }
    /**
     * -转移伤害-
     * @param _ev 事件
```

```
     */
    private SubstituteDamage(_ev:skill.Event)
    {
        return new Promise<void>(async (resolve, reject) =>
        {
            try
            {
                let spList =
singleton.netSingleton.battle.selfQueue;

                for (let element of _ev.recipient)
                {
                    if (spList.roleNodes[element.index]) {
                        await
spList.roleNodes[element.index].getComponent(RoleDis).Deflexio
nDamage();
                    }
                }
                if (this.parent) {
                    await
this.parent.getComponent(RoleDis).SubstituteDamage();
                }
                resolve();
            } catch (error)
            {
                console.error("SkillDis 下的 SubstituteDamage
错误 err:" + error);
                reject();
            }
        });
    }
}


/*
 * battle.ts
 * author: qianqians
 * 2023/9/24
 */
import * as common from "../../serverSDK/common";
import * as skill from "./skill/skill_base";
import * as role from "./role";
import * as team from "./team";
import * as battleEnums from "./BattleEnums";
import * as enums from "../../other/enums";

function splitEvs(evs: skill.Event[]) {
  let injuredEvs: skill.Event[] = [];
  let normalEvs: skill.Event[] = [];
```

```
  for (let ev of evs) {
    switch (ev.type) {
      case battleEnums.EventType.RemoteInjured:
      case battleEnums.EventType.AttackInjured:
      case battleEnums.EventType.Syncope:
      case battleEnums.EventType.ChangeLocation:
      case battleEnums.EventType.IntensifierProperties:
      case battleEnums.EventType.SwapProperties:
        {
          injuredEvs.push(ev);
        }
        break;
      default:
        {
          normalEvs.push(ev);
        }
        break;
    }
  }

  return [injuredEvs, normalEvs];
}

export class Battle {
  private selfTeam: team.Team;
  private enemyTeam: team.Team;

  public evs: skill.Event[] = [];

  public victory = 0;
  public faild = 0;
  public round = 0;
  public addCoin = 0;

  public gamemode: enums.GameMode;

  public on_event: (evs: skill.Event[]) => Promise<void> =
null;
  public onPlaySound: (eventSound: string) => void;
  public onPlayerOnShot: (eventSound: string) => void;
  public onKillRole: (r: common.Role) => void;

  public constructor(
    self: common.UserBattleData,
    enemy: common.UserBattleData,
    gamemode: enums.GameMode,
  ) {
    this.selfTeam = new team.Team(battleEnums.Camp.Self, self,
self.RoleList);
```

```
    this.enemyTeam = new team.Team(battleEnums.Camp.Enemy,
enemy, enemy.RoleList);

    this.selfTeam.onKillRole = (_r: common.Role) => {
      if (this.onKillRole) {
        this.onKillRole.call(null, _r);
      }
    };
    this.enemyTeam.onKillRole = (_r: common.Role) => {
      if (this.onKillRole) {
        this.onKillRole.call(null, _r);
      }
    };

    this.victory = self.victory;
    this.faild = self.faild;
    this.round = self.round;
    this.gamemode = gamemode;
  }

  public GetSelfTeam(): team.Team {
    return this.selfTeam;
  }

  public GetEnemyTeam(): team.Team {
    return this.enemyTeam;
  }

  public AddBattleEvent(ev: skill.Event) {
    this.evs.push(ev);
  }

  public StartBattle() {
    let ev = new skill.Event();
    ev.type = battleEnums.EventType.BattleBegin;
    console.log("推送战斗开始事件");
    this.AddBattleEvent(ev);
  }

  private battle() {
    console.log("battle begin!");

    let self = this.selfTeam.GetLasterRole();
    let enemy = this.enemyTeam.GetLasterRole();
    if (self && !self.CheckDead() && enemy && !
enemy.CheckDead()) {
      console.log("battle Attack");

      self.Attack(enemy, this);
      enemy.Attack(self, this);
```

```
      let ev = new skill.Event();
      ev.type = battleEnums.EventType.AfterAttack;
      ev.spellcaster = new skill.RoleInfo();
      ev.spellcaster.camp = self.selfCamp;
      ev.spellcaster.index = self.index;
      ev.recipient = [];
      this.AddBattleEvent(ev);

      ev = new skill.Event();
      ev.type = battleEnums.EventType.AfterAttack;
      ev.spellcaster = new skill.RoleInfo();
      ev.spellcaster.camp = enemy.selfCamp;
      ev.spellcaster.index = enemy.index;
      ev.recipient = [];
      this.AddBattleEvent(ev);
    }

    console.log("battle end!");
  }

  public GetWinCamp(): battleEnums.Camp {
    if (!this.CheckEndBattle()) {
      return battleEnums.Camp.None;
    }

    if (this.selfTeam.CheckDefeated() &&
this.enemyTeam.CheckDefeated()) {
      return battleEnums.Camp.Self;
    }

    if (this.selfTeam.CheckDefeated()) {
      return battleEnums.Camp.Enemy;
    }

    return battleEnums.Camp.Self;
  }

  public CheckEndBattle(): boolean {
    return (
      this.evs.length <= 0 &&
      (this.selfTeam.CheckDefeated() ||
this.enemyTeam.CheckDefeated())
    );
  }

  public CheckRemoveDeadRole() {
    return (
      this.selfTeam.CheckRemoveDeadRole(this) ||
      this.enemyTeam.CheckRemoveDeadRole(this)
```

```
    );
  }

  private async tickInjuredEventChain(evs: skill.Event[]) {
    if (evs == null || evs.length <= 0) {
      return;
    }

    console.log("tickInjuredEventChain evs:", evs);
    await this.on_event.call(null, evs);
    this.tickSkill(evs);

    if (this.CheckEndBattle()) {
      return;
    }
    this.CheckRemoveDeadRole();

    let _evs = this.evs.slice();
    this.evs = [];
    let [injuredEvs, normalEvs] = splitEvs(_evs);
    await this.tickInjuredEvent(injuredEvs);
    await this.tickInjuredEventChain(normalEvs);

    if (this.CheckEndBattle()) {
      return;
    }
    this.CheckRemoveDeadRole();

    _evs = this.evs.slice();
    this.evs = [];
    [injuredEvs, normalEvs] = splitEvs(_evs);
    await this.tickInjuredEvent(injuredEvs);
    await this.tickInjuredEventChain(normalEvs);
  }

  private async tickInjuredEvent(evs: skill.Event[]) {
    if (evs == null || evs.length <= 0) {
      return;
    }

    console.log("tickInjuredEvent evs:", evs);
    await this.on_event.call(null, evs);

    let tmpPriorityEvs: skill.Event[] = [];
    for (let ev of evs) {
      if (this.checkTriggerSkill([ev]) && !
tmpPriorityEvs.includes(ev)) {
        tmpPriorityEvs.push(ev);
        for (let _tmpev of evs) {
          if (
```

```
            _tmpev.roleParallel == ev.roleParallel &&
            !tmpPriorityEvs.includes(_tmpev)
          ) {
            tmpPriorityEvs.push(_tmpev);
          }
        }
      }
    }
    let tmpEvs: skill.Event[] = evs.filter(
      (element) => !tmpPriorityEvs.includes(element),
    );
    console.log("tickInjuredEvent tmpPriorityEvs:",
tmpPriorityEvs);
    console.log("tickInjuredEvent tmpEvs:", tmpEvs);
    if (tmpPriorityEvs.length > 0) {
      this.tickSkill(tmpPriorityEvs);

      if (this.CheckEndBattle()) {
        return;
      }
      this.CheckRemoveDeadRole();

      let _evs = this.evs.slice();
      this.evs = [];
      let [injuredEvs, normalEvs] = splitEvs(_evs);
      await this.tickInjuredEvent(injuredEvs);
      await this.tickInjuredEventChain(normalEvs);

      if (this.CheckEndBattle()) {
        return;
      }
      this.CheckRemoveDeadRole();

      _evs = this.evs.slice();
      this.evs = [];
      [injuredEvs, normalEvs] = splitEvs(_evs);
      await this.tickInjuredEvent(injuredEvs);
      await this.tickInjuredEventChain(normalEvs);
    }
  }

  private async tickBattleInjuredEvent(evs: skill.Event[]) {
    if (evs == null || evs.length <= 0) {
      return;
    }

    console.log("tickBattleInjuredEvent evs:", evs);

    await this.on_event.call(null, evs);
    this.tickSkill(evs);
```

```
    if (this.CheckEndBattle()) {
      return;
    }
    this.CheckRemoveDeadRole();

    let _evs = this.evs.slice();
    this.evs = [];
    let [injuredEvs, normalEvs] = splitEvs(_evs);
    await this.tickInjuredEvent(injuredEvs);
    await this.tickInjuredEventChain(normalEvs);
  }

  public CheckRoleTriggerSkill(role: role.Role, evs:
skill.Event[]) {
    let roleInfo = new skill.RoleInfo();
    roleInfo.properties = role.GetProperties();
    console.log("战斗事件中角色攻击力:");

    roleInfo.index = role.index;
    roleInfo.camp = battleEnums.Camp.Self;
    let p = 0;
    let skillImpl: skill.SkillBase = null;
    for (let skill of role.skill) {
      let flag = skill.trigger.CheckSkillTrigger(evs,
roleInfo);
      if (flag) {
        if (skill.skill.Priority > p) {
          skillImpl = skill.skill;
          p = skill.skill.Priority;
        }
      }
    }
    if (skillImpl) {
      return true;
    }

    if (
      role.fetterSkill &&
      role.fetterSkill.trigger.CheckSkillTrigger(evs,
roleInfo)
    ) {
      return true;
    }

    return false;
  }

  private checkTriggerSkill(evs: skill.Event[]) {
    let selfTeam = this.selfTeam.GetRoles();
```

```
    for (let index in selfTeam) {
      let role = selfTeam[index];
      if (!this.CheckRoleTriggerSkill(role, evs)) {
        continue;
      } else {
        return true;
      }
    }

    let enemyTeam = this.enemyTeam.GetRoles();
    for (let index in enemyTeam) {
      let role = enemyTeam[index];
      if (!this.CheckRoleTriggerSkill(role, evs)) {
        continue;
      } else {
        return true;
      }
    }

    return false;
  }

  private tickSkill(evs: skill.Event[]) {
    console.log("TickBattle evs:", evs);

    let selfTeam = this.selfTeam.GetRoles();
    for (let index in selfTeam) {
      let role = selfTeam[index];

      let roleInfo = new skill.RoleInfo();
      roleInfo.properties = role.GetProperties();
      console.log("战斗事件中角色攻击力:");

      roleInfo.index = role.index;
      roleInfo.camp = battleEnums.Camp.Self;
      let p = 0;
      let skillImpl: skill.SkillBase = null;
      let isPar = false;
      for (let skill of role.skill) {
        let flag = skill.trigger.CheckSkillTrigger(evs,
roleInfo);
        if (0 == flag) {
          flag = this.CheckRepeatSkill(evs, roleInfo);
        }
        if (flag) {
          if (2 == flag) {
            isPar = true;
            console.log("Parallel skill detected");
          }
          if (skill.skill.Priority > p) {
```

```
            skillImpl = skill.skill;
            p = skill.skill.Priority;
          }
        }
      }
      if (skillImpl) {
        console.log("Are skills parallel:" + isPar);
        skillImpl.UseSkill(roleInfo, this, isPar, evs);
      }

      if (
        role.fetterSkill &&
        role.fetterSkill.trigger.CheckSkillTrigger(evs,
roleInfo)
      ) {
        console.log("Are skills parallel:" + isPar);
        role.fetterSkill.skill.UseSkill(roleInfo, this, isPar,
evs);
      }
    }

    let enemyTeam = this.enemyTeam.GetRoles();
    for (let index in enemyTeam) {
      let role = enemyTeam[index];

      let roleInfo = new skill.RoleInfo();
      roleInfo.properties = role.GetProperties();

      roleInfo.index = role.index;
      roleInfo.camp = battleEnums.Camp.Enemy;
      let p = 0;
      let skillImpl: skill.SkillBase = null;
      let isPar = false;
      for (let skill of role.skill) {
        let flag = skill.trigger.CheckSkillTrigger(evs,
roleInfo);
        if (0 == flag) {
          flag = this.CheckRepeatSkill(evs, roleInfo);
        }
        if (flag) {
          if (2 == flag) isPar = true;
          if (skill.skill.Priority > p) {
            skillImpl = skill.skill;
            p = skill.skill.Priority;
          }
        }
      }
      if (skillImpl) {
        skillImpl.UseSkill(roleInfo, this, isPar, evs);
      }
```

```
      if (
        role.fetterSkill &&
        role.fetterSkill.trigger.CheckSkillTrigger(evs,
roleInfo)
      ) {
        console.log("Are skills parallel:" + isPar);
        role.fetterSkill.skill.UseSkill(roleInfo, this, isPar,
evs);
      }
    }
  }
```

```
  private CheckRepeatSkill(evs: skill.Event[], selfInfo:
skill.RoleInfo) {
    for (let ev of evs) {
      if (battleEnums.EventType.RepeatSkill == ev.type) {
        for (let roleinfo of ev.recipient) {
          if (
            roleinfo.index == selfInfo.index &&
            roleinfo.camp == selfInfo.camp
          ) {
            return 1;
          }
        }
      }
    }
    return 0;
  }
```

```
  private triggerBeforeAttack: boolean = true;
  public async TickBattle(): Promise<boolean> {
    let _evs = this.evs.slice();
    this.evs = [];
    let [injuredEvs, normalEvs] = splitEvs(_evs);
    await this.tickInjuredEvent(injuredEvs);
    await this.tickInjuredEventChain(normalEvs);

    if (this.CheckEndBattle()) {
      return false;
    }
    this.CheckRemoveDeadRole();

    let self = this.selfTeam.GetLasterRole();
    let enemy = this.enemyTeam.GetLasterRole();
    if (this.triggerBeforeAttack) {
      if (self && enemy) {
        let ev = new skill.Event();
        ev.type = battleEnums.EventType.BeforeAttack;
        ev.spellcaster = new skill.RoleInfo();
```

```
        ev.spellcaster.camp = self.selfCamp;
        ev.spellcaster.index = self.index;
        ev.recipient = [];
        this.AddBattleEvent(ev);

        ev = new skill.Event();
        ev.type = battleEnums.EventType.BeforeAttack;
        ev.spellcaster = new skill.RoleInfo();
        ev.spellcaster.camp = enemy.selfCamp;
        ev.spellcaster.index = enemy.index;
        ev.recipient = [];
        this.AddBattleEvent(ev);

        this.triggerBeforeAttack = false;

        return false;
      }
    }

    this.battle();
    this.triggerBeforeAttack = true;

    if (this.evs.length > 0) {
      let evs = this.evs.slice();
      this.evs = [];
      await this.tickBattleInjuredEvent(evs);
      return false;
    }

    return this.CheckEndBattle();
  }
}

export function GetSelfTeam() {
  throw new Error("Function not implemented.");
}

/*
 * enums.ts
 * author: qianqians
 * 2024/7/19
 */

export enum Property {
    HP = 1,
    TotalHP = 2,
    Attack = 3,
}

export enum EventType
```

```
{
    /** 出售 */
    Sold = 1,
    /** 购买 */
    Purchase = 2,
    /** 升级 */
    Upgrade = 3,
    /** 使用道具 */
    EatFood = 4,
    /** 回合开始 */
    RoundStarts = 5,
    /** 回合结束 */
    RoundEnd = 6,
    /** 战斗开始 */
    BattleBegin = 7,
    /** 攻击前 */
    BeforeAttack = 8,
    /** 攻击后 */
    AfterAttack = 9,
    /** 攻击受伤 */
    AttackInjured = 10,
    /** 远程受伤 */
    RemoteInjured = 11,
    /** 晕厥倒下 */
    Syncope = 12,
    /** 位置改变 */
    ChangeLocation = 13,
    /** 召唤 */
    Summon = 14,
    /** 战斗失败 */
    BattleFail=15,
    /** 使用技能*/
    UsedSkill=16,
    /** 交换属性 */
    SwapProperties = 17,
    /** 属性强化 */
    IntensifierProperties = 18,
    /** 退场 */
    Exit = 19,
    /** 队友受伤 */
    FriendlysideInjured=20,
    /** 击败 */
    Kill=21,
    /** 前方攻击 */
    FrontAtk=22,
    /** 经验提升 */
    IntensifierExp = 23,
    /** 我方前排为空 */
    SelfFrontNull = 24,
    /** 敌方前排为空 */
```

```
    EnemyFrontNull=25,
    /** 强制战斗 */
    ForcedAttack=26,
    /** 护盾 */
    GiveShields=27,
    /** 添加buff */
    AddBuff=28,
    /** 代替某角色承受伤害 */
    SubstituteDamage=29,
    /** 受伤前 */
    BeforeHurt=30,
    /** 重复使用一次技能 */
    RepeatSkill=31,
    /** 转移的伤害 */
    TransferInjured=31,
}

export enum ChangeLocationType {
    AssignChange = 1,
    RandomChange = 2,
    BackChange = 3,
}

export enum SwapPropertiesType {
    SelfSwap = 1,
    AssignSwap = 2,
    RandomSwap = 3,
    /** 把攻击交换到生命值 */
    AttackSwap=4,
    /** 把生命值交换到攻击 */
    HpSwap=5,
}

export enum SkillType
{
    Intensifier = 1,
    Attack = 2,
    Summon = 3,
    Support = 4,
    Economy = 5,
    SwapProperties=6,
}

export enum BufferType {
    InevitableKill = 1,
    ShareDamage = 2,
    OffsetDamage = 3,
    Weak = 4,
    ReductionDamage = 5,
    Strength = 6,
```

```
    Shields = 7,
}

export enum Camp {
    None = -1,
    Tie = 0,
    Self = 1,
    Enemy = 2,
}

{
  "ver": "1.2.0",
  "importer": "directory",
  "imported": true,
  "uuid": "b0499573-3481-4a6b-8330-787e25f68c82",
  "files": [],
  "subMetas": {},
  "userData": {
    "compressionType": {},
    "isRemoteBundle": {}
  }
}
/*
 * create_buffer.ts
 * author: qianqians
 * 2023/10/3
 */
import * as config from './config/config'
import * as enums from './BattleEnums'
import * as buffer from './buffer/buffer'

export function CreateBuff(bufferID:number, value?:number ,
round?:number ,frequency?:number) : buffer.Buffer
{
    let bufferConfig =
config.config.BufferConfig.get(bufferID);

    let bufferObj:buffer.Buffer = new buffer.Buffer();
    if(value)
    {
        bufferObj.Value = value;
    }
    else
    {
        bufferObj.Value = bufferConfig.Value;
    }

    if(round)
```

```
    {
        bufferObj.Round=round;
    }
    else
    {
        bufferObj.Round = bufferConfig.Round;
    }

    if(frequency)
    {
        bufferObj.Frequency = frequency;
    }
    else
    {
        bufferObj.Frequency=bufferConfig.Frequency;
    }
    bufferObj.BufferType = bufferConfig.Type;

    return bufferObj;
}
/*
 * create_equip_skill.ts
 * author: qianqians
 * 2025/2/26
 */
import * as config from './config/config'
import * as enums from './BattleEnums'
import * as common from '../../serverSDK/common'
import * as skill from './skill/skill_base'
import * as Skill_Summon from './skill/Skill_Summon'
import * as create_trigger from './create_trigger'
import * as create_buffer from './create_buffer'
import {SkillInfo, Role} from './role'
import { Skill_Avoid_Killed } from './equip/
Equip_Skill_Avoid_Killed'

export enum EquipEffectEM{
    AddHP = 1,                          //提高生命值
    AddAttack = 2,                      //提高攻击力
    DeDamage = 3,                       //抵抗伤害
    Summon = 7,                         //召唤
    AdditionalAttack = 8,               //额外伤害
    Resurrect = 9,                      //复活
    AddArmor = 10,                      //额外护甲
    Die = 11,                           //即死
    ImmuneFatalDamage = 12,             //免疫致命伤害
    FirstAttackDoubleDamage = 13,       //首次攻击双倍伤害
    PiercingDamage = 14,                //穿刺伤害
}
```

```
export function createEquipSkill(r:Role, equipID:number) {
    let equip = config.config.EquipConfig.get(equipID);
    if (!equip) {
        return null;
    }

    switch(equip.Effect) {
        case EquipEffectEM.Summon:
        {
            let skill = new SkillInfo();
            let p = new Map<enums.Property, number>();
            p.set(enums.Property.HP, equip.Vaule[1]);
            p.set(enums.Property.TotalHP, equip.Vaule[1]);
            p.set(enums.Property.Attack, equip.Vaule[2]);
            skill.trigger =
create_trigger.CreateTrigger(common.EMSkillEvent.syncope);
            skill.skill = new
Skill_Summon.Skill_Summon(common.Priority.Normal,
equip.Vaule[0], 1, p);
            r.skill.push(skill);
        }
        break;

        case EquipEffectEM.Resurrect:
        {
            let skill = new SkillInfo();
            let p = new Map<enums.Property, number>();
            p.set(enums.Property.HP, equip.Vaule[1]);
            p.set(enums.Property.TotalHP, equip.Vaule[1]);
            p.set(enums.Property.Attack, equip.Vaule[2]);
            skill.trigger =
create_trigger.CreateTrigger(common.EMSkillEvent.syncope);
            skill.skill = new
Skill_Summon.Skill_Summon(common.Priority.Normal, r.id, 1, p);
            r.skill.push(skill);
        }
        break;

        case EquipEffectEM.DeDamage:
        {
            let bufferConfig =
config.config.BufferConfig.get(enums.BufferType.ReductionDamag
e);
            if (bufferConfig) {
                let buf =
create_buffer.CreateBuff(bufferConfig.Id, equip.Vaule[0], 10);
                r.buffer.push(buf);
            }
        }
        break;
```

```
        case EquipEffectEM.AdditionalAttack:
        {
            let bufferConfig =
config.config.BufferConfig.get(enums.BufferType.Strength);
            if (bufferConfig) {
                let buf =
create_buffer.CreateBuff(bufferConfig.Id, equip.Vaule[0], 10);
                r.buffer.push(buf);
            }
        }
        break;

        case EquipEffectEM.AddArmor:
        {
            let bufferConfig =
config.config.BufferConfig.get(enums.BufferType.Shields);
            if (bufferConfig) {
                let buf =
create_buffer.CreateBuff(bufferConfig.Id, equip.Vaule[0]);
                r.buffer.push(buf);
            }
        }
        break;

        case EquipEffectEM.Die:
        {
            let bufferConfig =
config.config.BufferConfig.get(enums.BufferType.InevitableKill
);
            if (bufferConfig) {
                let buf =
create_buffer.CreateBuff(bufferConfig.Id, equip.Vaule[0], 10);
                r.buffer.push(buf);
            }
        }
        break;

        case EquipEffectEM.ImmuneFatalDamage:
        {
            let skill = new SkillInfo();
            skill.trigger =
create_trigger.CreateTrigger(common.EMSkillEvent.syncope);
            skill.skill = new
Skill_Avoid_Killed(common.Priority.Normal);
            r.skill.push(skill);
        }
        break;
    }
```

```
    return skill;
}
/*
 * create_fetters.ts
 * author: qianqians
 * 2023/12/15
 */
import * as config from './config/config'
import * as enums from './BattleEnums'
import * as common from '../../serverSDK/common'
import * as skill from './skill/skill_base'
import * as Skill_AttGain_1 from './skill/Skill_AttGain'
import * as Skill_RecoveryHP from './skill/Skill_RecoveryHP'
import * as Skill_RemoteAtk from './skill/Skill_RemoteAtk'
import * as Skill_Summon from './skill/Skill_Summon'
import * as Skill_SwapProperties_5 from './skill/
Skill_SwapProperties'
import * as Skill_Shields from './skill/Skill_Shields'
import * as Skill_ChangePosition_7 from './skill/
Skill_ChangeLocation'
import { Skill_AttackAll } from './skill/Skill_AttackAll'

export function CreateFetters(level:number,
fettersID:number) : skill.SkillBase {
    let fettersConfig =
config.config.FettersConfig.get(fettersID);

    let value0 = 0;
    let value1 = 0;
    if (1 == level) {
        value0 = fettersConfig.Stage1value_1;
        value1 = fettersConfig.Stage1value_1;
    }
    else if (2 == level) {
        value0 = fettersConfig.Stage2value_1;
        value1 = fettersConfig.Stage2value_1;
    }
    else if (3 == level) {
        value0 = fettersConfig.Stage3value_1;
        value1 = fettersConfig.Stage3value_1;
    }
    else if (4 == level) {
        value0 = fettersConfig.Stage4value_1;
        value1 = fettersConfig.Stage4value_1;
    }

    let skillObj:skill.SkillBase = null;
    switch(fettersConfig.Effect) {
        case common.SkillEffectEM.AddProperty:
        {
```

```
            console.log("Skill_AttGain_1 fettersConfig:",
fettersConfig);
            let count_index = fettersConfig.ObjCount.length <
level ? fettersConfig.ObjCount.length  - 1 : level - 1;
            let count = fettersConfig.ObjCount[count_index];
            skillObj = new
Skill_AttGain_1.Skill_AttGain(fettersConfig.Priority, value0,
value1,null,count,fettersConfig.EffectScope,fettersConfig.Fett
erAudio,true);
        }
        break;
        case common.SkillEffectEM.RecoverHP:
        {
            let count_index = fettersConfig.ObjCount.length <
level ? fettersConfig.ObjCount.length  - 1 : level - 1;
            let count = fettersConfig.ObjCount[count_index];
            skillObj = new
Skill_RecoveryHP.Skill_RecoveryHP(fettersConfig.Priority,
count, value0,fettersConfig.FetterAudio,true);
        }
        break;
        case common.SkillEffectEM.RemoteAttack:
        {
            let count_index = fettersConfig.ObjCount.length <
level ? fettersConfig.ObjCount.length  - 1 : level - 1;
            let count = fettersConfig.ObjCount[count_index];
            if(value0 >= 1)
            {
                skillObj = new
Skill_RemoteAtk.Skill_RemoteAtk(fettersConfig.Priority, count,
Math.floor(value0),fettersConfig.FetterAudio,true);
            }
            else
            {
                skillObj=new
Skill_RemoteAtk.Skill_RemoteAtkPre(fettersConfig.Priority,
count, value0,null,fettersConfig.FetterAudio,true);
            }
        }
        break;
        case common.SkillEffectEM.SummonBattle:
        {
            if (fettersConfig.SummonLevel == 0) {
                let p = new Map<enums.Property, number>();
                p.set(enums.Property.HP, value0);
                p.set(enums.Property.TotalHP, value0);
                p.set(enums.Property.Attack, value1);
                skillObj = new
Skill_Summon.Skill_Summon(fettersConfig.Priority,
fettersConfig.SummonId, 0, p,fettersConfig.FetterAudio,true);
```

```
            }
            else {
                skillObj = new
Skill_Summon.Skill_Summon(fettersConfig.Priority,
fettersConfig.SummonId,
fettersConfig.SummonLevel,null,fettersConfig.FetterAudio,true)
;
            }
        }
        break;
        case common.SkillEffectEM.GainShield:
        {
            skillObj = new
Skill_Shields.Skill_Shields(fettersConfig.Priority, value0,
value1,common.Direction.None,fettersConfig.FetterAudio,true);
        }
        break;
        case common.SkillEffectEM.AttackAll:
        {
            skillObj=new
Skill_AttackAll(fettersConfig.Priority,value0,fettersConfig.Fe
tterAudio,true);
        }
        break;
    }

    return skillObj;
}

export function CreateMechanicFettersSummon(level:number,
buildValue:number) : skill.SkillBase {
    // to do ...
    let t = config.config.FettersConfig.get(6);
    let skillObj:skill.SkillBase = null;
    skillObj=new
Skill_Summon.Skill_SummonMecha(null,null,level,buildValue,null
,t.FetterAudio,true);
    return null;
}
/*
 * create_skill.ts
 * author: qianqians
 * 2023/10/3
 */
import * as config from './config/config'
import * as enums from './BattleEnums'
import * as common from '../../serverSDK/common'
import * as skill from './skill/skill_base'
import * as Skill_AddTmpExp from './skill/Skill_AddTmpExp'
import * as Skill_AttackAll from './skill/Skill_AttackAll'
```

```
import * as Skill_AttGain from './skill/Skill_AttGain'
import * as Skill_ChangePosition from './skill/
Skill_ChangeLocation'
import * as Skill_Counterattack from './skill/
Skill_Counterattack'
import * as Skill_DeAttGain from './skill/Skill_DeAttGain'
import * as Skill_ForcedAttack from './skill/
Skill_ForcedAttack'
import * as Skill_RecoveryHP from './skill/Skill_RecoveryHP'
import * as Skill_RemoteAtk from './skill/Skill_RemoteAtk'
import * as Skill_Shields from './skill/Skill_Shields'
import * as Skill_SubstituteDamage from './skill/
Skill_SubstituteDamage'
import * as Skill_Summon from './skill/Skill_Summon'
import * as Skill_SwapProperties from './skill/
Skill_SwapProperties'
import * as Skill_AddBuff from './skill/Skill_AddBuff'
import * as Skill_AddAttackCoin from './skill/
Skill_AddAttackCoin'

export function CreateSkill(level:number, skillID:number) :
skill.SkillBase {
    let skillConfig = config.config.SkillConfig.get(skillID);
    let
soundConfig=config.config.SkillSoundConfig.get(skillID);
    if("null"===soundConfig.Path)
    {
        soundConfig.Path=null;
    }

    let value0 = 0;
    let value1 = 0;
    if (1 == level) {
        value0 = skillConfig.Level1Value_1;
        value1 = skillConfig.Level1Value_2;
    }
    else if (2 == level) {
        value0 = skillConfig.Level2Value_1;
        value1 = skillConfig.Level2Value_2;
    }
    else if (3 == level) {
        value0 = skillConfig.Level3Value_1;
        value1 = skillConfig.Level3Value_2;
    }

    let ObjCount=skillConfig.ObjCount;
    if(54==skillID)
    {
        ObjCount=level;
    }
```

```
    console.log("skillID:", skillID, " level:", level, "
value0:", value0, " value1", value1);

    let skillObj:skill.SkillBase = null;
    switch (skillConfig.Effect)
    {
        case common.SkillEffectEM.AddTmpExp:
            {
                skillObj = new
Skill_AddTmpExp.Skill_AddTmpExp(skillConfig.Priority, value0,
skillConfig.ObjectDirection, skillConfig.ObjCount,
soundConfig.Path);
            }
            break;
        case common.SkillEffectEM.AttackAll:
            {
                skillObj = new
Skill_AttackAll.Skill_AttackAll(skillConfig.Priority, value0,
soundConfig.Path);
            }
            break;
        case common.SkillEffectEM.CounterAttack:
            {
                skillObj = new
Skill_Counterattack.Skill_Counterattack(skillConfig.Priority,
skillConfig.ObjCount, value0, skillConfig.ObjCount == 6,
soundConfig.Path);
            }
            break;
        case common.SkillEffectEM.DeAttGain:
            {
                skillObj = new
Skill_DeAttGain.Skill_DeAttGain(skillConfig.Priority, value0,
value1, skillConfig.ObjCount == 6, skillConfig.ObjCount,
soundConfig.Path);
            }
            break;
        case common.SkillEffectEM.ForcedAttack:
            {
                skillObj = new
Skill_ForcedAttack.Skill_ForcedAttack(skillConfig.Priority,
soundConfig.Path);
            }
            break;
        case common.SkillEffectEM.AddProperty:
            {
                console.log("Skill_AttGain_1 skillConfig:",
skillConfig);
                if (skillConfig.ObjectDirection !=
common.Direction.None)
```

```
                {
                    skillObj = new
Skill_AttGain.Skill_AttGain(skillConfig.Priority, value0,
value1,
skillConfig.ObjectDirection,null,null,soundConfig.Path);
                }
                else
                {
                    skillObj = new
Skill_AttGain.Skill_AttGain(skillConfig.Priority, value0,
value1, null, skillConfig.ObjCount,null,soundConfig.Path);
                }
            }
            break;
        case common.SkillEffectEM.RecoverHP:
            {
                skillObj = new
Skill_RecoveryHP.Skill_RecoveryHP(skillConfig.Priority,
skillConfig.ObjCount, value0,soundConfig.Path);
            }
            break;
        case common.SkillEffectEM.RemoteAttack:
            {
                if (value0 >= 1)
                {
                    skillObj = new
Skill_RemoteAtk.Skill_RemoteAtk(skillConfig.Priority,
skillConfig.ObjCount, Math.floor(value0),soundConfig.Path);
                }
                else
                {
                    skillObj = new
Skill_RemoteAtk.Skill_RemoteAtkPre(skillConfig.Priority,
skillConfig.ObjCount, value0,null,soundConfig.Path);
                }
            }
            break;
        case common.SkillEffectEM.SummonBattle:
            {
                // if (skillConfig.SummonLevel == 0) {
                //     let p = new Map<enums.Property,
number>();
                //     p.set(enums.Property.HP, value0);
                //     p.set(enums.Property.TotalHP, value0);
                //     p.set(enums.Property.Attack, value1);
                //     skillObj = new
Skill_Summon_4.Skill_Summon_4(skillConfig.Priority,
skillConfig.SummonId[0], 0, p);
                // }
                // else {
```

```
                //     skillObj = new
Skill_Summon_4.Skill_Summon_4(skillConfig.Priority,
skillConfig.SummonId[0], skillConfig.SummonLevel);
                // }
                let p = new Map<enums.Property, number>();
                p.set(enums.Property.HP, value0);
                p.set(enums.Property.TotalHP, value0);
                p.set(enums.Property.Attack, value1);

                skillObj = new
Skill_Summon.Skill_Summon(skillConfig.Priority,
skillConfig.SummonId[0], skillConfig.SummonLevel,
p,soundConfig.Path);
            }
            break;
        case common.SkillEffectEM.ExchangeProperty:
            {
                switch (skillConfig.SwapPropertiesType)
                {
                    case enums.SwapPropertiesType.AssignSwap:
                    case enums.SwapPropertiesType.RandomSwap:
                    case enums.SwapPropertiesType.SelfSwap:
                        {
                            skillObj = new
Skill_SwapProperties.Skill_SwapProperties(skillConfig.Priority
, skillConfig.SwapPropertiesType ,0 , value0,
value1,soundConfig.Path);
                        }
                        break;
                    case enums.SwapPropertiesType.HpSwap:
                    case enums.SwapPropertiesType.AttackSwap:
                        {
                            skillObj = new
Skill_SwapProperties.Skill_SwapPropertiesSingle(skillConfig.Pr
iority, skillConfig.SwapPropertiesType,
value0,soundConfig.Path);
                        }
                        break;
                    default:
                        {
                            console.warn("类型
SkillEffectEM.ExchangeProperty下没有相关技能");
                        }
                        break;
                }
            }
            break;
        case common.SkillEffectEM.GainShield:
            {
                skillObj = new
```

```
Skill_Shields.Skill_Shields(skillConfig.Priority,
skillConfig.ObjCount ,value0 ,
skillConfig.ObjectDirection,soundConfig.Path);
            }
            break;
        case common.SkillEffectEM.ChangePosition:
            {
                skillObj = new
Skill_ChangePosition.Skill_ChangeLocation(skillConfig.Priority
, skillConfig.ChangeLocationType, value0,
value1,soundConfig.Path);
            }
            break;
        case common.SkillEffectEM.ReductionHurt:
            {
                skillObj = new
Skill_SubstituteDamage.Skill_SubstituteDamage(skillConfig.Prio
rity, skillConfig.ChangeLocationType, value0,
value1,soundConfig.Path);
            }
            break;
        case common.SkillEffectEM.AddBuffer:
            {
                skillObj = new
Skill_AddBuff.Skill_AddBuff(skillConfig.Priority ,
skillConfig.AddBufferID, value0 , value1 ,
ObjCount,soundConfig.Path);
            }
            break;
        case common.SkillEffectEM.AttackCoin:
            {
                skillObj = new
Skill_AddAttackCoin.Skill_AddAttackCoin(skillConfig.Priority,s
oundConfig.Path);
            }
            break;
    }

    return skillObj;
}

/*
 * create_trigger.ts
 * author: qianqians
 * 2023/10/3
 */
import * as skill from './skill/skill_base'
import * as common from '../../serverSDK/common'
import * as SkillTrigger_RoundStarts from './trigger/
SkillTrigger_RoundStarts'
```

```
import * as SkillTrigger_RoundEnd from './trigger/
SkillTrigger_RoundEnd'
import * as SkillTrigger_BattleBegin from './trigger/
SkillTrigger_BattleBegin'
import * as SkillTrigger_BeforeAtk from './trigger/
SkillTrigger_BeforeAtk'
import * as SkillTrigger_Injured from './trigger/
SkillTrigger_Injured'
import * as SkillTrigger_Syncope from './trigger/
SkillTrigger_Syncope'
import * as SkillTrigger_ChangeSelfLocation from './trigger/
SkillTrigger_ChangeSelfLocation'
import * as SkillTrigger_ChangeEnemyLocation from './trigger/
SkillTrigger_ChangeEnemyLocation'
import * as SkillTrigger_UsedSKill from './trigger/
SkillTrigger_UsedSKill'
import { SkillTrigger_AllMechaniSyncope } from './trigger/
SkillTrigger_AllMechaniSyncope'
import { SkillTrigger_FriendlysideInjured } from './trigger/
SkillTrigger_FriendlysideInjured'
import { SkillTrigger_Kill } from './trigger/
SkillTrigger_Kill'
import { SkillTrigger_CampSyncope } from './trigger/
SkillTrigger_CampSyncope'
import { SkillTrigger_FrontAtk } from './trigger/
SkillTrigger_FrontAtk'
import * as  SkillTrigger_AfterAtk from './trigger/
SkillTrigger_AfterAtk'
import * as SkillTrigger_EnemySummon from './trigger/
SkillTrigger_EnemySummon'
import * as SkillTrigger_SelfFrontNull from './trigger/
SkillTrigger_SelfFrontNull';
import * as SkillTirgger_EnemyFrontNulll from './trigger/
SkillTirgger_EnemyFrontNull';
import { SkillTirgger_Upgrade } from './trigger/
SkillTirgger_Upgrade'

export function CreateTrigger(triggerID:number) :
skill.SkillTriggerBase {
    let triggerObj: skill.SkillTriggerBase = null;

    switch (triggerID)
    {
        case common.EMSkillEvent.start_round:
            {
                triggerObj = new
SkillTrigger_RoundStarts.SkillTrigger_RoundStarts();//回合开始
            }
            break;
        case common.EMSkillEvent.end_round:
```

```
            {
                triggerObj = new
SkillTrigger_RoundEnd.SkillTrigger_RoundEnd();//回合结束
            }
            break;
        case common.EMSkillEvent.start_battle:
            {
                triggerObj = new
SkillTrigger_BattleBegin.SkillTrigger_BattleBegin();//战斗开始
            }
            break;
        case common.EMSkillEvent.before_attack:
            {
                triggerObj = new
SkillTrigger_BeforeAtk.SkillTrigger_BeforeAtk();//攻击前
            }
            break;
        case common.EMSkillEvent.after_attack:
            {
                triggerObj = new
SkillTrigger_AfterAtk.SkillTrigger_AfterAtk()//攻击后（或友方攻击
后）
            }
        case common.EMSkillEvent.be_hurt:
            {
                triggerObj = new
SkillTrigger_Injured.SkillTrigger_Injured();//受击
            }
            break;
        case common.EMSkillEvent.syncope:
            {
                triggerObj = new
SkillTrigger_Syncope.SkillTrigger_Syncope();//角色倒下
            }
            break;
        case common.EMSkillEvent.camp_summon:
            {

            }
            break;
        case common.EMSkillEvent.enemy_summon:
            {
                triggerObj = new
SkillTrigger_EnemySummon.SkillTrigger_EnemySummon();//敌方召唤
            }
            break;
        case common.EMSkillEvent.change_self_location:
            {
                triggerObj = new
SkillTrigger_ChangeSelfLocation.SkillTrigger_ChangeSelfLocatio
```

```
n();//我方被推动
                }
                break;
            case common.EMSkillEvent.change_enemy_location:
                {
                    triggerObj = new
SkillTrigger_ChangeEnemyLocation.SkillTrigger_ChangeEnemyLocat
ion();//敌方被推动
                }
                break;
            case common.EMSkillEvent.use_skill:
                {
                    triggerObj = new
SkillTrigger_UsedSKill.SkillTrigger_UsedSKill();//使用技能
                }
                break;
            case common.EMSkillEvent.front_be_hurt:
                {
                    triggerObj = new
SkillTrigger_FriendlysideInjured(common.Direction.Forward);//
前方受击
                }
                break;
            case common.EMSkillEvent.kill:
                {
                    triggerObj = new SkillTrigger_Kill();//击败敌人
                }
                break;
            case common.EMSkillEvent.camp_syncope:
                {
                    triggerObj = new SkillTrigger_CampSyncope();//
队友倒下
                }
                break;
            case common.EMSkillEvent.front_attack:
                {
                    triggerObj = new SkillTrigger_FrontAtk();//前方
攻击
                }
                break;
            case common.EMSkillEvent.all_mechanic_syncope:
                {
                    //还需要传入一个battle
                    triggerObj = new
SkillTrigger_AllMechaniSyncope();
                }
                break;
            case common.EMSkillEvent.self_front_null://我方前排为空
                {
                    triggerObj = new
```

```
SkillTrigger_SelfFrontNull.SkillTrigger_SelfFrontNull();
            }
            break;
        case common.EMSkillEvent.enemy_front_null://敌方前排为空
            {
                triggerObj = new
SkillTirgger_EnemyFrontNulll.SkillTirgger_EnemyFrontNull();
            }
            break;
        case common.EMSkillEvent.camp_attack3:
            {
                triggerObj = new
SkillTrigger_AfterAtk.SkillTrigger_AfterAtk(3);
            }
            break;
        case common.EMSkillEvent.upgrade://升级时
            {
                triggerObj=new SkillTirgger_Upgrade();
            }
    }

    return triggerObj;
}
/*
 * role.ts
 * author: qianqians
 * 2023/9/24
 */
import * as skill from './skill/skill_base'
import * as buffer from './buffer/buffer'
import * as battle from './battle'
import * as enums from './BattleEnums'
import * as common from '../../serverSDK/common'
import * as create_skill from './create_skill'
import * as create_fetters from './create_fetters'
import * as create_trigger from './create_trigger'
import * as create_equip_skill from './create_equip_skill'
import * as create_buffer from './create_buffer'
import * as config from './config/config'

export class SkillInfo {
    public trigger : skill.SkillTriggerBase;
    public skill : skill.SkillBase;
}

function createSkill(id:number, level:number) : SkillInfo {
    let skillConfig = config.config.SkillConfig.get(id);
    if (skillConfig) {
        let skill = new SkillInfo();
        skill.trigger =
```

```
create_trigger.CreateTrigger(skillConfig.EffectTime);
        skill.skill = create_skill.CreateSkill(level, id);

        if (skill.trigger && skill.skill) {
            return skill;
        }
    }
    return null;
}

function createFettersSkill(id:number, level:number) :
SkillInfo {
    let fettersConfig = config.config.FettersConfig.get(id);
    if (fettersConfig) {
        let skill = new SkillInfo();
        skill.trigger =
create_trigger.CreateTrigger(fettersConfig.EffectTime);
        skill.skill = create_fetters.CreateFetters(level, id);

        if (skill.trigger && skill.skill) {
            return skill;
        }
    }
    return null;
}

function createEquipSkill(r:Role, equipId:number) {
    create_equip_skill.createEquipSkill(r, equipId);
}

function createBuffer(id:number , value?:number ,
round?:number , frequency?:number) : buffer.Buffer {
    let bufferConfig = config.config.BufferConfig.get(id);
    if (bufferConfig) {
        return
create_buffer.CreateBuff(id,value,round,frequency);
    }
    return null;
}

export class Role {
    public c_role:common.Role;
    public index:number;
    public id:number;
    public skillid:number;
    public level:number;
    public exp:number;

    public foeRoleIndex:number;
    public isForcedAttack=false;
```

```
    public skill : SkillInfo[] = []; // 一般情况只有一个技能，使用特殊食物时添加一个技能
    public fetterSkill : SkillInfo = null;
    public fetter:common.Fetters;
    public buffer : buffer.Buffer[] = [];
    public equip:number=null;//装备id(一般只能装备一件装备)

    public properties : Map<enums.Property, number> = new Map<enums.Property, number>();
    private tempProperties:Map<enums.Property, number> = new Map<enums.Property, number>();
    public selfCamp: enums.Camp;

    private attackCnt;

    public constructor(c_role:common.Role,index:number, id:number, level:number, exp:number, selfCamp: enums.Camp, properties: Map<enums.Property, number>, fetters:common.Fetters, equipID:number, additionBuffer?:number[],additionSkill?:number[])
    {
        if(c_role)
        {
            this.c_role=c_role;
        }
        this.index = index;
        this.id=id;
        this.level=level;
        this.exp=exp;
        this.selfCamp = selfCamp;
        this.fetter=fetters;
        this.attackCnt=0;
        this.equip = equipID;

        properties.forEach((v, k) => {
            this.properties.set(k, v);
        })

        let roleConfig = config.config.RoleConfig.get(this.id);
        this.skillid = roleConfig.SkillID;
        let skill = createSkill(roleConfig.SkillID, this.level);
        if (skill) {
            this.skill.push(skill);
        }
        else {
            let buffer = createBuffer(roleConfig.SkillID);
            if (buffer) {
```

```
                this.buffer.push(buffer);
            }
        }

        if (this.equip) {
            createEquipSkill(this, this.equip);
        }

        if(additionSkill)
        {
            let additionSkills:SkillInfo[] = [];
            if(additionSkill&&additionSkill.length>0){
                for(let t of additionSkill){
                    additionSkills.push(createSkill(t,this.level));
                }
            }
            if (additionSkills.length>0) {
                for(let t of additionSkills)
                this.skill.push(t);
            }

            if (fetters && fetters.fetters_level > 0) {
                if (fetters.fetters_id == config.config.MechanicFetters) {
                    let skill = new SkillInfo();
                    skill.trigger = create_trigger.CreateTrigger(common.EMSkillEvent.all_mechanic_syncope);
                    skill.skill = create_fetters.CreateMechanicFettersSummon(fetters.fetters_level, id);

                    if (skill.trigger && skill.skill) {
                        this.fetterSkill = skill;
                    }
                }
                else {
                    let skill = createFettersSkill(fetters.fetters_id, fetters.fetters_level);
                    if (skill) {
                        this.fetterSkill = skill;
                    }
                }
            }
        }

        if (additionBuffer) {
            for (let id of additionBuffer) {
                let buffer = createBuffer(id);
```

```
                if (buffer) {
                    this.buffer.push(buffer);
                }
            }
        }
    }

    public onKillRole : (r:common.Role) => void;
    private sendHurtedEvent(enemy: Role, damage: number,
battle: battle.Battle, Injured: enums.EventType =
enums.EventType.AttackInjured,isParallel:boolean=false)
    {
        let selfIndex = this.index;
        let enemyIndex = enemy.index;

        if(Injured)
        {
            let ev = new skill.Event();
            ev.type = Injured;
            ev.spellcaster = new skill.RoleInfo();
            ev.spellcaster.camp = enemy.selfCamp;
            ev.spellcaster.index = enemyIndex;
            ev.recipient = [];
            let recipient = new skill.RoleInfo();
            recipient.camp = this.selfCamp;
            recipient.index = selfIndex;
            ev.recipient.push(recipient);
            ev.value = [];
            ev.value.push(damage);
            ev.priority=0;
            ev.isParallel=isParallel;
            ev.roleParallel = enemy;
            battle.AddBattleEvent(ev);
        }

        if (this.CheckDead())
        {
            if(this.isForcedAttack)
            {
                if(this.selfCamp == enums.Camp.Self)
                {
                    battle.GetSelfTeam().ReSetAttackRole();
                }
                if(this.selfCamp==enums.Camp.Enemy)
                {
                    battle.GetEnemyTeam().ReSetAttackRole();
                }
            }
            this.foeRoleIndex=enemy.selfCamp!=this.selfCamp?
enemy.index:null;
```

```
            let ev = new skill.Event();
            ev.type = enums.EventType.Syncope;
            ev.spellcaster = new skill.RoleInfo();
            ev.spellcaster.camp = this.selfCamp;
            ev.spellcaster.index = selfIndex;
            let role = new skill.RoleInfo();
            role.camp=enemy.selfCamp;
            role.index=enemy.index;
            ev.recipient = [role];
            ev.value = [];
            ev.value.push(damage);
            battle.AddBattleEvent(ev);
            //console.log("CheckDead sendHurtedEvent:",
JSON.stringify(battle.evs));
            //把单次对局加上的临时属性去掉
            this.tempProperties.forEach((value,key)=>{
                if(this.properties.has(key)){

this.properties.set(key,this.properties.get(key)+value);
                }
            })

            if (this.onKillRole) {
                this.onKillRole.call(null, enemy.c_role);
            }
        }
    }

    private SendBeforeHurtEvent(enemy: Role, battle:
battle.Battle,isParallel:boolean=false)
    {
        let ev = new skill.Event();

        let selfIndex = this.index;
        let enemyIndex = enemy.index;

        ev.type=enums.EventType.BeforeHurt;
        ev.spellcaster = new skill.RoleInfo();
        ev.spellcaster.camp = enemy.selfCamp;
        ev.spellcaster.index = enemyIndex;
        ev.recipient = [];
        let recipient = new skill.RoleInfo();
        recipient.camp = this.selfCamp;
        recipient.index = selfIndex;
        ev.recipient.push(recipient);
        ev.value = [];
        ev.priority=0;
        ev.isParallel=isParallel;
        ev.roleParallel = enemy;
        battle.AddBattleEvent(ev);
```

```
    }

    public SendExitEvent(battle: battle.Battle) {
        let ev = new skill.Event();
        ev.type = enums.EventType.Exit;
        ev.spellcaster = new skill.RoleInfo();
        ev.spellcaster.camp = this.selfCamp;
        ev.spellcaster.index = this.index;
        ev.recipient = [];
        ev.value = [];
        ev.priority=3;
        battle.AddBattleEvent(ev);
    }

    private checkShareDamageBuffer() : boolean {
        for (let b of this.buffer) {
            if (enums.BufferType.ShareDamage == b.BufferType
&& b.Round > 0) {
                --b.Round;
                return true;
            }
        }
        return false;
    }

    private checkInevitableKill() : boolean {
        for (let b of this.buffer) {
            if (enums.BufferType.InevitableKill ==
b.BufferType && b.Round > 0) {
                --b.Round;
                return true;
            }
        }
        return false;
    }

    private checkOffsetDamage() : number {
        for (let b of this.buffer) {
            if (enums.BufferType.OffsetDamage == b.BufferType
&& b.Round > 0) {
                --b.Round;
                return b.Value;
            }
        }
        return -1;
    }

    private checkReductionAtk() : number {
        for (let b of this.buffer) {
            if (enums.BufferType.Weak == b.BufferType &&
```

```
b.Round > 0) {
                --b.Round;
                return b.Value;
            }
        }
        return -1;
    }

    private checkShields():boolean
    {
        for(let b of this.buffer)
        {
            if(enums.BufferType.Shields==b.BufferType&&
b.Value>0)
            {
                --b.Round;
                return true;
            }
        }
        return false;
    }

    // 平摊伤害
    // Role[] 平摊伤害的所有角色
    private getShareDamageArray(battle: battle.Battle) :
Role[] {
        if (!this.checkShareDamageBuffer()) {
            return [this];
        }

        let list = [];
        let selfTeam = this.selfCamp == enums.Camp.Self ?
battle.GetSelfTeam().GetRoles() :
battle.GetEnemyTeam().GetRoles();
        for (let r of selfTeam) {
            if (this == r || (!r.CheckDead() &&
r.checkShareDamageBuffer())) {
                list.push(r);
            }
        }

        return list;
    }

    private getBackRole(battle: battle.Battle) : Role {
        let target_index = -1;
        if (this.index == 0)
        {
            target_index = 3;
        }
```

```
        else if (this.index == 1)
        {
            target_index = 4;
        }
        else if (this.index == 2)
        {
            target_index = 5;
        }

        if (target_index == -1) {
            return null;
        }

        let selfTeam = this.selfCamp == enums.Camp.Self ?
battle.GetSelfTeam() : battle.GetEnemyTeam();
        return selfTeam.GetRole(target_index);
    }

    // 代替当前被攻击角色受伤害
    // Role 代替受伤的角色
    // number 代替受伤的伤害上限
    private getSubstituteDamage(battle: battle.Battle) :
[Role,number]
    {
        let selfTeam = this.selfCamp == enums.Camp.Self ?
battle.GetSelfTeam() : battle.GetEnemyTeam();
        let selfIndex = selfTeam.GetRoleIndex(this);

        for (let index in selfTeam.GetRoles()) {
            let i = parseInt(index);
            let r = selfTeam.GetRole(i);

            if (!r) {
                continue;
            }

            let bufferNumber:number;
            bufferNumber = r.checkOffsetDamage();
            if ((i - selfIndex) == 1 && -1!=bufferNumber) {
                return [r,bufferNumber];
            }

            bufferNumber=r.checkReductionAtk();
            if (-1!=bufferNumber) {
                return [r,bufferNumber];
            }
        }

        return null;
    }
```

```
    private getReductionDamage() : number {
        for (let b of this.buffer) {
            if (enums.BufferType.ReductionDamage ==
b.BufferType && b.Round > 0) {
                return b.Value;
            }
        }
        return 0;
    }

    private getintensifierAtk():number{

        for(let b of this.buffer)
        {
            if(enums.BufferType.Strength == b.BufferType &&
b.Round > 0)
            {
                return b.Value;
            }
        }

        return 0;
    }

    private getEquipAttack():number {
        if (this.equip) {
            let equip =
config.config.EquipConfig.get(this.equip);
            if (equip) {
               if (equip.Effect ==
create_equip_skill.EquipEffectEM.FirstAttackDoubleDamage) {
                    return
this.GetProperty(enums.Property.Attack);
                }
            }
        }
        return 0;
    }

    public getShields(){
        for(let b of this.buffer)
        {
            if(enums.BufferType.Shields==b.BufferType&&
this.checkShields())
            {
                return b;
            }
        }
        return null;
```

```
    }

    public BeInevitableKill(enemy: Role, battle:
battle.Battle) {
        let damageSelf =
this.properties.get(enums.Property.HP);
        this.properties.set(enums.Property.HP, 0);
        this.sendHurtedEvent(enemy, damageSelf, battle,
enums.EventType.AttackInjured);
    }

    public
ChangeProperties(type:enums.Property,value:number,isTemp?:bool
ean) {
        //value = value > 50 ? 50 : value;
        if(isTemp){
            if(this.tempProperties.has(type)){
                this.tempProperties.set(type,
                    this.tempProperties.get(type)+
(this.properties.get(type)-value));
            }
            else{
                this.tempProperties.set(type,
this.properties.get(type)-value);
            }
        }
        this.properties.set(type, value);
    }

    public GetProperty(em: enums.Property) {
        if (this.properties.has(em)) {
            return this.properties.get(em);
        }
        return 0;
    }
    /**
     * 增加经验
     * @param _num 数值
     *
     * author:Hotaru
     * 2024/09/09
     */
    public AddExp(_num:number)
    {
        this.exp+=_num;

        this.ChangeProperties(enums.Property.HP ,
this.GetProperty(enums.Property.HP)+1);
        this.ChangeProperties(enums.Property.Attack ,
this.GetProperty(enums.Property.Attack)+1);
```

```
        let _level=1+Math.floor(this.exp/3);
        if(this.level < _level && _level <=3)
        {
            this.level=_level;
        }
        else
        {
            this.level=3;
        }
    }
    /*
     * 添加
     * 因为存在交换属性的技能，所以添加一个函数返回某个角色的所有属性Map
的副本
     * Editor: Guanliu
     * 2023/9/30
     */
    public GetProperties():Map<enums.Property, number>{
        let t = new Map<enums.Property, number>();
        this.properties.forEach((v, k) => {
            t.set(k, v);
        })
        return t;
    }

    public CheckDead()
    {
        let hp = this.properties.get(enums.Property.HP);
        //console.log(`CheckDead selfCamp:${this.selfCamp} id:
${this.id} hp:${hp}`);
        return hp <= 0;
    }

    /**
     * 查找受击替身
     * @param  battle 战斗模块
     * @returns 受击替身
     */
    private CheckScapegoat(_battle:battle.Battle):[Role ,
number]
    {
        for(let ev of _battle.evs)
        {
            if(enums.EventType.SubstituteDamage == ev.type)
            {
                let selfTeam = this.selfCamp ==
enums.Camp.Self ? _battle.GetSelfTeam() :
_battle.GetEnemyTeam();
                let role =
```

```
selfTeam.GetRole(ev.spellcaster.index);
                return [role , ev.value[0]];
            }
        }
        return null;
    }
```

```
    public Attack(enemy: Role, battle: battle.Battle) {
        if (enemy.checkInevitableKill()) {
            //console.log("role checkInevitableKill!");
            this.BeInevitableKill(enemy, battle);
        }

        let list = this.getShareDamageArray(battle);
        let damage = enemy.GetProperty(enums.Property.Attack)
+ enemy.getEquipAttack() + enemy.getintensifierAtk() /
list.length;
        console.log("role Attack list.length:", list.length +
" camp:", this.selfCamp);
        if(1 == list.length)
        {
            this.BeHurted(damage,enemy, battle,
enums.EventType.AttackInjured);
        }
        else
        {
            for(let r of list)
            {
                if(this == r)
                {
                    r.BeHurted(damage,enemy, battle,
enums.EventType.AttackInjured);
                }
                else
                {
                    r.BeHurted(damage,this, battle,
enums.EventType.RemoteInjured,true);
                }
            }
        }

        if (enemy.equip) {
            let equip =
config.config.EquipConfig.get(enemy.equip);
            if (equip) {
               if (equip.Effect ==
create_equip_skill.EquipEffectEM.PiercingDamage) {
                    let r = this.getBackRole(battle);
                    if (r) {
                        r.BeHurted(equip.Vaule[0], enemy,
```

```
battle, enums.EventType.AttackInjured);
                    }
                }
            }
        }

        this.attackCnt++;
    }

    public BeHurted(damage:number, enemy: Role, battle:
battle.Battle, Injured:
enums.EventType,isParallel:boolean=false)
    {
        this.SendBeforeHurtEvent(enemy,battle,isParallel);

        let hp = this.GetProperty(enums.Property.HP);
        let reduction = this.getReductionDamage();
        let Shields=this.getShields();

        damage -= reduction;
        damage = damage < 0 ? 0 : damage;


        if(null != Shields)
        {
            if(Shields.Value >= damage)
            {
                Shields.Value -= damage;
                damage = 0;
            }
            else
            {
                damage -= Shields.Value;
                Shields.Value = 0;
            }
        }
        hp -= damage;
        let scapegoat= this.CheckScapegoat(battle);
        if(scapegoat!=null)
        {
            scapegoat[0].ChangeProperties(enums.Property.HP,hp
+ scapegoat[1]);
            scapegoat[0].sendHurtedEvent(enemy , damage ,
battle , enums.EventType.TransferInjured , isParallel);
            damage=0;
        }
        else
        {
            this.ChangeProperties(enums.Property.HP, hp);
        }
```

```
        if(Injured!=enums.EventType.RemoteInjured)
        {
            this.sendHurtedEvent(enemy, damage, battle, 
Injured ,isParallel);
        }
    }

    public AddBuff(_id:number , _value:number=0 , 
_round:number=1 , _frequency:number=0)
    {
        let buffer = createBuffer(_id , _value , _round , 
_frequency);
        if (buffer)
        {
            this.buffer.push(buffer);
        }
    }
}
/*
 * team.ts
 * author: qianqians
 * 2023/9/24
 */
import * as common from "../../serverSDK/common"
import * as role from './role'
import * as battle from './battle'
import * as enums from './BattleEnums';

export class RoleBattleInfo {
    public roundAtkCount:number;
    public roundRemoteAtkCount : number;
    public roundBeAtkCount : number;
}

export class Team {
    private roleList : role.Role[]; //下标和站位不是对应的
    private battleData : common.UserBattleData;
    private roleBattleInfo : Map<number,RoleBattleInfo>;//根据
站位查找角色在当前回合的战斗相关数据

    public onKillRole : (r:common.Role) => void;

    public constructor(selfCamp: enums.Camp, 
battleData:common.UserBattleData, RoleList : common.Role[] ) {
        this.roleList = [];
        this.battleData = battleData;

        console.log("RoleList:", RoleList);

        let index = 0;
```

```
        for (let r of RoleList)
        {
            if(null!=r&&r.RoleID>100000)
            {
                let properties = new Map<enums.Property, 
number>();
                properties.set(enums.Property.HP, r.HP + 
r.TempHP);
                properties.set(enums.Property.TotalHP, r.HP + 
r.TempHP);
                properties.set(enums.Property.Attack, r.Attack 
+ r.TempAttack);
                let _role = new role.Role(r,index, r.RoleID, 
r.Level, r.Number, selfCamp, properties, 
r.FettersSkillID,r.equipID,
[...r.additionBuffer, ...r.TempAdditionBuffer]);
                _role.onKillRole = (_r:common.Role) => {
                    if (this.onKillRole) {
                        this.onKillRole.call(null, _r);
                    }
                };
                this.roleList.push(_role);
            }
            index++;
        }

        console.log("Team constructor this.roleList:", 
JSON.stringify(this.roleList));
    }

    public GetBattleData() {
        return this.battleData;
    }

    public GetRole(index:number) : role.Role {
        for (let r of this.roleList) {
            if (r.index == index) {
                return r;
            }
        }
        return null;
    }

    private AttackRoleIndex:number = -1;
    public SetAttackRole(index:number) {
        this.AttackRoleIndex = index;
    }
    public ReSetAttackRole() {
        this.AttackRoleIndex = -1;
    }
```

```
    public GetLasterRole() : role.Role {
        if (this.AttackRoleIndex > 0) {
            let r = this.GetRole(this.AttackRoleIndex);
            if (r) {
                return r;
            }
        }

        if(this.roleList[0])
        {
            return this.roleList[0];
        }
       else
        {
            console.warn("GetLasterRole 获取首位角色为空");
            return null;
        }
    }

    public GetRoleIndex(role: role.Role) : number {
        return role.index;
    }

    private removeRole(role: role.Role) {
        console.log(`removeRole id:${role.id} selfCamp:$
{role.selfCamp}`);
        let index = this.roleList.indexOf(role);
        if (index != -1) {
            console.log(`removeRole index:${index} id:$
{role.id} selfCamp:${role.selfCamp}`);
            this.roleList.splice(index, 1);
        }
    }

    public CheckRemoveDeadRole(battle: battle.Battle) {
        let removeRoles = [];
        for (let r of this.roleList) {
            try {
                if (r.CheckDead()) {
                    removeRoles.push(r);
                }
            }
            catch(error) {
                console.log("CheckRemoveDeadRole CheckDead:",
error);
                removeRoles.push(r);
            }
        }
```

```
        if (removeRoles.length > 0) {
            //console.log("CheckRemoveDeadRole
this.roleList:", JSON.stringify(this.roleList));
            //console.log("CheckRemoveDeadRole battle.evs:",
JSON.stringify(battle.evs));
            for (let r of removeRoles) {
                if (battle.CheckRoleTriggerSkill(r,
battle.evs)) {
                    console.log("CheckRemoveDeadRole not
remove r:", JSON.stringify(r));
                    continue;
                }
                this.removeRole(r);
                r.SendExitEvent(battle);
            }
            return true;
        }

        return false;
    }

    public CheckDefeated() : boolean {
        return this.roleList.length <= 0;
    }

    /*
     * 添加
     * 此函数返回场上所有角色
     * Editor: Hotaru
     * 2023/9/25
     */
    public GetRoles():role.Role[]
    {
        return this.roleList.slice();
    }
    /*
     * 添加
     * 向当前阵营队列添加一个角色
     * Editor: Guanliu
     * 2023/9/30
     */
    public AddRole(role:role.Role){
        this.roleList.splice(role.index,0,role);
    }
    /*
     * 添加
     * 获取场上当前阵营第一个空位置的下标
     * Editor: Guanliu
     * 2024/1/30
     */
```

```
    public GetVacancies():number{
        let flag:Number[]=[0,0,0,0,0,0];
        for(let i=0;i<this.roleList.length;i++){
            flag[this.roleList[i].index]=1;
        }
        for(let i=0;i<6;i++){
            if(flag[i]==0){
                console.log("场上存在空位")
                return i;
            }
        }

        console.log("场上不存在空位")
        //返回-1说明加入失败
        return -1;
    }

    /**
     * 判断前排是否为空
     * Editor:Hotaru
     * 2024/07/19添加
     */
    public CheckFront() : boolean
    {
        let flag = true;
        if(this.battleData.RoleList[0] ||
this.battleData.RoleList[1] || this.battleData.RoleList[2])
        {
            flag=false;
        }
        return flag;
    }

    /**
     * 换位
     * @param _begin 位置1
     * @param _end 位置2
     * Editor:Hotaru
     * 2024/08/19添加
     */
    public SwitchRole(_begin:number,_end:number)
    {
        console.log("switch role befor",this.roleList);
        let target_1 = null;
        let target_2 = null;
        let tempIndex;
        for (let i = 0; i < this.roleList.length; i++)
        {
            if (this.roleList[i].index == _begin)
            {
```

```
                target_1 = i;
            }
            if (this.roleList[i].index == _end)
            {
                target_2 = i;
            }
        }

        if (null!=target_2)
        {
            tempIndex = this.roleList[target_1].index;
            this.roleList[target_1].index =
this.roleList[target_2].index;
            this.roleList[target_2].index = tempIndex;
        }
        else
        {
            this.roleList[target_1].index = _end;
        }
        //换位完重新排序
        for (let i = 0; i < this.roleList.length; i++)
        {
            for (let j = i + 1; j < this.roleList.length; j++)
            {
                let t;
                if (this.roleList[j].index <
this.roleList[i].index)
                {
                    t = this.roleList[i];
                    this.roleList[i] = this.roleList[j];
                    this.roleList[j] = t;
                }
            }
        }
        console.log("switch role",this.roleList);

    }
}
/*
 * util.ts
 * author: qianqians
 * 2023/9/27
 */

export function random(min:number, max:number) : number {
    let r = Math.random();
    let n = max - min;
    return Math.floor(min + r*n);
}
```

```ts
export function isInteger(value: number): value is number {
    return Math.floor(value) === value;
  }

export function isDecimal(value: number): value is number {
    return !isInteger(value);
  }
```